

NGUYÊN LÝ VÀ TOÁN HỌC



TÓM TẮT

Phần A dựng đúng lượng nền cần thiết cho bốn phần sau, và nó được tổ chức theo *đường đi của một photon tới một con số*: camera biến điểm 3D thành pixel thế nào và méo ảnh phá hỏng gì, hình học phẳng cho ta công cụ gì, PnP biến tập góc thành pose ra sao, vì sao target phẳng luôn có hai nghiệm, một pixel nhiều biến thành bao nhiêu milimét, và cuối cùng là những hiệu ứng vật lý mà mô hình toán bỏ qua nhưng thực tế thì không. Nếu chỉ nhớ một điều: bốn chương đầu là công cụ, chương 5 là chương định giá — nó biến mọi lựa chọn thiết kế thành một con số milimét, và nó là chương mà cả Phần C dựa vào.

Phần này tồn tại để phục vụ Phần C, và nguyên tắc chi phối nó là *học theo nhu cầu của ngân sách*: mỗi chương chỉ đi tới mức đủ để ta biết khâu đó đóng góp bao nhiêu vào sai số cuối cùng, và đủ để ta tự dẫn lại được quan hệ tỉ lệ khi cần. Đây là lựa chọn có chủ ý và nó khác với một giáo trình thị giác máy tính. Một giáo trình sẽ dành nhiều trang cho việc phân rã homography thành pose vì đó là một kết quả đẹp; ở đây kết quả ấy được trình bày ngắn, vì trong thực hành nó chỉ dùng làm khởi tạo cho một phép tinh chỉnh phi tuyến, và chính phép tinh chỉnh mới quyết định sai số. Ngược lại, quan hệ $\delta Z \approx Z\sigma/w$ — một dòng trong hầu hết sách — được dành cả một mục ở chương 5, vì nó là dòng quyết định bạn có đạt 5 mm hay không.

Sáu chương có vai trò rất khác nhau, và mức đầu tư nên khác nhau theo. Chương 1 là mô hình camera: nó tồn tại chủ yếu để giải thích vì sao méo ảnh chưa khử sạch làm *cong* cạnh tag và vì thế làm sai chỗ giao của hai đường thẳng, và vì sao principal point sai gây ra một thiên lệch xoay có hệ thống — nguồn lỗi xoay lớn nhất trong thực tế mà hầu như không ai ngờ tới. Chương 2 cho hình học phẳng, và nó chứa một sự thật nhỏ mà hậu quả lớn: tâm của hình chiếu không bằng hình chiếu của tâm. Chương 3 là PnP, tức cỗ máy biến góc thành pose. Chương 4 là chương bệnh lý: với target phẳng luôn tồn tại đúng hai nghiệm, và đây là nguyên nhân số một của hiện tượng pose nhảy. Chương 5 là chương định giá. Chương 6 gom những hiệu ứng vật lý mà năm chương trước giả định là không có.

Thứ tự đọc đề xuất là tuần tự, với hai lưu ý. Chương 2 có thể đọc lướt nếu bạn đã quen hình học đa khung nhìn — chỉ cần dừng kỹ ở mục về cross-ratio và bias tâm ellipse. Chương 5 thì ngược lại: nên đọc kỹ ngay cả khi chưa hiểu hết chi tiết Jacobian, vì bốn quan hệ tỉ lệ trong đó là thứ bạn sẽ dùng hằng ngày, và chúng đủ đơn giản để thuộc lòng.

Bảng 1. Bản đồ Phần A: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
1	01-mo-hinh-camera-va-d:	Pinhole, ma trận K , tọa độ chuẩn hoá; méo radial/tangential/rational/KB; vì sao méo làm cong cạnh tag; sai số mô hình so với sai số tham số; principal point sai gây thiên lệch xoay	● ● ●
2	02-homography-va-hinh-t	$H = K[r_1 \ r_2 \ t]$ và phân rã; điều kiện suy biến và số điều kiện theo diện tích tag; cross-ratio; vì sao tâm chiếu $\neq$ chiếu của tâm	● ●
3	03-pnp-va-uoc-luong-tu	P3P và bốn nghiệm; EPnP, DLT, IPPE cho target phẳng; tinh chỉnh bằng sai số tái chiếu; PnP hợp nhất trên nhiều tag; PnP có trọng số	● ● ●
4	04-pose-ambiguity-targ	Hai nghiệm của target phẳng; khi nào tệ nhất; biểu hiện nhảy/flicker; vì sao không đồng phẳng khử được triệt để	● ● ●
5	05-lan-truyen-sai-so-p	Jacobian và $(J^T \Sigma^{-1} J)^{-1}$; bốn quan hệ tỉ lệ phải thuộc; vì sao xoay từ tag phẳng gần như không quan sát được; cải thiện tuyến tính theo baseline	● ● ●
6	06-hieu-ung-vat-ly-va-c	Rolling shutter, motion blur, lấy mẫu và răng cưa, độ sâu trường ảnh và MTF, bias tâm ellipse, đặc trưng quang học của vật liệu	● ●

Chương 5 là chương có giá trị cao nhất của cả phần và nên đọc kỹ nhất; bốn quan hệ tỉ lệ trong đó đủ gần để thuộc lòng và chúng là công cụ chẩn đoán hàng ngày. Chương 1 và 3 là nền bắt buộc. Chương 4 gần nhưng đừng bỏ: nó giải thích hiện tượng mà gần như ai làm marker cũng gặp và ít người gọi đúng tên. Chương 2 đọc lướt được nếu đã quen hình học đa khung nhìn, trừ mục cross-ratio. Chương 6 đọc một lượt rồi quay lại khi gặp triệu chứng cụ thể.

Mô hình camera và méo ảnh (Camera Model and Distortion)

PHỤ THUỘC Không có — đây là chương mở đầu của cây. Nó chỉ giả định bạn biết tọa độ thuần nhất và phép nhân ma trận. Mọi chương còn lại của Phần A đều gọi về đây, và A/05 lấy từ đây đại lượng f mà mọi công thức sai số của nó đều chia cho.

TÓM TẮT

Chương này dựng mô hình biến một điểm trong không gian thành một pixel, và nó tồn tại vì hai lý do rất cụ thể chứ không phải vì tính đầy đủ. Lý do thứ nhất: nó cấp *tiêu cự tính bằng pixel* f , đại lượng camera duy nhất xuất hiện trong mọi công thức sai số của cây. Lý do thứ hai, và đây mới là lý do chính: nó giải thích hai cơ chế hỏng mà người làm marker gặp thường xuyên và hầu như không bao giờ chẩn đoán đúng. Cơ chế một — méo ảnh chưa khử làm *cong* cạnh tag, mà detector thì khớp *đường thẳng* lên cạnh, nên giao điểm hai đường thẳng sai, nên góc sai, nên pose sai. Cơ chế hai — điểm chính (principal point) sai gây ra một thiên lệch xoay *có hệ thống* phụ thuộc vị trí tag trên ảnh. Nếu chỉ nhớ một điều: *méo ảnh không gây nhiễu, nó gây thiên lệch* — và thiên lệch thì không trung bình mất đi dù lấy bao nhiêu khung.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tọa độ thuần nhất (homogeneous coordinates)** — cách viết một điểm 2D bằng ba số (x, y, w) với quy ước $(x, y, w) \sim (\lambda x, \lambda y, \lambda w)$ cho mọi $\lambda \neq 0$; điểm Euclid tương ứng là $(x/w, y/w)$. Nó tồn tại để phép chiếu phối cảnh — vốn là một phép chia — viết được thành một phép nhân ma trận.
- **tọa độ chuẩn hoá (normalized coordinates)** — tọa độ của điểm sau khi chia cho chiều sâu nhưng *trước* khi nhân với K : $(x_n, y_n) = (X/Z, Y/Z)$. Đây là hệ tọa độ mà mọi mô hình méo được định nghĩa trên đó, và nhầm lẫn giữa nó với tọa độ pixel là lỗi cài đặt phổ biến nhất khi tự viết code undistort.
- **tiêu cự tính bằng pixel** — $f_x = F/s_x$ với F là tiêu cự vật lý của ống kính và s_x là kích thước một pixel. Nó *không* phải một chiều dài; nó là một tỉ số không thứ nguyên, và đó là lý do nó xuất hiện được ở mẫu số của các công thức sai số.
- **điểm chính (principal point)** — giao điểm của trục quang với mặt phẳng ảnh, ký hiệu (c_x, c_y) . Trục giác nói nó nằm ở giữa ảnh; thực tế nó lệch vài pixel tới vài chục pixel do sai lệch lắp ráp, và mục 5 giải thích vì sao sai lệch ấy nguy hiểm hơn vẻ ngoài.
- **méo xuyên tâm (radial distortion)** — sai lệch mà độ lớn chỉ phụ thuộc khoảng cách từ điểm chính, theo hướng ra xa hoặc vào gần tâm. Đây là thành phần méo lớn nhất trong hầu hết ống kính.
- **méo tiếp tuyến (tangential distortion)** — sai lệch theo hướng vuông góc với bán kính, sinh ra khi mặt phẳng cảm biến không song song hoàn hảo với mặt phẳng thấu kính. Thường nhỏ hơn méo xuyên tâm một bậc, nhưng nó là thành phần *bất đối xứng* nên nó không tự triệt tiêu.
- **sai số mô hình và sai số tham số** — sai số tham số là khi bạn dùng đúng dạng hàm nhưng ước lượng sai hệ số; sai số mô hình là khi dạng hàm bản thân nó không đủ để mô tả hiện thực. Loại thứ hai nguy hiểm hơn vì thêm dữ liệu không chứa được nó, và mục 6 dành riêng cho việc phân biệt hai loại.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn hiệu chuẩn camera, sai số tái chiếu trung bình là 0,25 px — một con số nghe rất tốt. Vì sao con số ấy có thể đang che giấu một lỗi mô hình nghiêm trọng, và bạn nhìn vào đâu để phát hiện?
- Q2. Detector AprilTag khớp một đường *thẳng* lên mỗi cạnh tag rồi lấy giao điểm làm góc. Nếu ảnh còn méo 3 px ở rìa, sai số góc mà bạn nhận được là bao nhiêu — và vì sao câu trả lời không phải “3 px”?
- Q3. Điểm chính của bạn bị sai 10 px so với giá trị thật. Sai số ấy gây ra hậu quả gì cho góc của pose, và vì sao hậu quả ấy thay đổi khi tag di chuyển trong khung hình?
- Q4. Có hai cách xử lý méo: khử méo toàn ảnh rồi chạy detector, hoặc chạy detector trên ảnh gốc rồi khử méo cho từng tọa độ góc. Cách nào chính xác hơn, và cách nào nhanh hơn — và vì sao câu trả lời cho hai câu ấy không giống nhau?
- Q5. Bạn dùng mô hình hai hệ số k_1, k_2 cho một ống kính góc rộng 120° . Hiệu chuẩn hội tụ, residual nhỏ. Vì sao điều đó *không* chứng minh mô hình của bạn đủ, và dấu hiệu nào sẽ lộ ra sự thiếu?

1.1 Đặt vấn đề

Mọi thứ trong cây này đều bắt đầu từ một phép biến đổi: một điểm ở đâu đó trong không gian trở thành một tọa độ pixel trên một tấm ảnh. Nếu phép biến đổi ấy được mô tả đúng, mọi công thức phía sau có nghĩa. Nếu nó được mô tả sai — kể cả sai một chút — thì mọi thứ phía sau là chính xác quanh một giá trị sai.

Bài toán gốc thuộc về ngành trắc lượng ảnh (photogrammetry) chứ không phải thị giác máy tính, và nó đã có từ trước khi có máy tính: người ta cần đo đặc địa hình từ ảnh chụp trên không, và để làm được thì phải biết chính xác ống kính đã biến dạng ảnh như thế nào. Mô hình méo vẫn đang dùng ngày nay mang tên Brown–Conrady và được công bố ở dạng hiện đại năm 1971 [1]; phương pháp hiệu chuẩn từ một target phẳng mà mọi hộp công cụ đang dùng thì đến sau đó gần ba mươi năm [2].

Điều làm chương này đáng đọc với người làm marker — thay vì chỉ tra một công thức khi cần — nằm ở một quan sát mà tôi cho là chưa được nói đủ rõ ở đâu cả. Méo ảnh ảnh hưởng tới các bài toán thị giác theo những cách rất khác nhau, và nó ảnh hưởng tới *phát hiện marker* theo một cách đặc biệt xấu. Lý do là thế này: hầu hết thuật toán thị giác dùng đặc trưng điểm, và một điểm bị méo vẫn là một điểm — nó chỉ nằm sai chỗ, và khử méo là đủ. Nhưng detector marker không làm việc với điểm; nó làm việc với *cạnh thẳng*. AprilTag khớp một đường thẳng lên mỗi cạnh của tứ giác rồi lấy giao của bốn đường thẳng ấy làm bốn góc [3]. Và phép méo xuyên tâm biến một đường thẳng thành một *đường cong*. Khớp một đường thẳng lên một đường cong thì được một đường thẳng sai, và giao của hai đường thẳng sai thì sai nhiều hơn cả hai — mục 4 định lượng chuyện này.

Cách hiển nhiên để xử lý là khử méo toàn ảnh trước khi chạy detector, và cách ấy đúng. Chỗ nó không đủ nằm ở hai điểm mà mục 6 và mục 7 bàn tới: khử méo chỉ tốt đúng bằng mô hình méo bạn đã ước lượng, và nếu mô hình *sai dạng* — chẳng hạn dùng hai hệ số cho một ống kính góc rộng — thì phần méo còn sót lại không phải nhiễu ngẫu nhiên mà là một mẫu hình có cấu trúc, lớn nhất ở đúng chỗ tệ nhất là rìa ảnh.

Còn một lý do thứ hai để chương này không phải một phụ lục, và nó là lý do làm tôi xếp chương này ở mức xương sống thay vì mức tham khảo. Trong tất cả các tham số nội của camera, có một tham số mà sai lệch của nó gây hậu quả không tương xứng với vẻ ngoài của nó: điểm chính. Sai điểm chính không làm ảnh trông khác đi chút nào — không ai nhìn ảnh mà phát hiện được — nhưng nó tạo ra một thiên lệch trong *hướng* của pose, và thiên lệch ấy thay đổi theo vị trí tag trong khung hình nên nó trông hết như một sai số phụ thuộc tư thế. Mục 5 dành riêng cho nó.

TRỰC GIÁC

Hình dung camera như một cái thấu.

Đỉnh thấu là tâm quang học — một điểm. Mọi tia sáng đi tới ảnh đều phải chui qua đúng điểm ấy. Đáy thấu là mặt phẳng ảnh, đặt cách đỉnh một khoảng f . Một điểm trong không gian được chiếu lên ảnh bằng cách kẻ một đường thẳng từ nó tới đỉnh thấu và xem đường ấy cắt mặt phẳng ảnh ở đâu.

Mô hình pinhole nói rằng thấu là hoàn hảo: mọi tia đi thẳng, đỉnh thấu là một điểm toán học, mặt phẳng ảnh vuông góc với trục. Ống kính thật không phải thế. Thấu kính bẻ tia sáng, và nó bẻ các tia ở rìa nhiều hơn các tia ở giữa — đó là méo xuyên tâm. Cảm biến không lắp vuông góc tuyệt đối — đó là méo tiếp tuyến. Trục quang không đi qua đúng giữa cảm biến — đó là điểm chính lệch.

Ba sai lệch ấy khác nhau về bản chất theo một cách quan trọng. Hai cái đầu làm ảnh *trông* khác: đường thẳng cong đi, và mắt người phát hiện được nếu chụp một cái cửa sổ. Cái thứ ba thì không: dịch cả ảnh sang trái ba pixel thì không ai nhìn ra, vì không có gì trong ảnh để so sánh. Đó là lý do cái thứ ba nguy hiểm nhất — nó là sai lệch duy nhất không có triệu chứng thị giác.

1.2 Mô hình pinhole và ma trận K

Dựng mô hình theo ba bước, mỗi bước một phép biến đổi, vì việc tách bạch ba bước ấy là điều kiện để biết mình đang làm việc trong hệ toạ độ nào.

Bước một, từ thế giới sang camera. Một điểm $\mathbf{X}_w$ trong hệ thế giới được đưa sang hệ camera bằng một phép biến đổi cứng:

$$\mathbf{X}_c = R\mathbf{X}_w + \mathbf{t},$$

với $R \in SO(3)$ và $\mathbf{t} \in \mathbb{R}^3$. Cặp $(R, \mathbf{t})$ chính là *pose* mà toàn bộ cây này tìm cách ước lượng; sáu tham số của nó là sáu ẩn của bài toán PnP ở A/03.

Bước hai, chiếu phối cảnh. Chia cho chiều sâu:

$$\begin{pmatrix} x_n \\ y_n \end{pmatrix} = \begin{pmatrix} X_c/Z_c \\ Y_c/Z_c \end{pmatrix}.$$

Đây là toạ độ chuẩn hoá. Chúng không thứ nguyên, chúng là tang của góc nhìn theo hai hướng, và — quan trọng nhất cho mục đích của ta — *mọi mô hình méo được định nghĩa trên hệ toạ độ này*, không phải trên pixel. Đây là bước duy nhất trong ba bước mang tính phi tuyến, và toàn bộ độ khó của bài toán pose nằm ở phép chia này.

Bước ba, từ toạ độ chuẩn hoá sang pixel. Nhân với ma trận nội:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x_d \\ y_d \\ 1 \end{pmatrix}, \quad K = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

với (x_d, y_d) là toạ độ chuẩn hoá đã méo (mục 3). Bốn tham số thật sự quan trọng là f_x, f_y, c_x, c_y ; hệ số trượt s mô tả trường hợp trục pixel không vuông góc và với cảm biến số hiện đại thì nó bằng không tới mức không đo được — đừng ước lượng nó trừ khi có lý do cụ thể, vì một tham số dư luôn hút một phần sai số của các tham số khác.

Hai điều đáng dừng lại về f_x và f_y . Thứ nhất, chúng là *tỉ số không thứ nguyên*, không phải chiều dài — $f_x = F/s_x$ với F là tiêu cự vật lý và s_x là bề rộng một pixel. Đây là lý do f đứng được ở

mẫu số trong $\delta x \approx Z\sigma/f$ mà vẫn ra đúng thứ nguyên chiều dài. Thứ hai, có một cách rất nhanh để ước lượng f mà không cần hiệu chuẩn, dùng khi thiết kế: với trường nhìn ngang θ_{FOV} và độ phân giải ngang N_x pixel,

$$f_x \approx \frac{N_x/2}{\tan(\theta_{\text{FOV}}/2)}.$$

Ví dụ: camera 1280 px ngang, FOV 60°. Ta có $f_x \approx 640/\tan 30^\circ = 640/0,577 = 1109$ px. Con số ấy đủ chính xác để lập ngân sách sai số ở C/15 trước khi có camera trong tay — và đó chính là cách dùng nó.

1.3 Các mô hình méo

Méo được mô hình hoá như một phép biến đổi phi tuyến *trên toạ độ chuẩn hoá*, chen vào giữa bước hai và bước ba: $(x_n, y_n) \mapsto (x_d, y_d)$.

A. Brown–Conrady: radial cộng tangential

Mô hình phổ biến nhất, và là mặc định của OpenCV. Đặt $r^2 = x_n^2 + y_n^2$:

$$\begin{aligned} x_d &= x_n \underbrace{(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)}_{\text{xuên tâm}} + \underbrace{2p_1 x_n y_n + p_2 (r^2 + 2x_n^2)}_{\text{tiếp tuyến}}, \\ y_d &= y_n (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y_n^2) + 2p_2 x_n y_n. \end{aligned} \quad (1.1)$$

Đây là dạng chuẩn [1], viết theo quy ước của [2].

Ba điều cần đọc ra. Thứ nhất, thành phần xuyên tâm chỉ chứa *luỹ thừa chẵn* của r — đó không phải tùy tiện mà là hệ quả của tính đối xứng quay của thấu kính: nếu phép méo đối xứng quanh trục thì nó phải là hàm chẵn. Thứ hai, thành phần xuyên tâm *giữ nguyên hướng* của điểm so với tâm và chỉ đổi bán kính; hệ quả là một đường thẳng đi *qua* điểm chính vẫn thẳng sau khi méo, còn mọi đường thẳng khác thì cong. Ghi nhớ điều này — nó là chìa khoá của mục 4. Thứ ba, $k_1 < 0$ cho méo thùng (barrel, điển hình của ống kính góc rộng: các điểm ở rìa bị kéo vào trong) và $k_1 > 0$ cho méo gối (pincushion).

B. Mô hình hữu tỉ (rational)

Với ống kính góc rất rộng, đa thức bậc sáu không đủ. Mô hình hữu tỉ thay đa thức bằng một phân thức:

$$x_d = x_n \cdot \frac{1 + k_1 r^2 + k_2 r^4 + k_3 r^6}{1 + k_4 r^2 + k_5 r^4 + k_6 r^6} + (\text{tangential}).$$

Nó linh hoạt hơn nhiều nhưng nó có nhiều tham số hơn, và nhiều tham số hơn nghĩa là dễ khớp quá mức (overfitting) nếu dữ liệu hiệu chuẩn không phủ kín trường nhìn — một cạm bẫy mà B/12 bàn kỹ.

C. Mô hình fisheye Kannala–Brandt (KB) và các mô hình góc

Với ống kính trên khoảng 120°, toàn bộ họ mô hình trên gặp một vấn đề cấu trúc chứ không phải vấn đề độ chính xác: phép chiếu pinhole $r_n = \tan \theta$ phân kỳ khi $\theta \rightarrow 90^\circ$, nên không đa thức nào sửa được. Mô hình KB bỏ hẳn pinhole và mô hình hoá trực tiếp quan hệ giữa *góc tới* θ và bán kính ảnh [4]:

$$r_d = \theta + k_1 \theta^3 + k_2 \theta^5 + k_3 \theta^7 + k_4 \theta^9.$$

Đây là mô hình mặc định cho fisheye trong hầu hết hộp công cụ hiện đại, và có các mô hình mới hơn với ít tham số hơn cho cùng độ chính xác, chẳng hạn Double Sphere [5].

Bảng 1.1. Bốn họ mô hình méo, phạm vi áp dụng, và chế độ hỏng.

Mô hình	Số tham số	Dùng cho	Hỏng khi
Radial 2 hệ số (k_1, k_2)	2	Ống kính hẹp, dưới $\sim 50^\circ$	FOV rộng: residual có cấu trúc ở rìa, lớn dần theo r
Brown–Conrady đầy đủ	5	Phổ thông, $50\text{--}90^\circ$	Fisheye: dạng hàm không đủ dù thêm bao nhiêu ảnh
Hữu tỉ (rational)	8	Góc rộng $90\text{--}120^\circ$	Dữ liệu hiệu chuẩn không phủ rìa: khớp quá mức, ngoại suy sai
KB / fisheye	4	Trên 120° , tới $180^\circ+$	Dùng nhằm cho ống kính hẹp: tham số hoá kém điều kiện

1.4 Vì sao méo phá hoại detector marker một cách đặc biệt

Đây là mục quan trọng nhất của chương, và nó là chỗ chương này khác một chương về mô hình camera trong sách giáo khoa.

A. Cơ chế: khớp thẳng lên đường cong

AprilTag định vị góc theo một quy trình rất cụ thể: sau khi tìm được tứ giác ứng viên, nó khớp một *đường thẳng* lên mỗi cạnh, dùng trọng số theo độ lớn gradient, rồi lấy giao của các cặp đường thẳng kề nhau làm bốn góc [3]. Cách làm ấy chính xác hơn nhiều so với việc lấy trực tiếp điểm cực trị của contour, vì nó dùng *toàn bộ* chiều dài cạnh — hàng chục tới hàng trăm pixel — để xác định một đường, thay vì dựa vào vài pixel quanh góc.

Nhưng nó có một giả định ngầm: *cạnh của tag là một đường thẳng trên ảnh*. Giả định ấy đúng với mô hình pinhole (ảnh phối cảnh của một đường thẳng là một đường thẳng) và *sai* khi ảnh còn méo. Như mục 3A đã chỉ ra, méo xuyên tâm giữ nguyên hướng của điểm so với tâm nhưng đổi bán kính theo một hàm phi tuyến, nên một đường thẳng không đi qua điểm chính trở thành một đường cong.

Hậu quả có ba tầng, và cả ba đều đi theo chiều xấu.

Tầng một, đường thẳng khớp bị lệch. Khớp bình phương tối thiểu một đường thẳng lên một cung cong cho ra một đường thẳng đi qua giữa cung — nó cắt cung ở hai chỗ và lệch khỏi cung ở hai đầu và ở giữa, theo hướng ngược nhau.

Tầng hai, và đây là tầng quyết định: sai số được khuếch đại tại giao điểm. Hai đường thẳng gần vuông góc giao nhau thì sai số góc nhỏ của mỗi đường chuyển thành sai số vị trí ở giao điểm theo một hệ số phụ thuộc góc giao. Với hai cạnh của một tag nhìn chếch, góc giao có thể khá nhỏ, và khi đó hệ số khuếch đại lớn. Đây là câu trả lời cho Q2: sai lệch của góc *không* bằng sai lệch của cạnh — nó có thể lớn hơn nhiều.

Tầng ba, sai số là thiên lệch chứ không phải nhiễu. Với một tag ở một vị trí cố định trên ảnh, méo luôn đẩy cạnh theo cùng một hướng, mỗi khung hình như nhau. Lấy trung bình một nghìn khung không giúp gì — đây chính là loại nguồn “hệ thống” ở A/05 mục 8, loại không chia cho $\sqrt{N}$.

B. Ví dụ số: méo lớn cỡ nào ở rìa ảnh

Để thấy bậc độ lớn, lấy một ống kính góc rộng phổ thông. Giả sử $f = 600$ px, ảnh 1280×960 , và $k_1 = -0,30$ (giá trị điển hình cho góc rộng; *không phải số đo của một ống kính cụ thể*). Xét một điểm ở rìa ngang, cách tâm $r_{\text{px}} = 400$ px.

Toạ độ chuẩn hoá: $r_n = 400/600 = 0,667$, nên $r_n^2 = 0,444$.

Hệ số méo xuyên tâm: $1 + k_1 r_n^2 = 1 - 0,30 \times 0,444 = 1 - 0,133 = 0,867$.

Điểm bị kéo vào trong: bán kính méo là $0,667 \times 0,867 = 0,578$ trong toạ độ chuẩn hoá, tức 347 px.
Độ lệch: $400 - 347 = 53$ px.

Năm mươi ba pixel. Đặt cạnh nhiều định vị góc $\sigma = 0,2$ px, méo lớn hơn nhiều hơn 250 lần. Đây là lý do khử méo không phải một bước tinh chỉnh mà là một bước bắt buộc: bỏ qua nó thì mọi thứ khác trong cây là vô nghĩa.

Con số quan trọng hơn cho mục đích của ta là *méo còn sót lại sau khi khử*. Nếu mô hình đúng dạng và hiệu chuẩn tốt, phần sót cỡ 0,1–0,3 px — cùng bậc với σ , chấp nhận được. Nếu mô hình *sai dạng* — hai hệ số cho ống kính góc rộng — phần sót có thể vẫn là vài pixel ở rìa, và vì nó có cấu trúc chứ không ngẫu nhiên nên nó thành một thiên lệch phụ thuộc vị trí. Mục 6 bàn cách phát hiện.

CẠM BẮY

Có một lỗi cài đặt cực kỳ phổ biến khi tự viết code khử méo, và nó gây ra triệu chứng rất khó truy: *áp công thức méo lên toạ độ pixel thay vì lên toạ độ chuẩn hoá*.

Công thức (1.1) định nghĩa trên (x_n, y_n) , tức sau khi chia cho Z nhưng *trước khi nhân K* . Nếu bạn tính $r^2 = u^2 + v^2$ bằng pixel thay vì $r^2 = x_n^2 + y_n^2$, kết quả sai theo hệ số f^2 — tức sai hàng trăm nghìn lần ở số hạng $k_1 r^2$.

Triệu chứng: hiệu chuẩn vẫn “hội tụ” vì bộ tối ưu sẽ tìm ra một k_1 nhỏ bù lại tỉ lệ, và residual trông không tệ. Nhưng các hệ số ra sẽ khác vài bậc độ lớn so với giá trị điển hình, và mô hình sẽ ngoại suy sai kinh khủng ở những vùng ảnh không có dữ liệu hiệu chuẩn.

Phép kiểm rẻ: hệ số k_1 của ống kính thường nằm trong khoảng -1 tới $+1$. Nếu bạn thấy $k_1 \sim 10^{-7}$ thì gần như chắc chắn bạn đang tính r bằng pixel.

1.5 Điểm chính sai: nguồn thiên lệch xoay không có triệu chứng

Mục này bàn một cơ chế mà tôi cho là bị đánh giá thấp nhất trong toàn bộ chuỗi đo, và nó xứng đáng được nhìn kỹ.

Điểm chính (c_x, c_y) là giao của trục quang với mặt phẳng ảnh. Trục giác nói nó ở giữa ảnh, và hầu hết người ta khởi tạo nó bằng $(N_x/2, N_y/2)$. Thực tế nó lệch: sai lệch lắp ráp cảm biến so với ống kính cỡ vài chục micromet là bình thường, và với pixel 3 μm thì vài chục micromet là *cả chục pixel*.

Vì sao sai lệch ấy nguy hiểm? Hãy xem nó làm gì với phép chiếu. Giả sử giá trị thật là c_x nhưng ta dùng $\hat{c}_x = c_x + \Delta$. Toạ độ chuẩn hoá mà ta tính ra từ một pixel đo được u là

$$\hat{x}_n = \frac{u - \hat{c}_x}{f} = \frac{u - c_x}{f} - \frac{\Delta}{f} = x_n - \frac{\Delta}{f}.$$

Nghĩa là: *mọi tia nhìn đều bị dịch đi cùng một lượng góc Δ/f radian*. Đây không phải nhiều; đây là một phép quay hệ thống của toàn bộ chùm tia.

Với $\Delta = 10$ px và $f = 600$ px: $\Delta/f = 0,0167$ rad = $0,95^\circ$. Gần một độ — và ngân sách góc của ta là $0,5^\circ$.

Bây giờ đến phần tinh tế, và nó là phần giải thích vì sao triệu chứng khó truy. Nếu sai lệch ấy *chỉ* là một phép quay hệ thống của cả camera, thì nó vô hại trong bài toán docking theo teach-by-showing: nó xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu (A/05 mục 8). Nhưng nó không chỉ là một phép quay, vì x_n vào công thức chiếu *cùng với* phép chia cho Z . Hậu quả là hiệu

ứng của Δ lên pose ước lượng phụ thuộc vào *tag đang nằm ở đâu trong khung hình và ở khoảng cách nào* — một tag ở giữa ảnh chịu ảnh hưởng khác một tag ở rìa.

Triệu chứng thực tế theo đó: pose sai một lượng thay đổi trơn theo tư thế, không nhảy, không nhiễu, và không có gì trong ảnh trông bất thường. Nó trông hết như một sai số hand-eye, hoặc như một constellation sai — và đó là lý do người ta hay đi tìm nhầm chỗ.

Ghi chú trung thực về mức kiểm chứng. Tôi dẫn cơ chế này bằng cách viết lại phép chiếu (cửa a) và tôi tin nó đúng về cấu trúc. Tôi *chưa* chạy mô phỏng để đo biên độ thiên lệch pose theo Δ (cửa b), và tôi *chưa* tìm được một nguồn phát biểu tường minh đúng dạng này để đối chiếu (cửa c). Vì vậy phần định lượng của mục này nên được coi là ước lượng bậc độ lớn của tôi chứ không phải kết quả đã kiểm. Đây là mục đứng đầu danh sách nợ kiểm chứng của chương trong 99-chua-biet.md.

CẦN NHỚ

Trong bốn tham số nội, f_x và f_y được ước lượng rất tốt bởi mọi quy trình hiệu chuẩn, còn c_x và c_y thì *không*: chúng tương quan mạnh với tư thế của target, và nếu dữ liệu hiệu chuẩn chỉ gồm các ảnh chụp gần vuông góc thì bộ tối ưu không tách được “điểm chính lệch sang trái” khỏi “target nghiêng sang phải”.

Hệ quả thực hành trực tiếp, và nó sẽ được nhắc lại ở B/12: dữ liệu hiệu chuẩn *phải* gồm các ảnh ở tư thế nghiêng mạnh và target phải phủ tới sát bốn góc khung hình. Một bộ ảnh hiệu chuẩn “đẹp” — target luôn ở giữa, luôn vuông góc — cho residual nhỏ và điểm chính sai.

Đây là một trường hợp cụ thể của nguyên tắc chung: residual nhỏ không chứng minh tham số đúng; nó chỉ chứng minh mô hình khớp được dữ liệu bạn đã cho.

1.6 Sai số mô hình so với sai số tham số

Hai loại sai lệch rất khác nhau thường bị gộp làm một, và phân biệt chúng là kỹ năng chẩn đoán quan trọng nhất của chương.

Sai số tham số (*parameter error*) là khi dạng hàm đúng nhưng hệ số ước lượng chưa chính xác — chẳng hạn mô hình Brown–Conrady đúng cho ống kính của bạn nhưng k_1 lệch 2% vì dữ liệu hiệu chuẩn ít. Loại này *chữa được bằng thêm dữ liệu*: chụp thêm ảnh, phủ rộng hơn, và hệ số hội tụ về giá trị đúng.

Sai số mô hình (*model error*) là khi dạng hàm bản thân nó không đủ để mô tả ống kính — chẳng hạn dùng hai hệ số radial cho một ống kính 120°. Loại này *không* chữa được bằng thêm dữ liệu. Bộ tối ưu sẽ tìm ra bộ hệ số tốt nhất trong họ hàm bạn đã chọn, và nếu họ hàm ấy không chứa hàm đúng thì “tốt nhất” vẫn sai. Thêm một nghìn ảnh cho ra cùng một kết quả sai, chỉ với khoảng tin cậy hẹp hơn — một tình huống đặc biệt nguy hiểm vì nó làm bạn tin hơn vào một con số sai.

Phép thử phân biệt hai loại rất đơn giản và nó là kỹ thuật chẩn đoán giá trị nhất trong chương: *vẽ sai số tái chiếu theo vị trí trên ảnh*.

Nếu chỉ có sai số tham số, các vectơ residual trông như nhiễu trắng — hướng ngẫu nhiên, độ lớn đều, không có cấu trúc không gian. Nếu có sai số mô hình, chúng tạo thành một *mẫu hình*: thường là các vectơ hướng xuyên tâm với độ lớn tăng dần theo bán kính, hoặc các xoáy có tổ chức ở bốn góc ảnh. Kỹ thuật này đã được dùng trong hiệu chuẩn camera chính xác từ lâu [6] và nó rõ đến mức không có lý do gì để bỏ qua.

CẠM BẮY

Sai số tái chiếu trung bình (RMS reprojection error) là con số mà mọi công cụ hiệu chuẩn in ra, và nó là con số *tệ nhất* để đánh giá chất lượng hiệu chuẩn khi dùng một mình. Ba lý do.

Thứ nhất, nó là một trung bình trên toàn ảnh, nên một vùng rìa sai 2 px chìm mất trong hàng nghìn điểm ở giữa sai 0,1 px.

Thứ hai, nó *giảm* khi bạn thêm tham số, kể cả tham số vô nghĩa — đó là bản chất của khớp bình phương tối thiểu. Một mô hình hữu tỉ 8 tham số luôn cho RMS nhỏ hơn Brown–Conrady 5 tham số trên cùng dữ liệu, và điều đó không nói gì về việc mô hình nào đúng hơn.

Thứ ba, và nghiêm trọng nhất: nó đo độ khớp trên *dữ liệu hiệu chuẩn*, không đo độ đúng khi ngoại suy sang vùng không có dữ liệu. Nếu target của bạn không bao giờ chạm tới góc khung hình thì RMS không biết gì về góc khung hình, và tag của robot lúc tiếp cận thì rất có thể nằm đúng ở đó.

Thay vào đó hãy nhìn: bản đồ residual theo vị trí (có cấu trúc hay không), độ phủ của dữ liệu hiệu chuẩn (có chạm bốn góc không), và độ ổn định của tham số qua nhiều lần hiệu chuẩn độc lập.

1.7 Khử méo: toàn ảnh hay từng điểm

Hai cách, và chúng không tương đương — đây là câu trả lời cho Q4.

Cách một, khử méo toàn ảnh rồi chạy detector. Tạo một ảnh mới trong đó mọi pixel đã được đặt lại đúng vị trí pinhole, rồi chạy detector trên ảnh ấy. Ưu điểm: mọi giả định của detector về đường thẳng trở thành đúng, nên khớp cạnh hợp lệ. Nhược điểm: phép khử méo đòi *nội suy* giá trị pixel ở các vị trí không nguyên, và nội suy làm nhoè ảnh — tức làm tăng σ . Với nội suy song tuyến tính (bilinear), mức nhoè đáng kể; với nội suy bậc cao hơn thì đỡ hơn nhưng chậm hơn.

Cách hai, chạy detector trên ảnh gốc rồi khử méo cho bốn toạ độ góc. Ưu điểm: không nội suy, nên không mất độ sắc nét, nên σ giữ nguyên; và rẻ hơn nhiều vì chỉ xử lý bốn điểm thay vì một triệu. Nhược điểm: detector đang khớp đường thẳng lên cạnh cong, nên bốn góc nó trả về đã sai *trước khi* bạn khử méo — và khử méo cho một điểm sai chỉ cho ra một điểm sai ở chỗ khác.

Vậy cách nào đúng? Câu trả lời phụ thuộc vào độ lớn của méo trong vùng tag chiếm, và nó dẫn tới một quy tắc thực hành khá rõ. Nếu tag nhỏ và nằm ở vùng ảnh có méo gần tuyến tính cục bộ — tức gần tâm, hoặc ống kính ít méo — thì cách hai tốt hơn, vì phần cong trên chiều dài một cạnh tag là không đáng kể còn lợi ích về độ sắc nét thì thật. Nếu tag lớn hoặc nằm ở rìa ảnh của một ống kính góc rộng, cách một bắt buộc.

Có một cách thứ ba, tốt hơn cả hai nhưng ít được dùng: *đưa mô hình méo vào ngay trong hàm sai số tái chiếu* của bước tinh chỉnh PnP. Tức là không khử méo gì cả; thay vào đó, khi dự đoán vị trí của một góc, ta chiếu điểm 3D qua pinhole rồi *áp méo*, và so với toạ độ đo được trên ảnh gốc. Cách này đúng về mặt mô hình và nó không mất độ sắc nét; giá phải trả là bước phát hiện ban đầu vẫn dùng ảnh gốc nên vẫn có vấn đề khớp thẳng-lên-cong ở tầng detector. Nó là cách mà các hộp công cụ hiệu chuẩn chất lượng cao dùng, và nó là lựa chọn đúng khi bạn kiểm soát được toàn bộ chuỗi.

1.8 Giới hạn và chế độ hồng

Giả thiết tâm chiếu là một điểm. Ống kính thật có *đồng tử vào* (entrance pupil) với kích thước hữu hạn, và vị trí hiệu dụng của nó dịch chuyển theo góc tới. Với chụp gần — và docking là chụp gần — hiệu ứng này không hoàn toàn bỏ qua được. Nó thường bị hấp thụ vào các hệ số méo trong quá trình hiệu chuẩn, nghĩa là các hệ số ấy trở thành phụ thuộc khoảng cách. Hệ quả thực hành: *hiệu chuẩn ở khoảng cách gần với khoảng cách vận hành*.

Giả thiết mô hình méo không đổi. Ống kính trôi theo nhiệt độ: giãn nở nhiệt làm tiêu cự đổi và làm điểm chính dịch. Với ống kính nhựa và chênh lệch nhiệt độ vài chục độ, hiệu ứng có thể ở mức pixel. B/12 bàn chuyện này; ở đây chỉ cần ghi nhận rằng “đã hiệu chuẩn” không phải một trạng thái vĩnh viễn.

Giả thiết lấy nét cố định. Đổi tiêu điểm làm đổi cả f lẫn hệ số méo. Với ống kính lấy nét tự động, mọi tham số nội của bạn chỉ đúng ở đúng vị trí lấy nét lúc hiệu chuẩn. Đây là lý do hệ thống đo lường dùng ống kính khoá nét, và là lý do nên dán keo vào vòng lấy nét sau khi hiệu chuẩn.

Giả thiết cảm biến phẳng và lưới pixel đều. Đúng ở mức rất tốt với cảm biến hiện đại, nên đây hiếm khi là vấn đề — ghi ra để hoàn chỉnh danh sách giả thiết chứ không phải để lo lắng.

1.9 Thực hành

Bốn việc, theo thứ tự.

Một, ước lượng f từ FOV ngay khi thiết kế, bằng công thức ở mục 2. Bạn cần nó cho ngân sách sai số ở C/15 và bạn cần nó *trước* khi mua camera.

Hai, chọn mô hình méo theo FOV chứ không theo mặc định của thư viện, dùng bảng 1.1. Dùng mô hình quá nghèo là sai số mô hình và nó không chứa được bằng dữ liệu; dùng mô hình quá giàu là rủi ro khớp quá mức và nó chứa được bằng dữ liệu phủ rộng.

Ba, sau mỗi lần hiệu chuẩn, vẽ bản đồ residual theo vị trí ảnh trước khi nhìn con số RMS. Nếu thấy cấu trúc xuyên tâm hoặc xoáy ở góc thì mô hình sai dạng, và thêm ảnh không cứu được.

Bốn, kiểm c_x, c_y bằng cách hiệu chuẩn lại vài lần với các bộ ảnh khác nhau và xem chúng có ổn định không. Nếu điểm chính nhảy hàng chục pixel giữa các lần thì dữ liệu của bạn không đủ đa dạng về tư thế, và con số bạn đang dùng là ngẫu nhiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
● ● ● Ba bước của mô hình chiếu	Thế giới sang camera (R, t) ; chia cho Z ra toạ độ chuẩn hoá (bước phi tuyến duy nhất); áp méo rồi nhân K ra pixel. Méo định nghĩa trên toạ độ <i>chuẩn hoá</i> , không phải pixel.
● ● ● Tiêu cự tính bằng pixel	$f_x = F/s_x$, một tỉ số <i>không thứ nguyên</i> . Ước lượng nhanh từ FOV: $f_x \approx (N_x/2)/\tan(\theta_{\text{FOV}}/2)$. Ví dụ 1280 px, $60^\circ \Rightarrow 1109$ px.
● ● ● Vì sao méo phá detector marker đặc biệt	Detector khớp <i>đường thẳng</i> lên cạnh tag rồi lấy giao [3], nhưng méo xuyên tâm biến đường thẳng thành đường cong. Khớp thẳng lên cong cho đường sai, và giao hai đường sai bị khuếch đại.
● ● ● Méo là nhiều hay thiên lệch?	<i>Thiên lệch</i> . Với tag ở một vị trí cố định, méo đẩy cạnh theo cùng hướng ở mọi khung. Không chia cho $\sqrt{N}$ — đây là nguồn “hệ thống” ở A/05 mục 8.
● ● ● Độ lớn méo ở rìa	Ví dụ: $f = 600$, $k_1 = -0,3$, điểm cách tâm 400 px $\Rightarrow$ lệch 53 px. Lớn hơn $\sigma = 0,2$ px hơn 250 lần. Nên khử méo là bắt buộc, không phải tinh chỉnh.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Điểm chính sai gây gì	Dịch mọi tia nhìn đi Δ/f radian. Với $\Delta = 10$ px, $f = 600$: $0,95^\circ$ — gần gấp đôi ngân sách góc. Không có triệu chứng thị giác nào.
●●● Vì sao c_x, c_y khó hiệu chuẩn	Chúng tương quan mạnh với tư thế target. Ảnh chỉ chụp gần vuông góc thì không tách được “điểm chính lệch trái” khỏi “target nghiêng phải”. Phải có tư thế nghiêng mạnh và phủ tới bốn góc.
●●● Sai số mô hình so với sai số tham số	Tham số: dạng hàm đúng, hệ số lệch — chứa được bằng thêm dữ liệu. Mô hình: dạng hàm không đủ — <i>không</i> chứa được bằng dữ liệu, thêm nghìn ảnh vẫn sai với khoảng tin cậy hẹp hơn.
●●● Phép thử phân biệt hai loại	Vẽ residual theo vị trí ảnh [6]. Nhiều trắng $\Rightarrow$ sai số tham số. Mẫu hình xuyên tâm hoặc xoáy ở góc $\Rightarrow$ sai số mô hình.
●● Vì sao RMS reprojection là chỉ số tệ	Trung bình che vùng rìa; nó giảm khi thêm tham số bất kể tham số có nghĩa hay không; và nó đo độ khớp trên dữ liệu hiệu chuẩn chứ không đo độ đúng khi ngoại suy sang vùng chưa có dữ liệu.
●● Khử méo toàn ảnh hay từng điểm	Toàn ảnh: giả định detector thành đúng, nhưng nội suy làm nhoè $\Rightarrow$ tăng σ . Từng điểm: giữ độ sắc nét, nhưng góc đã sai từ khâu khớp cạnh. Tốt hơn cả: đưa méo vào hàm sai số tái chiếu của PnP.
●● Chọn mô hình méo theo FOV	Dưới 50° : 2 hệ số radial. $50\text{--}90^\circ$: Brown–Conrady 5 tham số. $90\text{--}120^\circ$: hữu tỉ. Trên 120° : KB/fisheye [4]. Dùng nhằm theo hướng nghèo là sai số mô hình.
● Lỗi cài đặt phổ biến nhất	Tính r^2 bằng pixel thay vì tọa độ chuẩn hoá — sai theo hệ số f^2 . Triệu chứng: hiệu chuẩn vẫn hội tụ nhưng $k_1 \sim 10^{-7}$ thay vì cỡ -1 tới $+1$.
● Ba giả thiết hay bị quên	Tâm chiếu là điểm (sai khi chụp gần — nên hiệu chuẩn ở khoảng cách vận hành); mô hình không đổi (sai vì trôi nhiệt); lấy nét cố định (đổi nét là đổi cả f lẫn hệ số méo — nên khoá nét).

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV <code>calibrateCamera</code> , <code>undistortPoints</code>	Cài Brown–Conrady và hữu tỉ; <code>undistortPoints</code> chính là “cách hai” ở mục 7	Đọc kỹ thứ tự tham số <code>distCoeffs</code> : k_1, k_2, p_1, p_2, k_3 — không phải thứ tự trong công thức
Kalibr [7]	Hỗ trợ nhiều mô hình méo và báo cáo residual theo vị trí ảnh	Công cụ duy nhất trong danh sách mặc định cho bạn bản đồ residual ở mục 6
deltille [8]	Target lưới tam giác cho định vị góc chính xác hơn, dùng khi cần intrinsic chất lượng cao	Nổi sang B/08; target tốt hơn cho cùng thuật toán
Hiệu chuẩn từ target phẳng [2]	Phương pháp nền của mọi hộp công cụ; giải thích vì sao cần nhiều tư thế nghiêng	Đọc §3 để thấy vì sao tư thế fronto-parallel không cho thông tin về c_x, c_y
Double Sphere [5]	Mô hình ít tham số cho FOV rất rộng	Lựa chọn hiện đại thay cho KB khi cần ít tham số

MINI PROJECT · Đo méo còn sót lại sau khi khử, và vẽ bản đồ residual · 1–2 ngày

Mục tiêu	biến câu “hiệu chuẩn của tôi có tốt không” thành một bản đồ hai chiều, và tự tay nhìn thấy sự khác nhau giữa sai số mô hình và sai số tham số.
Dữ liệu	một camera thật và một target ChArUco hoặc chessboard in trên nền cứng phẳng. Nếu chưa có phần cứng, mô phỏng: sinh ảnh bằng một mô hình méo “thật” (chẳng hạn hữu tỉ 8 tham số) rồi hiệu chuẩn bằng một mô hình nghèo hơn (2 hệ số) — cách này còn rõ hơn vì bạn biết chân trị.
Các bước	chụp ít nhất 30 ảnh phủ kín khung hình, <i>bắt buộc</i> có ảnh mà target chạm sát bốn góc và có ảnh nghiêng trên 45° ; hiệu chuẩn với ba mô hình khác nhau (2 hệ số, Brown–Conrady đầy đủ, hữu tỉ); với mỗi mô hình, lấy vectơ residual của từng điểm và vẽ chúng như một trường vectơ trên mặt phẳng ảnh, phóng đại 50 lần; so sánh cả ba bản đồ; đồng thời ghi lại c_x, c_y của ba lần và của các tập con ảnh khác nhau.
Xong khi	bạn có ba bản đồ residual đặt cạnh nhau và phát biểu được một câu kiểu “với mô hình 2 hệ số, residual có cấu trúc xuyên tâm rõ với độ lớn tới X px ở bán kính $r > Y$; với Brown–Conrady đầy đủ thì cấu trúc biến mất”. Kiểm tra chéo bắt buộc: nếu c_x, c_y nhảy hơn 5 px giữa các tập con ảnh, thì dữ liệu của bạn chưa đủ đa dạng tư thế và mọi kết luận về mô hình méo đều chưa đáng tin — chụp thêm ảnh nghiêng trước đã.
Nối với	B/12 cho quy trình hiệu chuẩn đầy đủ; A/05 dùng f mà bạn vừa đo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì RMS là một trung bình trên toàn ảnh và trên toàn bộ dữ liệu hiệu chuẩn, nên nó có ba điểm mù. Nó che vùng rìa: hàng nghìn điểm giữa ảnh sai 0,1 px chìm chết vài chục điểm rìa sai 2 px. Nó luôn giảm khi thêm tham số, kể cả tham số vô nghĩa — nên so RMS giữa hai mô hình có số tham số khác nhau là vô nghĩa. Và quan trọng nhất, nó đo độ khớp *trên dữ liệu bạn đã cho* chứ không đo độ đúng ở vùng không có dữ liệu, mà tag của robot lúc tiếp cận rất có thể nằm đúng ở vùng ấy. Chỗ cần nhìn là bản đồ residual theo vị trí ảnh: nhiều trắng thì ổn, còn thấy cấu trúc xuyên tâm hay xoáy ở bốn góc thì mô hình sai dạng và thêm ảnh không cứu được.
- A2. Không phải 3 px, và thường là lớn hơn. Lý do là detector không lấy trực tiếp vị trí góc mà khớp một đường thẳng lên mỗi cạnh rồi lấy *giao* của hai đường. Méo làm cạnh cong, nên đường thẳng khớp lên nó bị lệch cả về vị trí lẫn về *hướng*; và một sai lệch hướng nhỏ của hai đường giao nhau sẽ chuyển thành sai lệch vị trí lớn ở giao điểm, với hệ số khuếch đại phụ thuộc góc giao — càng chệch thì càng khuếch đại. Cần thêm một điều: sai lệch này là thiên lệch chứ không phải nhiễu, nên lấy trung bình nghìn khung không giúp gì.
- A3. Nó dịch *mọi* tia nhìn đi một góc Δ/f radian — với $\Delta = 10$ px và $f = 600$ px thì là $0,95^\circ$, tức gần gấp đôi ngân sách góc của cây. Hậu quả thay đổi theo vị trí tag trong khung hình vì x_n vào phép chiếu cùng với phép chia cho Z , nên hiệu ứng của Δ lên sáu tham số pose không phải một phép quay thuần mà phụ thuộc cả vị trí lẫn khoảng cách. Điều làm nó nguy hiểm đặc biệt: nó không có triệu chứng thị giác nào — không ai nhìn ảnh mà phát hiện được điểm chính sai — và triệu chứng ở đầu ra thì trông hết như sai số hand-eye hoặc constellation sai, nên người ta hay đi tìm nhầm chỗ.
- A4. Khử méo toàn ảnh chính xác hơn về mặt mô hình, vì nó làm giả định “cạnh là đường thẳng” của detector trở thành đúng; nhưng nó đòi nội suy pixel, mà nội suy làm nhoè, mà nhoè làm tăng σ . Khử méo từng điểm nhanh hơn nhiều và giữ nguyên độ sắc nét, nhưng nó không sửa được việc bốn góc đã bị định vị sai từ khâu khớp cạnh trên ảnh cong. Hai câu trả lời khác nhau vì hai câu hỏi đo hai thứ khác nhau: một câu hỏi về tính đúng của mô hình, một câu hỏi về chi phí tính toán và về σ . Quy tắc thực hành: tag nhỏ và gần tâm thì dùng cách hai; tag lớn hoặc ở rìa ống kính góc rộng thì bắt buộc cách một; và tốt hơn cả hai là đưa mô hình méo thẳng vào hàm sai số tái chiếu của bước tinh chỉnh PnP.
- A5. Vì residual nhỏ chỉ chứng minh mô hình khớp được *dữ liệu bạn đã cho*, không chứng minh nó mô tả đúng ống kính. Với ống kính 120° , phép chiếu pinhole $r_n = \tan \theta$ đã gần phân kỳ, và không đa thức bậc bốn nào theo dõi kịp — nhưng bộ tối ưu vẫn tìm ra bộ (k_1, k_2) tốt nhất trong họ hàm nghèo ấy và báo về một con số RMS nghe được. Đây là sai số mô hình, và thêm một nghìn ảnh chỉ làm khoảng tin cậy của các hệ số hẹp lại quanh một giá trị vẫn sai — tệ hơn, vì bạn sẽ tin nó hơn. Dấu hiệu lộ ra: bản đồ residual có cấu trúc xuyên tâm với độ lớn tăng theo

bán kính; và nếu bạn hiệu chuẩn hai lần với hai tập ảnh phủ vùng khác nhau, các hệ số sẽ khác nhau đáng kể vì mỗi lần mô hình đang “cố” khớp một vùng khác.

NÓI LẠI THẬT ĐƠN GIẢN

Một cái camera về bản chất là một cái hộp có một lỗ nhỏ. Ánh sáng từ mọi vật chui qua cái lỗ ấy rồi đập vào thành sau của hộp, và ảnh hiện ra. Nếu cái lỗ thật sự nhỏ như một điểm và ánh sáng đi thẳng, thì mọi thứ rất dễ tính: cứ kẻ một đường từ vật qua cái lỗ, nó đập vào đâu thì ảnh ở đó. Vấn đề là camera thật không có lỗ nhỏ, nó có thấu kính — vì lỗ nhỏ cho quá ít ánh sáng. Thấu kính thu được nhiều sáng hơn nhưng nó bẻ cong tia sáng, và nó bẻ tia ở rìa nhiều hơn tia ở giữa. Kết quả là một cái cửa sổ hình chữ nhật chụp bằng ống kính góc rộng trông phình ra như quả trứng.

Với phần lớn bài toán thị giác thì chuyện đó không quá tệ, vì người ta chỉ cần biết từng điểm nằm đâu, và có công thức để trả điểm về đúng chỗ. Nhưng cái tag thì khác, và đây là chỗ cần chú ý. Máy tính tìm góc của tag bằng cách nhìn bốn cạnh, kẻ bốn đường thẳng dọc theo chúng, rồi lấy chỗ các đường cắt nhau. Cách ấy rất khôn vì nó dùng cả trăm pixel dọc cạnh chứ không chỉ vài pixel ở góc. Nhưng nó chỉ đúng nếu cạnh thật sự thẳng — mà ống kính vừa làm chúng cong.

Kẻ một đường thẳng dọc theo một đường cong thì được một đường lệch. Lấy chỗ hai đường lệch cắt nhau thì được một điểm lệch nhiều hơn nữa — giống như hai cây gậy dài chạm nhau ở đầu: nghiêng mỗi cây một chút thôi, đầu gậy đã trượt đi khá xa.

Còn một thứ nữa, kín đáo hơn và vì thế phiền hơn. Cái lỗ của camera đáng lẽ nằm đúng giữa thành sau, nhưng khi lắp ráp thì nó lệch đi một chút. Chuyện lệch ấy không làm ảnh trông khác đi chút nào — bạn không có cách nào nhìn một tấm ảnh rồi bảo “lỗ này lệch sang trái mười pixel”, vì trong ảnh không có gì để so. Nhưng máy tính lại dùng vị trí của cái lỗ để tính hướng nhìn, nên lệch mười pixel tương đương với tính sai hướng gần một độ. Và một độ thì đã quá ngân sách của chúng ta rồi.

Bài học chung của cả chương: có những sai lệch làm ảnh trông xấu, và có những sai lệch không làm ảnh trông khác gì cả. Loại thứ hai mới là loại phải cẩn thận, vì không có gì nhắc bạn rằng chúng đang ở đó.

Chỗ không rút gọn được: việc chọn mô hình méo nào cho ống kính nào thì không có quy tắc đơn giản đúng cho mọi trường hợp. Bảng trong chương là hướng dẫn theo góc nhìn, và nó đúng phần lớn thời gian, nhưng cách chắc chắn duy nhất là hiệu chuẩn với vài mô hình rồi nhìn bản đồ residual xem cái nào hết cấu trúc.

Một câu: Ống kính bẻ cong đường thẳng, còn máy tính thì cứ tưởng chúng thẳng — và cái lỗ lệch mười pixel thì không ai nhìn thấy nhưng nó ăn gần một độ.

1.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *méo chưa khử làm cong cạnh tag, và việc khớp đường thẳng lên cạnh cong khuếch đại sai số tại giao điểm* — bị bác bỏ nếu một mô phỏng cho thấy sai số góc do méo gây ra nhỏ hơn hoặc bằng độ lớn của méo tại vị trí góc đó. Tôi dẫn mệnh đề bằng hình học (cửa a) và bằng ví dụ số cho độ lớn méo (cửa b cho phần độ lớn), nhưng phần *khuếch đại tại giao điểm* thì tôi chưa đo bằng mô phỏng — đó là một nợ kiểm chứng.

Mệnh đề thứ hai — *điểm chính sai Δ px gây thiên lệch hướng cỡ Δ/f radian* — là mệnh đề yếu nhất về mặt kiểm chứng trong chương, như đã ghi thẳng ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy sai số pose do Δ gây ra được hấp thụ gần hết vào các tham số khác và biên độ còn lại nhỏ hơn nhiều so với Δ/f . Đây là mệnh đề tôi khuyên kiểm sớm nhất, vì nếu nó sai thì mức ưu tiên tôi gán cho việc hiệu chuẩn điểm chính ở B/12 là quá cao.

Mệnh đề thứ ba — *sai số mô hình không chữa được bằng thêm dữ liệu* — là mệnh đề chắc chắn nhất, vì nó là một mệnh đề về bản chất của khớp trong một họ hàm hạn chế, không phải một mệnh

để thực nghiệm. Nó “bị bác bỏ” chỉ nếu ai đó chỉ ra rằng họ hàm bạn dùng thật ra có chứa hàm đúng — tức là chỉ ra rằng bạn chẩn đoán nhầm, chứ không phải mệnh đề sai.

Cuối cùng, một mệnh đề mềm: chương này khẳng định rằng *trong thực tế, điểm chính là tham số nội gây hại nhất*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã hiệu chuẩn điểm chính cẩn thận, sai số xoay không cải thiện đáng kể so với khi dùng giá trị mặc định giữa ảnh. (Tôi phát biểu dựa trên cấu trúc của phép chiếu và trên sự thiếu triệu chứng của lỗi này, không dựa trên thống kê ca hòng thực tế.)

1.11 Kết nối

Chương này là gốc của Phần A và nó cấp vật liệu cho gần như mọi chương sau.

Nối xuống dưới trong Phần A. [A/02-homography-va-hinh-hoc-phang](#) dùng ma trận K trực tiếp trong phân rã $H = K[r_1 \ r_2 \ t]$, và nó giả định ảnh đã được khử méo — một giả định mà chương này vừa cho thấy không hiển nhiên. [A/03-pnp-va-uoc-luong-tu-the](#) dùng mô hình chiếu ở mục 2 làm hàm $\hat{u}(\xi)$ trong sai số tái chiếu, và mục 7 ở đây bàn chỗ đưa méo vào hàm ấy. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) lấy f làm mẫu số của công thức sai số ngang, và nó lấy từ mục 6 ở đây luận điểm quan trọng rằng “mô hình đúng” là một giả thiết chứ không phải một sự thật — khi giả thiết ấy vi phạm, hiệp phương sai mà chương 5 tính ra là sai theo chiều nguy hiểm.

Nối sang Phần B. [B/07-detection-pipeline](#) là chương mô tả chi tiết cơ chế khớp cạnh mà mục 4 ở đây dựa vào; đọc hai mục ấy cùng nhau thì thấy rõ vì sao méo và detector tương tác xấu. [B/08-corner-refinement](#) quyết định σ , và mục 7 ở đây cảnh báo rằng khử méo toàn ảnh làm tăng chính σ ấy — một đánh đổi phải cân. [B/12-calibration-cho-docking](#) là chương nhận trực tiếp và đầy đủ nhất: toàn bộ mục 5 và mục 6 ở đây trở thành quy trình cụ thể ở đó, gồm cả yêu cầu về độ phủ dữ liệu và phép thử residual có cấu trúc.

Nối sang Phần C. [C/15-ngan-sach-sai-so](#) có một dòng riêng cho “méo còn sót lại” và một dòng riêng cho “sai số intrinsic”, và cả hai dòng ấy được định giá bằng các cơ chế ở chương này. Đáng chú ý là chúng thuộc loại *hệ thống* theo phân loại ở [A/05](#) mục 8, nên chúng không giảm khi lấy trung bình nhiều khung — và đó là lý do chúng thường chiếm phần lớn ngân sách trong một hệ thống đã tối ưu tốt phần nhiều.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao phép méo xuyên tâm biến một đường thẳng thành đường cong, nhưng có đúng một họ đường thẳng vẫn thẳng sau khi méo — họ nào, và vì sao?
2. Detector khớp đường thẳng lên cạnh tag. Nếu chính xác chuỗi ba bước biến “ảnh còn méo 3 px” thành “pose sai”, và chỉ ra bước nào khuếch đại sai số.
3. Bạn hiệu chuẩn hai lần với hai tập ảnh khác nhau và được hai giá trị c_x cách nhau 15 px. Điều đó nói gì về dữ liệu của bạn, và vì sao f_x của hai lần lại có thể gần như trùng nhau?
4. Vì sao thêm tham số vào mô hình méo luôn làm RMS reprojection giảm, và vì sao điều đó không chứng minh mô hình tốt hơn?
5. Một đồng nghiệp khử méo toàn ảnh bằng nội suy song tuyến tính rồi thấy σ tăng. Giải thích cơ chế, và nêu hai cách giữ được cả tính đúng của mô hình lẫn độ sắc nét.
6. Với $\Delta c_x = 10$ px và $f = 600$ px, thiên lệch hướng là bao nhiêu độ? Vì sao thiên lệch ấy phần lớn triệt tiêu trong sơ đồ teach-by-showing nhưng *không* triệt tiêu hoàn toàn?
7. Nêu ba giả thiết ngầm của mô hình pinhole cộng méo mà chương này liệt kê, và với mỗi giả thiết, một tình huống docking cụ thể làm nó bị vi phạm.

Nguồn

- [1] D. C. Brown, "Close-Range Camera Calibration," *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, "AprilTag 2: Efficient and Robust Fiducial Detection," in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, "The Double Sphere Camera Model," in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, "Accurate Calibration of CCD-Cameras," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, "Unified Temporal and Spatial Calibration for Multi-Sensor Systems," in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., "Deltile Grids for Geometric Camera Calibration," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, "Geometric Camera Calibration Using Circular Control Points," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, "Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods," *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, "Infinitesimal Plane-Based Pose Estimation," *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, “Propagating Covariance in Computer Vision,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, “Statistical Optimization for Geometric Computation: Theory and Practice,” 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., “Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera,” in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., “Rolling Shutter Camera Calibration,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, “Introduction to Fourier Optics,” 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

Homography và hình học phẳng (Homography and Planar Geometry)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (ma trận K , tọa độ chuẩn hoá, và — quan trọng — cảnh báo rằng mọi thứ trong chương này giả định ảnh đã được khử méo; nếu chưa thì không có homography nào khớp được).

TÓM TẮT

Chương này dựng công cụ cho trường hợp riêng mà toàn bộ bài toán marker rơi vào: mọi điểm điều khiển nằm trên một mặt phẳng. Trường hợp riêng ấy đơn giản hơn trường hợp tổng quát một cách đáng kể — quan hệ giữa mặt phẳng vật và mặt phẳng ảnh là một phép biến đổi tuyến tính trong toạ độ thuần nhất, gọi là homography, chỉ có tám bậc tự do và giải được bằng bốn điểm. Ba điều đáng mang ra khỏi chương. Một, công thức $H = K[r_1 \ r_2 \ t]$ và phép phân rã ngược để lấy pose — nó là cách khởi tạo chuẩn cho PnP. Hai, số điều kiện của H xấu đi nhanh khi tag nhỏ trên ảnh, và đó là lý do định lượng cho việc đặt ngưỡng kích thước tối thiểu. Ba, và đây là mục ít người biết nhất: *tâm của hình chiếu không bằng hình chiếu của tâm*. Với marker tròn, sự thật ấy tạo ra một thiên lệch có hệ thống phụ thuộc tư thế, và nó là lý do marker tròn cần hiệu chỉnh conic chứ không dùng thẳng tâm ellipse.

KIẾN THỨC CẦN CÓ TRƯỚC

- **homography** — phép biến đổi tuyến tính khả nghịch trên toạ độ thuần nhất 2D, biểu diễn bởi một ma trận 3×3 xác định sai khác một hệ số. Nó là quan hệ chính xác giữa hai ảnh của cùng một mặt phẳng, và cũng là quan hệ giữa mặt phẳng vật và ảnh của nó.
- **bậc tự do của H** — tám, không phải chín: ma trận có chín phần tử nhưng nhân toàn bộ với một hằng số cho cùng một phép biến đổi, nên một bậc tự do là dư. Hệ quả thực hành: cần bốn cặp điểm tương ứng (mỗi cặp cho hai phương trình) để xác định H .
- **DLT** — *Direct Linear Transform*, phương pháp giải H bằng cách viết các ràng buộc thành một hệ tuyến tính thuần nhất $Ah = 0$ rồi lấy vectơ kỳ dị ứng với giá trị kỳ dị nhỏ nhất.
- **chuẩn hoá Hartley** — phép biến đổi tiền xử lý đưa tập điểm về trung bình không và khoảng cách trung bình tới gốc bằng $\sqrt{2}$, trước khi chạy DLT. Nó không phải một mẹo làm đẹp: nó thay đổi số điều kiện của A vài bậc độ lớn, và bỏ nó là lỗi cài đặt kinh điển [9, §4.4].
- **số điều kiện (condition number)** — tỉ số giữa giá trị kỳ dị lớn nhất và nhỏ nhất của một ma trận. Nó đo mức khuếch đại sai số: số điều kiện 10^6 nghĩa là nhiễu đầu vào có thể bị nhân lên triệu lần ở một hướng nào đó của nghiệm.
- **tỉ số kép (cross-ratio)** — đại lượng gồm bốn điểm thẳng hàng, $(AC/BC)/(AD/BD)$, và là bất biến *duy nhất* của phép chiếu phối cảnh trên một đường thẳng. Mọi thứ khác — khoảng cách, tỉ số hai đoạn, trung điểm — đều không bảo toàn.
- **conic** — đường cong bậc hai trên mặt phẳng, biểu diễn bởi ma trận đối xứng 3×3 : $\mathbf{x}^T C \mathbf{x} = 0$. Đường tròn và ellipse đều là conic, và ảnh phối cảnh của một conic vẫn là một conic — đây là tính chất làm marker tròn xử lý được bằng đại số tuyến tính.

CÂU HỎI MỞ ĐẦU

- Q1. Homography có tám bậc tự do còn pose chỉ có sáu. Hai bậc dư ấy chứa gì, và tại sao sự dư thừa ấy lại là một công cụ chẩn đoán miễn phí?
- Q2. Bạn vẽ một vòng tròn lên tám marker rồi tìm tâm của ellipse mà nó tạo ra trên ảnh. Vì sao điểm bạn vừa tìm *không* phải hình chiếu của tâm vòng tròn, và sai lệch ấy phụ thuộc vào cái gì?
- Q3. Tag của bạn co lại từ 200 px xuống 40 px trên ảnh khi robot lùi ra xa. Nhiều góc σ không đổi. Pose lại xấu đi nhanh hơn nhiều so với tỉ lệ $1/w$ mà A/05 dự đoán. Cơ chế nào trong chương này giải thích phần vượt trội đó?
- Q4. Một đồng nghiệp cài DLT để giải homography, không chuẩn hoá điểm trước, và kết quả “chạy được” trên dữ liệu thử. Vì sao nó sẽ hỏng trên dữ liệu thật, và hỏng theo kiểu nào?
- Q5. Trong bốn bất biến sau, cái nào được phép chiếu phối cảnh bảo toàn: khoảng cách, tỉ số hai đoạn thẳng hàng, trung điểm, tỉ số kép? Và vì sao câu trả lời quyết định cách bạn thiết kế marker?

2.1 Đặt vấn đề

Bài toán ước lượng pose từ ảnh nói chung là một bài toán khó: sáu ẩn, quan hệ phi tuyến, nhiều nghiệm, cần lập. Nhưng bài toán marker rơi vào một trường hợp riêng mà hình học ưu ái đặc biệt — *mọi điểm điều khiển nằm trên một mặt phẳng* — và trường hợp riêng ấy có một cấu trúc đẹp đến mức đáng dành một chương.

Cấu trúc ấy là thế này. Khi vật là một mặt phẳng, quan hệ giữa toạ độ trên mặt phẳng vật và toạ độ trên ảnh là *tuyến tính* trong toạ độ thuần nhất. Không có phép chia phi tuyến còn lại, không cần lập, không có nhiều nghiệm ở tầng này: bốn điểm là đủ và lời giải là một bài toán trị riêng. Đây là lý do lịch sử vì sao hiệu chuẩn camera bằng target phẳng — phương pháp Zhang [2] — lại thay thế được các phương pháp cũ dùng vật hiệu chuẩn ba chiều đắt tiền: mặt phẳng dễ chế tạo chính xác, và toán học của nó dễ hơn.

Cấu trúc ấy cũng là lý do vì sao mọi detector marker đều dùng homography ở giữa quy trình. ArUco rectify vùng tag về hình vuông chuẩn bằng homography rồi mới lấy mẫu các ô bit [10]; không có bước ấy thì việc đọc bit từ một tứ giác méo là một cơn ác mộng. AprilTag dùng homography để khởi tạo pose trước khi tinh chỉnh. ChArUco dùng nó để dự đoán vị trí các góc chessboard sau khi đã định danh bằng các ô ArUco.

Nhưng chương này không tồn tại chỉ để trình bày một công cụ tiện lợi. Nó tồn tại vì hình học phẳng mang theo *hai cái bẫy* mà người dùng công cụ không thấy.

Cái bẫy thứ nhất là về điều kiện số. Homography luôn giải được về mặt đại số, kể cả khi bốn điểm gần trùng nhau hoặc gần thẳng hàng — bộ giải trả về một ma trận và không báo lỗi. Nhưng chất lượng của ma trận ấy phụ thuộc dữ dội vào hình học của bốn điểm, và sự phụ thuộc ấy không xuất hiện trong bất kỳ thông báo nào. Mục 4 định lượng chuyện này, và nó giải thích một hiện tượng mà A/05 một mình không giải thích được: vì sao pose xấu đi *nhANH* hơn tỉ lệ tuyến tính khi tag co lại.

Cái bẫy thứ hai tinh vi hơn nhiều và nó là lý do chính tôi xếp chương này ở mức quan trọng thay vì tham khảo. Phép chiếu phối cảnh *không bảo toàn trung điểm*. Nghe thì vô hại, nhưng nó có một hệ quả rất cụ thể: nếu bạn vẽ một hình tròn và tìm tâm của ellipse mà nó tạo trên ảnh, điểm bạn tìm được *không* phải hình chiếu của tâm hình tròn. Sai lệch ấy là một thiên lệch có hệ thống, nó phụ thuộc tư thế, và nó là nguồn sai số chính khi dùng marker tròn nếu không hiệu chỉnh. Kết quả này đã được biết rõ trong cộng đồng hiệu chuẩn camera từ hai thập niên [11], [12] và nó vẫn tiếp tục làm người mới bất ngờ.

TRỰC GIÁC

Hình dung việc chiếu bóng một tấm bìa lên tường bằng một cái đèn pin.

Nếu tấm bìa song song với tường, bóng của nó là một bản phóng to trung thực: hình vuông vẫn vuông, trung điểm vẫn ở giữa, hai đoạn bằng nhau vẫn bằng nhau. Đây là phép đồng dạng, và mọi trực giác hình học quen thuộc đều còn đúng.

Nghiêng tấm bìa đi, và mọi thứ bắt đầu lệch. Hình vuông thành hình thang. Cạnh xa đèn trông ngắn hơn cạnh gần. Và — đây là chỗ đáng chú ý — *trung điểm của cạnh không còn chiếu thành trung điểm của bóng*: vì phần gần đèn bị phóng to nhiều hơn phần xa, điểm giữa bị đẩy về phía xa.

Cái mà phép chiếu *vẫn* bảo toàn không phải là khoảng cách hay tỉ lệ mà là một đại lượng phức tạp hơn gồm bốn điểm, gọi là tỉ số kép. Bốn điểm thẳng hàng có một con số gắn với chúng, và con số ấy giống hệt nhau ở vật và ở bóng, dù nghiêng thế nào. Nó là bất biến *duy nhất*, và mọi thứ đơn giản hơn nó đều không bảo toàn.

Giờ áp vào marker. Một cái tag vuông có bốn góc, và bốn góc ấy là bốn điểm rời rạc xác định rõ — phép chiếu đưa chúng đi đâu thì đi, ta vẫn biết điểm nào là điểm nào. Nhưng một cái marker tròn thì không có góc: cái ta đo được là *tâm của đường cong*, mà tâm là một khái niệm dựa trên trung điểm, mà trung điểm thì không bảo toàn. Ta đang đo một thứ không phải thứ ta nghĩ mình đang đo.

2.2 Homography của một mặt phẳng

Dẫn công thức, vì nó ngắn và vì mỗi bước đều nói một điều.

Chọn hệ toạ độ trên mặt phẳng marker sao cho mặt phẳng ấy là $Z = 0$. Một điểm trên marker có toạ độ $(X, Y, 0)$. Áp mô hình chiếu của A/01:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3 \ \mathbf{t}] \begin{pmatrix} X \\ Y \\ 0 \\ 1 \end{pmatrix} = K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}] \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix},$$

trong đó $\mathbf{r}_i$ là các cột của R , và dấu $\sim$ nghĩa là bằng nhau sai khác một hệ số khác không.

Cột thứ ba $\mathbf{r}_3$ biến mất vì nó nhân với $Z = 0$. Đó là toàn bộ mẹo, và nó đáng dừng lại một câu: chính việc *mất* một cột làm cho quan hệ trở thành một ma trận 3×3 — tức một phép biến đổi tuyến tính giữa hai mặt phẳng — thay vì một phép chiếu từ ba chiều xuống hai chiều. Ta định nghĩa

$$H = K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]. \quad (2.1)$$

Đây cũng là chỗ trả lời Q1 về hai bậc tự do dư. H có tám bậc tự do (chín phần tử trừ một hệ số tỉ lệ). Pose có sáu. Vậy hai bậc dư nằm ở đâu? Chúng nằm ở chỗ một ma trận 3×3 bất kỳ *không* nhất thiết có dạng $K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ với $\mathbf{r}_1, \mathbf{r}_2$ trực chuẩn. Hai ràng buộc $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$ và $\mathbf{r}_1^\top \mathbf{r}_2 = 0$ chính là hai bậc tự do bị loại đi khi ta đi từ H tổng quát về một pose hợp lệ.

Và đây là chỗ sự dư thừa trở thành công cụ chẩn đoán miễn phí — phần thứ hai của Q1. Sau khi tính H từ dữ liệu bằng DLT, ta có thể kiểm xem $K^{-1}H$ có *thật sự* thoả hai ràng buộc trực chuẩn không. Nếu hai cột đầu của $K^{-1}H$ có độ dài lệch nhau 20% hoặc tích vô hướng của chúng khác không đáng kể, thì một trong ba điều đã sai: các điểm không thực sự đồng phẳng, ảnh chưa khử méo đúng, hoặc K sai. Phép kiểm ấy tốn vài dòng code và nó bắt được cả ba lỗi phổ biến nhất — đáng ngạc nhiên là rất ít cài đặt làm nó.

2.3 Phân rã H thành pose

Đi ngược từ H về $(R, \mathbf{t})$. Từ (2.1), đặt $M = K^{-1}H = [\mathbf{m}_1 \ \mathbf{m}_2 \ \mathbf{m}_3]$. Về nguyên tắc $\mathbf{m}_1 = \lambda \mathbf{r}_1$, $\mathbf{m}_2 = \lambda \mathbf{r}_2$, $\mathbf{m}_3 = \lambda \mathbf{t}$ với một hệ số λ chưa biết.

Vì $\|\mathbf{r}_1\| = 1$, ta có $\lambda = \|\mathbf{m}_1\|$. Trong thực tế, do nhiễu, $\|\mathbf{m}_1\| \neq \|\mathbf{m}_2\|$, nên cách dùng phổ biến là lấy trung bình:

$$\lambda = \frac{1}{2}(\|\mathbf{m}_1\| + \|\mathbf{m}_2\|).$$

Rồi

$$\mathbf{r}_1 = \mathbf{m}_1/\lambda, \quad \mathbf{r}_2 = \mathbf{m}_2/\lambda, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \mathbf{m}_3/\lambda.$$

Cột thứ ba lấy bằng tích có hướng vì R trực giao thuận. Đây là quy trình chuẩn, chép ý từ [2, §3.1] với ký hiệu đổi sang của cây.

Ma trận R thu được theo cách này *không* trực giao chính xác vì nhiễu, nên phải chiếu nó về $SO(3)$ gần nhất. Cách đúng là qua phân tích SVD: nếu $R = U\Sigma V^T$ thì ma trận trực giao gần nhất theo chuẩn Frobenius là UV^T — một kết quả cổ điển gọi là bài toán Procrustes trực giao. Cần chú ý một chi tiết cài đặt: phải kiểm $\det(UV^T) = +1$; nếu ra -1 thì đã thu được một phép phản xạ chứ không phải phép quay, và phải đổi dấu cột cuối của V .

Cũng có một tình huống dấu cần xử lý: H xác định sai khác một hệ số, kể cả dấu, nên có thể ra $\mathbf{t}$ với $t_z < 0$ — tức marker nằm *sau* camera. Vô lý về vật lý, nên nếu gặp thì đổi dấu toàn bộ H rồi làm lại.

CẠM BẮY

Phép phân rã ở mục này cho ra một pose *hợp lệ*, không phải pose *tốt nhất*. Ba lý do, và cả ba đều dẫn tới cùng một kết luận thực hành.

Thứ nhất, DLT cực tiểu hoá một sai số đại số ($\|A\mathbf{h}\|$) chứ không phải sai số hình học có ý nghĩa. Đại lượng ta thật sự quan tâm là sai số tái chiếu tính bằng pixel, và hai thứ đó không tỉ lệ với nhau.

Thứ hai, phép trung bình λ và phép chiếu về $SO(3)$ đều là những cách sửa *sau khi* đã có nghiệm, không phải cách tìm nghiệm tốt hơn. Chúng đảm bảo kết quả là một pose hợp lệ nhưng không đảm bảo nó tối ưu theo tiêu chí nào cả.

Thứ ba, phép phân rã không nói gì về bài toán hai nghiệm của A/04. Nó cho ra một nghiệm, và nghiệm ấy có thể là nghiệm sai.

Kết luận: *luôn coi kết quả phân rã H là điểm khởi đầu, không phải kết quả*. Chạy tinh chỉnh phi tuyến trên sai số tái chiếu sau đó (A/03), và với target phẳng thì dùng IPPE để lấy *cả hai* nghiệm thay vì phân rã H — IPPE làm đúng việc này và làm tốt hơn [13].

2.4 Điều kiện số: vì sao tag nhỏ tệ hơn công thức dự đoán

Đây là mục có giá trị thực hành cao nhất trong chương, và nó trả lời Q3.

A/05 cho quan hệ $\delta Z \approx Z\sigma/w$ — sai số dọc trực tỉ lệ nghịch với kích thước tag trên ảnh. Đó là một quan hệ tuyến tính và nó đúng trong chế độ hoạt động tốt. Nhưng người ta quan sát được rằng khi tag co xuống dưới một ngưỡng nào đó, chất lượng pose xuống dốc nhanh hơn nhiều so với $1/w$. Cơ chế của phần vượt trội ấy nằm ở chương này.

Bài toán DLT là giải hệ thuần nhất $A\mathbf{h} = 0$, với A dựng từ toạ độ các điểm tương ứng. Chất lượng nghiệm phụ thuộc vào *số điều kiện* của A , tức tỉ số giữa giá trị kỳ dị lớn nhất và nhỏ nhất. Số điều kiện càng lớn thì nhiễu đầu vào càng bị khuếch đại.

Có hai cơ chế làm số điều kiện xấu đi.

Cơ chế thứ nhất, và nó khắc phục được: thang đo không đồng nhất. Các phần tử của A gồm các tích như uX , u , X , và 1 . Nếu u cỡ 10^3 (toạ độ pixel) còn X cỡ 10^{-1} (mét), thì các phần tử của A trải qua nhiều bậc độ lớn, và số điều kiện tệ ngay từ đầu bất kể hình học có tốt hay không. Cách khắc phục là *chuẩn hoá Hartley*: đưa cả hai tập điểm về trung bình không và khoảng cách trung bình $\sqrt{2}$ trước khi dựng A , rồi biến đổi ngược H sau khi giải [9, §4.4]. Đây không phải tùy chọn — nó cải thiện số điều kiện vài bậc độ lớn, và bỏ nó là lỗi cài đặt kinh điển. Đây cũng là câu trả lời cho Q4: code không chuẩn hoá “chạy được” trên dữ liệu thử vì dữ liệu thử thường có thang đo hiển, và nó hỏng trên dữ liệu thật theo kiểu khó truy nhất — kết quả không sai hẵn mà chỉ kém chính xác một cách thất thường, tệ hơn ở một số tư thế và ổn ở tư thế khác.

Cơ chế thứ hai, và nó không khắc phục được bằng code: hình học của bốn điểm. Kể cả sau khi chuẩn hoá, số điều kiện vẫn phụ thuộc vào việc bốn điểm phân bố thế nào. Bốn điểm trải rộng thành một tứ giác gần vuông cho điều kiện tốt; bốn điểm gần trùng nhau, gần thẳng hàng, hoặc tạo thành một tứ giác rất dẹt cho điều kiện xấu. Và khi tag co nhỏ trên ảnh, bốn góc của nó gần trùng nhau theo nghĩa toạ độ pixel của chúng chênh nhau ít — nên điều kiện xấu đi.

Vì sao điều đó gây tệ hơn tuyến tính? Vì hai hiệu ứng chồng lên nhau. Sai số góc σ cố định chiếm một *tỉ lệ* ngày càng lớn trong kích thước tag khi tag co lại (hiệu ứng thứ nhất, tuyến tính, đã có trong [A/05](#)), và đồng thời phép giải trở nên kém điều kiện nên khuếch đại phần sai số ấy nhiều hơn (hiệu ứng thứ hai). Tích của hai hiệu ứng tệ hơn mỗi cái riêng lẻ.

Ghi chú trung thực: tôi dẫn cơ chế này bằng lập luận về số điều kiện (cửa a) nhưng tôi *chưa* có ví dụ số đo mức độ vượt trội so với $1/w$ (cửa b chưa qua). Vì vậy tôi phát biểu nó ở dạng định tính — “tệ hơn tuyến tính” — chứ không gán một số mũ. Đây là một mục trong 99-chua-biet.md, và nó đo được dễ dàng bằng script Monte Carlo của C/16.

CẦN NHỚ

Hai quy tắc thực hành rút ra từ mục này, và cả hai đều rẻ.

Một, *luôn chuẩn hoá điểm trước DLT*. Nếu bạn dùng `findHomography` của OpenCV thì nó đã làm sẵn; nếu bạn tự cài thì đừng bỏ qua.

Hai, *đặt một ngưỡng kích thước tag tối thiểu và từ chối phát hiện dưới ngưỡng ấy*. Ngưỡng thường dùng là khoảng 40–60 px cho cạnh tag, nhưng con số đúng cho hệ của bạn phải đo: chạy Monte Carlo với w giảm dần và tìm chỗ sai số bắt đầu chệch khỏi đường $1/w$. Đó là ngưỡng của bạn.

Hệ quả thiết kế quan trọng hơn cả hai: đây chính là lý do của chiến lược *nested multi-scale tags* ở B/09 — tag lớn cho cự ly xa, tag nhỏ cho cự ly gần, sao cho luôn có ít nhất một tag nằm trong dải kích thước tốt trên ảnh.

2.5 Bất biến của phép chiếu: chỉ có tỉ số kép

Mục này ngắn nhưng nó là nền của mục 6, nên đừng lướt.

Câu hỏi: phép chiếu phối cảnh bảo toàn cái gì? Danh sách ngắn hơn trực giác gợi ý nhiều.

Không bảo toàn: khoảng cách (hiển nhiên); tỉ số hai đoạn thẳng hàng (cạnh xa trông ngắn hơn); trung điểm (hệ quả của điều trước); song song (hai đường ray gặp nhau ở đường chân trời); góc; diện tích.

Có bảo toàn: tính thẳng hàng (đường thẳng vẫn là đường thẳng); tính đồng quy; bậc của đường cong đại số (conic vẫn là conic); và *tỉ số kép*.

Tỉ số kép của bốn điểm thẳng hàng A, B, C, D là

$$\text{cr}(A, B; C, D) = \frac{\overline{AC}/\overline{BC}}{\overline{AD}/\overline{BD}},$$

và nó giống hệt nhau ở vật và ở ảnh. Đây là câu trả lời cho Q5, và điều đáng chú ý không phải bản thân công thức mà là *tính duy nhất* của nó: tỉ số kép là bất biến độc lập duy nhất của phép chiếu trên một đường thẳng, nên bất kỳ đại lượng nào bạn muốn đo mà bảo toàn qua phép chiếu thì phải viết được theo nó.

Hệ quả thiết kế cho marker, và đây là chỗ mục này trả giá trị. Một marker mã hoá thông tin bằng *khoảng cách* hoặc bằng *tỉ lệ hai đoạn* sẽ không đọc được tin cậy dưới góc nghiêng, vì những đại lượng ấy không bảo toàn. Một marker mã hoá bằng *cấu trúc rời rạc* — các ô đen trắng trong một lưới, như ArUco và AprilTag — thì đọc được, vì sau khi rectify bằng homography thì lưới trở lại đều. Và một marker mã hoá bằng tỉ số kép trực tiếp thì đọc được mà *không cần* rectify — một ý tưởng đẹp đã được dùng trong một số thiết kế marker nhưng nó không thắng được cách tiếp cận lưới về mặt thực dụng, vì đo bốn điểm chính xác khó hơn đọc một ô sáng hay tối.

2.6 Bias tâm ellipse: tâm chiếu không bằng chiếu của tâm

Đây là mục ít được biết nhất và có giá trị cao nhất trong chương, và nó là câu trả lời cho Q2.

Vẽ một hình tròn bán kính R lên tấm marker. Ảnh của nó dưới phép chiếu phối cảnh là một ellipse — điều này đúng và nó là hệ quả của việc conic chiếu thành conic. Bây giờ câu hỏi: tâm của ellipse ấy có phải là ảnh của tâm hình tròn không?

Không. Và lý do nằm đúng ở mục 5: tâm của hình tròn là trung điểm của mọi dây cung đi qua nó, mà trung điểm *không* bảo toàn qua phép chiếu. Cụ thể hơn: xét một đường kính của hình tròn nằm theo hướng nghiêng ra xa camera. Nửa gần camera được phóng to nhiều hơn nửa xa, nên trên ảnh, ảnh của tâm bị đẩy về phía *xa*, trong khi tâm của ellipse — theo định nghĩa là trung điểm của các dây cung của chính ellipse — nằm ở chỗ khác.

Kết quả này là kiến thức chuẩn trong hiệu chuẩn camera chính xác [11], và Mallon cùng Whelan đã so sánh định lượng mức thiên lệch của các loại target khác nhau, cho thấy target tròn chịu thiên lệch mà target chessboard thì không [12]. Đây là một trong số ít chỗ trong cây mà tôi có được hai nguồn độc lập khẳng định cùng một điều, nên mức tin cậy cao.

Thiên lệch phụ thuộc vào ba thứ, và biết chúng là biết khi nào lo. Nó tăng theo *kích thước hình tròn so với khoảng cách* — hình tròn nhỏ ở xa thì thiên lệch nhỏ. Nó tăng theo *góc nghiêng* — fronto-parallel thì thiên lệch bằng không (một trong số ít chỗ mà fronto-parallel là tư thế tốt). Và nó tăng theo *độ lệch khỏi tâm ảnh*, vì tia nhìn tới một hình tròn ở rìa ảnh chếch nhiều hơn.

Cách chữa là dùng *đại số conic* thay vì lấy tâm ellipse. Ảnh của hình tròn là một conic C mà ta khớp được từ các điểm biên; từ C và K đã biết, có thể giải ra mặt phẳng chứa hình tròn và vị trí tâm thật của nó bằng cách phân tích trị riêng của $K^T C K$. Chi tiết nằm ngoài phạm vi chương này — B/08 bàn trong bối cảnh tinh chỉnh — nhưng điều cần nhớ ở đây là: *tồn tại cách chữa đúng, và nó bắt buộc nếu dùng marker tròn cho phép đo chính xác*.

CẦN NHỚ

Bias tâm ellipse là một *thiên lệch có hệ thống phụ thuộc tư thế*, không phải nhiễu. Ba hệ quả. Một, nó không giảm khi lấy trung bình nhiều khung. Nó thuộc loại nguồn “hệ thống” ở A/05 mục 8.

Hai, nó thay đổi khi robot di chuyển, vì góc nghiêng và vị trí trên ảnh đều đổi. Nghĩa là nó *không* triệt tiêu hoàn toàn trong sơ đồ teach-by-showing của B/13 — nó chỉ triệt tiêu ở đúng tư thế đã ghi, và sai lệch tăng dần khi robot lệch khỏi tư thế ấy.

Ba, và đây là điều đáng nói nhất: nó là một lý do thực chất để *ưu tiên góc chessboard* hơn *marker tròn* cho khâu đo tinh, dù marker tròn có độ chính xác định vị tâm rất cao. Một phép đo chính xác quanh một giá trị thiên lệch thì không bằng một phép đo kém chính xác hơn mà không thiên lệch — nhất là khi ngân sách của bạn là 5 mm và các nguồn hệ thống là những nguồn chiếm phần lớn ngân sách ấy.

2.7 Giới hạn và chế độ hỏng

Giả thiết ảnh đã khử méo. Toàn bộ chương giả định phép chiếu là pinhole thuần. Nếu còn méo, không tồn tại homography nào khớp được các điểm, và DLT sẽ trả về một ma trận “tốt nhất” trong nghĩa bình phương tối thiểu — tức một ma trận sai với residual không giải thích được. Dấu hiệu: residual của H có cấu trúc không gian (A/01 mục 6), và phép kiểm trực chuẩn ở mục 2 thất bại.

Giả thiết các điểm thực sự đồng phẳng. Nếu bốn góc tag không đồng phẳng do tấm marker cong, thì cũng không có homography nào khớp. Điều này quan trọng hơn vẻ ngoài: một tờ giấy in tag dán lên bề mặt cong lệch vài milimét khỏi mặt phẳng, và với tag 100 mm thì vài milimét là một sai lệch tương đối đáng kể. Đây là lý do 00-map nhấn mạnh nền cứng phẳng.

Giả thiết tương ứng điểm đúng. DLT giả định bạn biết điểm nào tương ứng điểm nào. Với tag có mã thì đúng — decode cho biết hướng. Nhưng nếu decode sai hướng (xoay 90 độ) thì bạn có bốn tương ứng sai và H sẽ là một phép quay 90 độ so với đúng, mà residual thì vẫn nhỏ vì hình vuông đối xứng. Đây là một chế độ hỏng thầm lặng và nó thuộc về B/07.

Homography chỉ áp dụng cho một mặt phẳng. Nếu constellation của bạn không đồng phẳng — và A/04 vừa khuyên mạnh rằng nó *nên* không đồng phẳng — thì không có một homography chung cho cả tập. Bạn có một homography cho mỗi mặt phẳng, và việc gộp chúng lại phải làm ở tầng PnP chứ không ở tầng homography. Đây là một giới hạn có cấu trúc, không phải một bất tiện: nó nói rằng homography là công cụ cho *từng tag*, còn công cụ cho *constellation* là PnP hợp nhất (B/09).

2.8 Thực hành

Ba việc.

Một, *thêm phép kiểm trực chuẩn vào pipeline*. Sau khi có H , tính $M = K^{-1}H$ và kiểm hai đại lượng: tỉ số $\|\mathbf{m}_1\|/\|\mathbf{m}_2\|$ (nên gần 1) và $|\mathbf{m}_1^T \mathbf{m}_2|/(\|\mathbf{m}_1\|\|\mathbf{m}_2\|)$ (nên gần 0). Ghi cả hai vào log. Chúng bắt được méo chưa khử, K sai, và marker không phẳng — ba lỗi mà không có chỉ số nào khác trong pipeline phát hiện được.

Hai, *đặt ngưỡng kích thước tag tối thiểu* và ghi nhận khi bị từ chối, thay vì âm thầm trả về một pose kém.

Ba, nếu dùng marker tròn, *hiệu chỉnh conic hoặc chấp nhận thiên lệch một cách có ý thức*. “Có ý thức” nghĩa là: ước lượng biên độ thiên lệch cho dải tư thế vận hành của bạn, đưa nó vào bảng ngân sách ở C/15 như một dòng hệ thống, và kiểm rằng tổng vẫn trong ngưỡng. Cách tệ nhất là không biết nó tồn tại.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Homography của mặt phẳng	$H = K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$; có được vì đặt $Z = 0$ làm cột $\mathbf{r}_3$ biến mất, biến phép chiếu 3D-2D thành biến đổi tuyến tính 2D-2D.
••• Bậc tự do của H	Tám (9 phần tử trừ 1 hệ số tỉ lệ). Pose có sáu. Hai bậc dư chính là hai ràng buộc trực chuẩn $\ \mathbf{r}_1\ = \ \mathbf{r}_2\ $ và $\mathbf{r}_1^\top \mathbf{r}_2 = 0$.
••• Dùng hai bậc dư làm gì	Phép kiểm miễn phí: tính $M = K^{-1}H$ và kiểm hai cột đầu có trực chuẩn không. Thất bại $\Rightarrow$ méo chưa khử, hoặc K sai, hoặc điểm không đồng phẳng.
•• Phân rã H thành pose	$\lambda = \frac{1}{2}(\ \mathbf{m}_1\ + \ \mathbf{m}_2\)$; $\mathbf{r}_i = \mathbf{m}_i/\lambda$; $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$; $\mathbf{t} = \mathbf{m}_3/\lambda$; rồi chiếu về $SO(3)$ bằng SVD, nhớ kiểm $\det = +1$ [2].
••• Phân rã H là khởi tạo, không phải kết quả	DLT cực tiểu sai số <i>đại số</i> , không phải sai số tái chiếu; các phép sửa λ và chiếu $SO(3)$ đảm bảo hợp lệ chứ không tối ưu; và nó không xử lý bài toán hai nghiệm. Luôn tinh chỉnh sau đó.
••• Chuẩn hoá Hartley	Đưa điểm về trung bình không, khoảng cách trung bình $\sqrt{2}$ trước DLT [9, §4.4]. Cải thiện số điều kiện vài bậc. Bỏ nó là lỗi cài đặt kinh điển, và triệu chứng là kém chính xác thất thường chứ không sai hẳn.
••• Vì sao tag nhỏ tệ hơn $1/w$	Hai hiệu ứng chồng nhau: σ chiếm tỉ lệ lớn hơn trong w (tuyến tính, đã có ở A/05), và bốn góc gần nhau làm DLT kém điều kiện nên khuếch đại thêm. Tích của hai hiệu ứng.
•• Ngưỡng kích thước tag	Thường 40–60 px cho cạnh, nhưng phải đo cho hệ mình: chạy Monte Carlo với w giảm dần, tìm chỗ sai số chệch khỏi đường $1/w$. Đây là lý do của chiến lược nested multi-scale ở B/09.
••• Bất biến duy nhất của phép chiếu	Tỉ số kép của bốn điểm thẳng hàng. <i>Không</i> bảo toàn: khoảng cách, tỉ số hai đoạn, trung điểm, song song, góc, diện tích. Có bảo toàn: thẳng hàng, đồng quy, bậc của conic.
••• Tâm chiếu $\neq$ chiếu của tâm	Hệ quả trực tiếp của việc trung điểm không bảo toàn. Với hình tròn, tâm của ellipse ảnh không phải ảnh của tâm hình tròn [11], [12].
••• Bias tâm ellipse phụ thuộc gì	Tăng theo kích thước hình tròn so với khoảng cách; tăng theo góc nghiêng (bằng 0 ở fronto-parallel); tăng theo độ lệch khỏi tâm ảnh.
••• Vì sao bias ellipse nguy hiểm	Nó là <i>thiên lệch hệ thống phụ thuộc tư thế</i> : không giảm theo $\sqrt{N}$, và không triệt tiêu hoàn toàn trong teach-by-showing vì nó đổi khi tư thế đổi. Đây là lý do thực chất để ưu tiên góc chessboard cho khâu đo tính.
•• Hệ quả thiết kế của tỉ số kép	Marker mã hoá bằng khoảng cách hoặc tỉ lệ hai đoạn không đọc được dưới góc nghiêng. Marker mã hoá bằng lưới ô rời rạc thì đọc được sau khi rectify — đó là lý do ArUco và AprilTag đều dùng lưới.
• Homography không dùng cho constellation không đồng phẳng	Mỗi mặt phẳng một H riêng; gộp phải làm ở tầng PnP. Giới hạn có cấu trúc: H là công cụ cho từng tag, PnP hợp nhất là công cụ cho constellation.

••• phải thuộc •• nên nắm • biết là có

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV findHomography	DLT có chuẩn hoá Hartley sẵn, kèm RANSAC tùy chọn	Đã làm đúng phần chuẩn hoá — đừng tự cài lại trừ khi có lý do
ArUco [10]	Rectify vùng tag về hình vuông chuẩn bằng H rồi lấy mẫu ô bit	Ứng dụng trực tiếp nhất của homography trong pipeline marker

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Hiệu chuẩn Zhang [2]	Lấy H từ mỗi ảnh target phẳng, rồi từ nhiều H giải ra K qua ràng buộc trục chuẩn	Dùng đúng hai bậc tự do dư ở mục 2 làm phương trình cho K
Heikkilä [11]	Hiệu chỉnh bias tâm bằng đại số conic cho target điểm tròn	Nguồn chuẩn cho mục 6; đọc nếu dùng marker tròn
CCTag [14], WhyCon [15]	Marker tròn đồng tâm; độ chính xác định vị tâm rất cao, nhưng phải xử lý bias	Minh hoạ đánh đổi ở mục 6: precision cao, bias khác không

MINI PROJECT · Đo bias tâm ellipse bằng mô phỏng, và so với góc chessboard · 1 ngày

Mục tiêu	biến “tâm chiếu khác chiều của tâm” từ một mệnh đề thành một con số milimét cho dải tư thế của bạn, để quyết định có dùng được marker tròn hay không.
Dữ liệu	mô phỏng thuần. Sinh một hình tròn bán kính R trên mặt phẳng marker bằng cách lấy mẫu dày các điểm trên biên; chiếu chúng bằng K và một pose đã biết.
Các bước	với mỗi tư thế, chiếu tập điểm biên ra ảnh; khớp một ellipse lên tập điểm chiếu; lấy tâm ellipse; so sánh với hình chiếu của tâm hình tròn thật (tính trực tiếp); ghi lại độ lệch tính bằng pixel; quét góc nghiêng từ 0° tới 60° , quét tỉ số R/Z , quét vị trí trên ảnh từ tâm ra rìa; cuối cùng quy độ lệch pixel thành milimét bằng hệ số Z/f của A/05.
Xong khi	bạn có một bảng ba chiều và phát biểu được một câu kiểu “với $R/Z < 0,05$ và góc nghiêng dưới 20° , bias tâm dưới X px tức Y mm, chấp nhận được”. Kiểm tra chéo bất buộc: ở 0° nghiêng, bias phải bằng không tới sai số số học — nếu không thì code khớp ellipse của bạn có lỗi, không phải bias có thật.
Nối với	B/08 cho cách hiệu chỉnh bằng conic; C/15 nhận con số milimét này như một dòng hệ thống trong ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Hai bậc dư chứa hai ràng buộc trục chuẩn: một ma trận 3×3 bất kỳ không nhất thiết có dạng $K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ với $\mathbf{r}_1, \mathbf{r}_2$ là hai cột của một ma trận quay, và hai điều kiện $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$ cùng $\mathbf{r}_1^\top \mathbf{r}_2 = 0$ chính là hai bậc bị loại khi đi từ H tổng quát về pose. Sự dư thừa ấy thành công cụ chẩn đoán vì nó cho ta một phép kiểm không tốn thêm dữ liệu: tính $M = K^{-1}H$ và xem hai cột đầu có thật sự trục chuẩn không. Nếu không, thì hoặc ảnh chưa khử méo đúng, hoặc K sai, hoặc các điểm không thực sự đồng phẳng — ba lỗi phổ biến nhất, và không có chỉ số nào khác trong pipeline bắt được cả ba.
- A2. Vì tâm hình tròn được định nghĩa qua trung điểm của các dây cung, mà phép chiếu phối cảnh không bảo toàn trung điểm: nửa đường kính gần camera được phóng to nhiều hơn nửa xa, nên ảnh của tâm bị đẩy lệch so với tâm hình học của ellipse ảnh [11]. Sai lệch phụ thuộc ba thứ: kích thước hình tròn so với khoảng cách (tỉ số R/Z càng lớn càng lệch), góc nghiêng (bằng không ở fronto-parallel, tăng theo góc), và độ lệch khỏi tâm ảnh. Điểm quan trọng: đây là *thiên lệch có hệ thống phụ thuộc tư thế*, nên nó không giảm khi lấy trung bình nhiều khung và nó không triệt tiêu hoàn toàn trong teach-by-showing.
- A3. Số điều kiện của bài toán DLT. Sai số của A/05 giả định bài toán được giải tốt và chỉ tính việc σ chiếm tỉ lệ ngày càng lớn trong w — một hiệu ứng tuyến tính. Nhưng khi bốn góc tag gần nhau trên ảnh, ma trận A trong $A\mathbf{h} = 0$ trở nên kém điều kiện, nên nó *khuếch đại* phần sai số ấy thêm một hệ số nữa. Hai hiệu ứng nhân nhau chứ không cộng, nên tổng thể tệ hơn $1/w$. Cách chữa không phải thuật toán tốt hơn mà là đảm bảo luôn có tag đủ lớn trên ảnh — tức chiến lược nested multi-scale.
- A4. Vì dữ liệu thử thường có thang đo hiện: toạ độ nhỏ, tag ở giữa ảnh, tư thế đơn giản. Trên dữ liệu thật, các phần tử của ma trận A trải qua nhiều bậc độ lớn (tích uX với $u \sim 10^3$ và $X \sim 10^{-1}$), nên số điều kiện tệ ngay từ cách dựng bài toán. Kiểu hỏng đặc biệt khó chịu: nó *không* sai hẳn — kết quả vẫn gần đúng — mà chỉ kém chính xác một cách thất thường, tốt ở tư thế này và tệ ở tư thế khác, không lặp lại được. Người ta sẽ đi tìm nguyên nhân ở detector, ở ánh sáng, ở camera, và không tìm ra, vì nguyên nhân nằm ở một dòng code tiền xử lý bị thiếu.

A5. Chỉ tỉ số kép. Ba cái kia đều không bảo toàn, và trung điểm không bảo toàn là hệ quả của việc tỉ số hai đoạn không bảo toàn. Câu trả lời quyết định thiết kế marker theo một cách rất cụ thể: bất kỳ sơ đồ mã hoá nào dựa trên khoảng cách hoặc trên tỉ lệ hai đoạn đều không đọc được tin cậy dưới góc nghiêng, còn sơ đồ dựa trên *cấu trúc rời rạc* — các ô sáng tối trong một lưới — thì đọc được, vì homography đưa lưới méo trở lại lưới đều và việc còn lại chỉ là hỏi mỗi ô sáng hay tối. Đó chính là lý do cả ArUco lẫn AprilTag đều chọn lưới ô, và đó cũng là lý do bước rectify bằng homography xuất hiện trong mọi pipeline marker.

NÓI LẠI THẬT ĐƠN GIẢN

Chiếu bóng một tấm bia lên tường bằng đèn pin. Nếu tấm bia song song với tường thì bóng là bản phóng to trung thực — hình vuông vẫn vuông, điểm giữa vẫn ở giữa. Nghiêng tấm bia đi thì bóng thành hình thang, và ở đây bắt đầu những chuyện bất ngờ.

Bất ngờ thứ nhất, và nó là tin tốt: dù nghiêng thế nào, có một công thức đơn giản biến toạ độ trên tấm bia thành toạ độ trên tường. Chỉ cần biết bốn điểm đi đâu là suy ra được mọi điểm khác. Đây là lý do máy tính đọc được cái tag dù nó nghiêng: nó “nắn” cái hình thang trở lại thành hình vuông rồi mới đọc các ô đen trắng.

Bất ngờ thứ hai là tin xấu, và ít người để ý. Điểm giữa của một cạnh trên tấm bia *không* chiếu thành điểm giữa của cạnh bóng. Nghe thì nhỏ nhặt, nhưng nó phá hỏng một thứ ta hay tin tưởng: cái tâm. Nếu bạn vẽ một vòng tròn lên tấm bia, bóng của nó là một hình bầu dục — và tâm của hình bầu dục ấy không phải bóng của tâm vòng tròn. Chúng lệch nhau. Càng nghiêng thì càng lệch.

Điều này quan trọng vì có những loại marker dùng vòng tròn thay vì hình vuông, và cách đo của chúng là tìm tâm cái hình bầu dục. Cách ấy đo được rất chính xác — tâm một hình bầu dục thì xác định sắc nét hơn nhiều so với góc một hình vuông. Nhưng nó chính xác quanh một chỗ hơi lệch, và chỗ lệch ấy thay đổi khi robot di chuyển. Đo chính xác quanh một chỗ sai thì không bằng đo hơi kém quanh chỗ đúng — nhất là khi cái sai ấy không lộ ra ở bất cứ con số nào.

Còn một chuyện nữa về kích thước. Khi cái tag còn to trên ảnh, việc tính toán rất ổn định. Khi nó co nhỏ lại, bốn góc xúm gần nhau, và việc suy ra hướng nghiêng từ bốn điểm sát nhau giống như đoán hướng của một cây kim từ việc nhìn hai đầu của nó — hai đầu càng gần thì đoán càng tệ, và tệ nhanh hơn bạn nghĩ. Đó là lý do người ta hay dán một cái tag to và một cái tag nhỏ cạnh nhau: xa thì đọc cái to, gần thì đọc cái nhỏ, và lúc nào cũng có một cái ở kích thước vừa phải.

Chỗ không rút gọn được: vì sao đúng *tỉ số kép* chứ không phải một đại lượng nào khác là bất biến duy nhất — chuyện này có chứng minh, nhưng nó thuộc hình học xạ ảnh và không rút gọn thành một câu trực quan được. Điều dùng được thì đơn giản: đừng tin bất cứ phép đo nào dựa trên khoảng cách hay điểm giữa, trừ khi đã nắn ảnh về đúng dạng trước.

Một câu: *Nắn được hình thang về hình vuông thì đọc được mọi thứ, nhưng cái tâm thì đừng tin — nó không nằm ở chỗ bạn tưởng.*

2.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *tâm của ellipse ảnh không phải ảnh của tâm hình tròn, và sai lệch tăng theo góc nghiêng* — là mệnh đề tôi tin ở mức cao vì nó qua cửa (c) với hai nguồn độc lập [11], [12] và qua cửa (a) bằng lập luận về trung điểm. Nó bị bác bỏ nếu mô phỏng của mini project cho ra bias bằng không ở mọi góc nghiêng — và nếu bạn thấy thế thì gần như chắc chắn code khớp ellipse của bạn đang có lỗi chứ không phải mệnh đề sai.

Mệnh đề thứ hai — *tag nhỏ làm pose xấu nhanh hơn $1/w$ do số điều kiện* — là mệnh đề yếu nhất về kiểm chứng trong chương, như đã ghi ở mục 4. Nó bị bác bỏ nếu Monte Carlo cho thấy sai số pose bám sát đường $1/w$ xuống tới kích thước tag rất nhỏ, chỉ xấu đi khi decode bắt đầu thất bại. Nếu vậy thì ngưỡng kích thước tối thiểu nên được đặt theo tiêu chí decode chứ không theo tiêu chí điều kiện số, và lời khuyên ở mục 4 cần sửa.

Mệnh đề thứ ba — *phép kiểm trực chuẩn bất được méo chưa khử, K sai, và điểm không đồng phẳng* — bị bác bỏ nếu có một trường hợp trong ba trường hợp ấy mà hai chỉ số trực chuẩn vẫn nằm trong ngưỡng bình thường. Tôi dẫn nó từ cấu trúc của công thức (của a) và chưa kiểm bằng mô phỏng cả ba trường hợp; đây là một phép kiểm rẻ và nên làm.

2.10 Kết nối

Chương này là chương công cụ, và nó cấp ba thứ cho phần còn lại của cây.

Nối lên trên. [A/01-mo-hinh-camera-va-distortion](#) là điều kiện tiên quyết theo nghĩa mạnh: nếu ảnh còn méo thì không tồn tại homography nào khớp, và mọi kết quả của chương này vô nghĩa. Phép kiểm trực chuẩn ở mục 2 chính là cách phát hiện tình trạng ấy, nên hai chương gắn với nhau theo cả hai chiều.

Nối ngang trong Phần A. [A/03-pnp-va-uoc-luong-tu-the](#) nhận phép phân rã H làm một trong các cách khởi tạo, nhưng nó cũng giải thích vì sao IPPE là lựa chọn tốt hơn cho target phẳng. [A/04-pose-ambiguity-target-phang](#) có gốc rễ ở đây: phép phân rã H vốn đã cho nhiều nghiệm, và chương 4 giải thích ý nghĩa hình học của chúng. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) nhận từ mục 4 phần bổ sung cho công thức $1/w$ của nó — chương 5 cho quan hệ tiệm cận, chương này cho biết quan hệ ấy hỏng ở đâu.

Nối xuống Phần B. [B/07-detection-pipeline](#) dùng homography ở khâu rectify, và mục 5 ở đây giải thích vì sao khâu ấy cần thiết: không rectify thì lưới ô không đều và việc đọc bit không tin cậy. [B/08-corner-refinement](#) nhận mục 6 làm cơ sở cho phần hiệu chỉnh conic của marker tròn, và nó cũng nhận luận điểm rằng góc chessboard không chịu thiên lệch này — một lý do thực chất cho lựa chọn target lai ở 00-map. [B/09-multi-marker-constellation](#) nhận từ mục 4 lý do định lượng cho chiến lược nested multi-scale, và từ mục 7 lý do vì sao constellation không đồng phẳng phải xử lý ở tầng PnP chứ không ở tầng homography.

Cuối cùng, nối sang [C/15-ngan-sach-sai-so](#): bias tâm ellipse ở mục 6 là một dòng *hệ thống* trong bảng ngân sách, và mini project của chương này là cách định giá dòng ấy bằng số.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Homography có tám bậc tự do, pose có sáu. Chỉ ra chính xác hai ràng buộc chiếm phần dư, và mô tả một phép kiểm dùng chúng bất được ba lỗi khác nhau.
2. Vì sao đặt $Z = 0$ lại biến một phép chiếu 3D–2D thành một biến đổi 2D–2D, và điều đó ngừng đúng khi nào?
3. Một đồng nghiệp bỏ bước chuẩn hoá Hartley. Mô tả kiểu hỏng — cụ thể là vì sao nó khó truy hơn một lỗi làm kết quả sai hẳn.
4. Vì sao sai số pose xấu đi nhanh hơn $1/w$ khi tag co nhỏ, và hai hiệu ứng nào đang nhân với nhau?
5. Phép chiếu phối cảnh bảo toàn đúng một bất biến trên đường thẳng. Nêu nó, và giải thích vì sao điều đó buộc mọi họ tag hiện đại phải mã hoá bằng lưới ô rời rạc.
6. Bạn dùng marker tròn vì nó cho σ nhỏ nhất trong mọi loại target. Nêu lý do có thể làm lựa chọn ấy vẫn sai cho bài toán 5 mm, và nêu điều kiện mà dưới đó nó lại đúng.
7. Constellation của bạn gồm ba tag trên ba mặt phẳng khác nhau. Bạn tính được mấy homography, và vì sao không gộp chúng lại ở tầng homography được?

Nguồn

- [1] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điển bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, “Propagating Covariance in Computer Vision,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, “Statistical Optimization for Geometric Computation: Theory and Practice,” 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., “Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera,” in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., “Rolling Shutter Camera Calibration,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, “Introduction to Fourier Optics,” 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

PnP và ước lượng tư thế (Perspective-n-Point)

PHỤ THUỘC A/01-mô-hình-camera-va-distortion (mô hình chiếu và ma trận K — PnP giả định cả hai đã biết chính xác);
A/02-homography-va-hình-học-phang (phân rã homography, một trong các cách khởi tạo).

TÓM TẮT

Chương này dựng cỗ máy trung tâm của cả cây: cho một tập điểm 3D đã biết toạ độ trong hệ vật và hình chiếu của chúng trên ảnh, tìm pose của camera. Bức tranh gồm hai tầng và tách bạch chúng là điều quan trọng nhất cần mang ra khỏi chương. Tầng thứ nhất là các lời giải *dạng đóng* — P3P, EPnP, DLT, IPPE — chúng nhanh, không cần khởi tạo, và *không* tối ưu theo tiêu chí có ý nghĩa. Tầng thứ hai là *tinh chỉnh phi tuyến* cực tiểu hoá sai số tái chiếu, và đây mới là nghiệm cuối cùng. Ba điều thực hành: với target phẳng hãy dùng IPPE vì nó cho cả hai nghiệm; với nhiều tag hãy chạy *một* PnP trên toàn bộ tập góc chứ đừng trung bình pose; và hãy trọng số hoá theo chất lượng từng góc thay vì coi mọi góc như nhau. Nếu chỉ nhớ một điều: *lời giải dạng đóng là để khởi tạo, tinh chỉnh mới là kết quả* — và Jacobian của bước tinh chỉnh chính là ma trận mà A/05 đảo để ra mọi công thức sai số của cây.

KIẾN THỨC CẦN CÓ TRƯỚC

- **PnP — Perspective-n-Point**: bài toán tìm pose camera từ n điểm 3D đã biết và hình chiếu 2D của chúng, với K đã biết. Ba điểm là số tối thiểu cho một tập nghiệm hữu hạn; bốn điểm thường đủ để chọn ra một nghiệm.
- **sai số tái chiếu (reprojection error)** — với một pose giả định ξ , chiếu điểm 3D ra ảnh được $\hat{u}_i(\xi)$, rồi lấy hiệu với góc đo được u_i . Đây là hàm mục tiêu có ý nghĩa vật lý vì nó đo đúng đại lượng mà nhiễu tác động lên — toạ độ pixel.
- **sai số đại số (algebraic error)** — đại lượng mà các lời giải tuyến tính như DLT cực tiểu hoá, thường có dạng $\|Ah\|$. Nó dễ giải vì tuyến tính, và nó *không* tỉ lệ với sai số tái chiếu — đó là toàn bộ lý do cần bước tinh chỉnh.
- **Gauss-Newton** — phương pháp lặp cho bình phương tối thiểu phi tuyến: tuyến tính hoá phần dư quanh ước lượng hiện tại, giải một hệ tuyến tính cho bước cập nhật, lặp lại. Nó xấp xỉ Hessian bằng $J^T J$, bỏ số hạng chứa đạo hàm bậc hai của mô hình.
- **Levenberg-Marquardt (LM)** — Gauss-Newton có thêm số hạng chính quy hoá λI trên đường chéo, nội suy giữa Gauss-Newton (khi λ nhỏ) và bước giảm gradient (khi λ lớn). Nó bền hơn khi khởi tạo xa hoặc khi $J^T J$ kém điều kiện.
- **tham số hoá tối thiểu của xoay** — pose có sáu bậc tự do nhưng R có chín phần tử với sáu ràng buộc. Để tối ưu trên $SO(3)$ cần một tham số hoá cục bộ sáu tham số — thường là vectơ xoay qua ánh xạ mũ, tức $se(3)$. Dùng góc Euler thay vào là nguồn của kỳ dị gimbal lock.
- **RANSAC — Random Sample Consensus**: chọn ngẫu nhiên con tối thiểu, giải, đếm số điểm phù hợp, lặp và giữ tập con tốt nhất [16]. Nó loại điểm ngoại lai chứ không cải thiện độ chính xác của điểm nội lai.

CÂU HỎI MỞ ĐẦU

- Q1. P3P với ba điểm cho tối đa bốn nghiệm hình học hợp lệ. Vì sao bốn mà không phải hai hay tám, và điều đó có ý nghĩa gì với việc bạn phải làm ở tầng code?

- Q2.** EPnP chạy nhanh, không cần khởi tạo, xử lý được hàng trăm điểm. Vì sao nó vẫn không đủ, và cụ thể đại lượng nào nó cực tiểu hoá mà không phải đại lượng bạn quan tâm?
- Q3.** Bạn có bốn tag, mỗi tag bốn góc. Có hai cách: bốn lần PnP rồi gộp, hoặc một lần PnP trên mười sáu góc. Chúng cho kết quả khác nhau đáng kể — vì sao, và cách nào đúng?
- Q4.** Bộ tối ưu của bạn dùng góc Euler để tham số hoá xoay. Nó chạy tốt trong thử nghiệm rồi thỉnh thoảng phân kỳ hoàn toàn ở một số tư thế. Cơ chế gì, và cách sửa đúng là gì?
- Q5.** Một tag ở xa và nghiêng mạnh, một tag ở gần và nhìn thẳng, cả hai đều nhìn thấy. Nếu bạn cho chúng trọng số như nhau trong PnP hợp nhất, bạn đang giả định gì — và giả định ấy sai theo chiều nào?

3.1 Đặt vấn đề

Toàn bộ cây này quy về một phép biến đổi: từ vài chục tọa độ pixel sang sáu con số pose. Chương này là chương mô tả phép biến đổi ấy.

Bài toán có một lịch sử dài hơn thị giác máy tính. Trường hợp ba điểm được Grunert giải năm 1841 trong bối cảnh trắc địa — xác định vị trí một trạm quan trắc từ ba mốc đã biết — và phân tích đầy đủ về số nghiệm của nó được Haralick và cộng sự tổng hợp lại một thế kỷ rưỡi sau [17]. Trường hợp n điểm tổng quát thì có nhiều lời giải hiện đại, mỗi lời giải đánh đổi khác nhau giữa tốc độ, độ bền và độ chính xác.

Điều đáng nói với người làm marker — và lý do chương này không chỉ là một danh mục thuật toán — là một sự nhầm lẫn rất phổ biến về *vai trò* của các thuật toán ấy. Người ta hay hỏi “dùng PnP nào tốt nhất”, như thể đó là một lựa chọn duy nhất. Câu hỏi ấy đặt sai, vì bức tranh đúng có hai tầng và chúng phục vụ hai mục đích khác nhau.

Tầng thứ nhất gồm các lời giải *dạng đóng*: P3P, EPnP, DLT, IPPE, SQPnP. Chúng giải một hệ đại số và cho ra nghiệm trong một số hữu hạn phép tính, không cần điểm khởi đầu. Ưu điểm rõ ràng; nhược điểm thì kín đáo hơn: chúng cực tiểu hoá một đại lượng đại số nào đó *không phải* sai số tái chiếu. Hệ quả là nghiệm chúng cho ra nói chung không phải nghiệm hợp lý cực đại, và độ lệch ấy đáng kể khi nhiễu không nhỏ.

Tầng thứ hai là *tinh chỉnh phi tuyến*: xuất phát từ một pose khởi tạo, lặp Gauss–Newton hoặc LM để cực tiểu hoá sai số tái chiếu. Đây là bước cho ra nghiệm hợp lý cực đại dưới giả thiết nhiễu Gauss, và đây là nghiệm duy nhất mà công thức hiệp phương sai của A/05 áp dụng được. Bỏ bước này là bỏ đi cả độ chính xác lẫn khả năng nói gì đó có cơ sở về bất định.

Cách hiển nhiên mà nhiều cài đặt thực tế làm — gọi một hàm PnP dạng đóng rồi dùng thẳng kết quả — do đó hỏng theo hai cách cùng lúc. Nó mất một phần độ chính xác, và nó mất luôn ma trận thông tin mà bước tinh chỉnh sinh ra như một sản phẩm phụ miễn phí. Cái thứ hai đắt hơn cái thứ nhất, vì không có ma trận thông tin thì không có trọng số để hợp nhất, không có cơ sở để loại ngoại lai theo chi-square, và không có gì để đưa vào ngân sách sai số.

Có một cách hiển nhiên thứ hai cũng hỏng, và nó phổ biến hơn: khi có nhiều tag, chạy PnP cho từng tag rồi gộp các pose lại. A/05 mục 7 đã nêu kết luận; mục 6 của chương này giải thích cơ chế bằng ngôn ngữ của bài toán tối ưu.

TRƯỚC GIÁC

Hình dung bài toán PnP như việc định vị bằng giác kế.

Bạn đứng ở một chỗ chưa biết và nhìn thấy vài cột mốc mà bạn biết chính xác toạ độ. Với mỗi cột mốc, bạn đo được *hướng* tới nó — không phải khoảng cách, chỉ hướng. Câu hỏi: bạn đang ở đâu và đang quay mặt về phía nào?

Mỗi cột mốc cho bạn một tia: “từ chỗ tôi đứng, cột mốc này nằm theo hướng đó”. Với một cột mốc thì vô số vị trí thoả mãn. Với hai thì bớt đi. Với ba thì — và đây là chỗ bất ngờ — vẫn còn tới bốn vị trí có thể, vì bài toán đại số ẩn sau nó là bậc bốn. Phải có cột mốc thứ tư để loại bớt.

Bây giờ thêm nhiều. Bạn đo hướng không chính xác tuyệt đối, nên các tia không giao nhau ở đúng một điểm — chúng tạo thành một chùm tia hơi lệch nhau. Không có vị trí nào thoả mãn *tất cả* các phép đo. Câu hỏi đổi thành: vị trí nào làm tổng bình phương độ lệch nhỏ nhất?

Và đây là chỗ hai tầng xuất hiện. Lời giải dạng đóng giống như việc dùng một công thức sẵn có để ra ngay một đáp án gần đúng — nhanh, và nó tối thiểu hoá một thứ gì đó tiện tính chứ không phải thứ bạn thật sự quan tâm. Tình hình giống như việc đứng ở đáp án ấy rồi dịch chuyển từng chút một theo hướng làm tổng lệch giảm, cho tới khi không giảm được nữa.

Điều đáng nói cuối: khi bạn dừng lại, độ *cong* của mặt tổng-lệch quanh chỗ bạn đứng cho bạn biết bạn chắc chắn tới mức nào. Dốc đứng theo mọi hướng thì vị trí xác định tốt; có một hướng gần phẳng thì theo hướng ấy bạn không biết gì cả. Đó chính là ma trận thông tin của $A/05$, và nó chỉ tồn tại nếu bạn đã làm bước tinh chỉnh.

3.2 P3P: ba điểm và bốn nghiệm

Trường hợp tối thiểu, và nó đáng hiểu vì nó là hạt nhân của RANSAC.

Cho ba điểm P_1, P_2, P_3 đã biết khoảng cách đôi một d_{12}, d_{13}, d_{23} , và ba hướng tia đơn vị $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ từ tâm camera tới chúng (tính được từ toạ độ ảnh và K^{-1}). Ẩn là ba khoảng cách s_1, s_2, s_3 từ tâm camera tới ba điểm.

Định lý cosin trong ba tam giác cho ba phương trình:

$$s_i^2 + s_j^2 - 2s_i s_j \cos \theta_{ij} = d_{ij}^2, \quad \cos \theta_{ij} = \mathbf{v}_i^\top \mathbf{v}_j.$$

Ba phương trình bậc hai, ba ẩn. Khử hai ẩn cho ra một đa thức bậc bốn theo ẩn còn lại, và đó là nguồn gốc của con số bốn trong “tối đa bốn nghiệm” — đây là câu trả lời cho Q1. Không phải hai vì hệ thực sự bậc bốn; không phải tám vì các nghiệm âm (điểm sau camera) bị loại bởi vật lý.

Sau khi có s_i , toạ độ ba điểm trong hệ camera là $s_i \mathbf{v}_i$, và pose thu được bằng cách khớp hai tập ba điểm — một bài toán Procrustes giải bằng SVD.

Ý nghĩa thực hành của “bốn nghiệm” là rất cụ thể: *bạn phải có điểm thứ tư để chọn*. Quy trình chuẩn là dùng ba điểm giải P3P, được tới bốn ứng viên, rồi chiếu điểm thứ tư bằng từng ứng viên và giữ ứng viên cho sai số tái chiếu nhỏ nhất. Đây là lý do mọi cài đặt RANSAC cho PnP dùng tập con bốn điểm chứ không phải ba.

Có nhiều công thức hoá khác nhau cho P3P, khác nhau ở độ ổn định số học và tốc độ; công thức của Kneip và cộng sự tránh được bước khớp Procrustes bằng cách xây trực tiếp một hệ toạ độ trung gian [18]. Với mục đích của cây thì chi tiết ấy không quan trọng — điều quan trọng là biết P3P tồn tại, nó cho nhiều nghiệm, và vai trò của nó là *hạt nhân tối thiểu cho RANSAC*, không phải bộ giải chính.

CẠM BÃY

Đừng nhầm bốn nghiệm của P3P với hai nghiệm của target phẳng ở A/04. Chúng là hai hiện tượng khác nhau và cách xử lý khác nhau.

Bốn nghiệm của P3P là hệ quả của việc *thiếu dữ liệu*: ba điểm không đủ để xác định sáu bậc tự do một cách duy nhất. Thêm một điểm thứ tư (không suy biến) là loại được ba trong bốn, và vấn đề biến mất hoàn toàn.

Hai nghiệm của target phẳng là hệ quả của một *đối xứng hình học* và nó *không* biến mất khi thêm điểm — miễn là các điểm thêm vào vẫn đồng phẳng. Đó là một loại bệnh hoàn toàn khác, chữa bằng cách phá tính đồng phẳng chứ không bằng cách thêm dữ liệu.

Hệ quả cài đặt: một tag vuông đơn lẻ có bốn góc, đủ để loại bớt nghiệm P3P, nhưng *vẫn* có hai nghiệm phẳng. Nếu bạn thấy hai nghiệm sau khi đã dùng cả bốn góc thì đó không phải bug.

3.3 Các lời giải dạng đóng cho n điểm

Bốn họ, mỗi họ một ý tưởng.

DLT. Viết phương trình chiếu thành hệ tuyến tính theo mười hai phần tử của $[R|t]$, bỏ qua ràng buộc trực giao, giải bằng SVD, rồi chiếu về $SO(3)$ sau. Đơn giản nhất, và cũng kém chính xác nhất, vì nó “quên” sáu ràng buộc rồi sửa sau. Nó cũng không dùng được K đã biết một cách hiệu quả. Vai trò thực tế: giá trị giáo dục, và khởi tạo khi không có gì tốt hơn.

EPnP. Ý tưởng đẹp: biểu diễn n điểm 3D bằng tổ hợp affine của bốn “điểm điều khiển ảo”, rồi giải cho toạ độ camera của bốn điểm ảo ấy thay vì cho n điểm. Bài toán rút gọn về kích thước cố định bất kể n , nên độ phức tạp là $O(n)$ [19]. Đây là lựa chọn tốt cho bài toán có nhiều điểm.

IPPE. Chuyên cho target phẳng. Nó khai thác cấu trúc phẳng bằng cách phân tích phép biến đổi *vi phân* quanh mỗi điểm, và điều quan trọng nhất với ta: nó trả về *cả hai* nghiệm cùng sai số tái chiếu của từng nghiệm [13]. Với marker vuông — tức gần như toàn bộ bài toán của cây — đây là lựa chọn đúng, và biến thể cho tag vuông có trong OpenCV dưới tên SOLVEPNP_IPPE_SQUARE.

SQPnP và các phương pháp tối ưu toàn cục. Giải bài toán bình phương tối thiểu trên $SO(3)$ bằng lập trình bậc hai tuần tự, đảm bảo tìm được cực tiểu toàn cục của một hàm mục tiêu đại số [20]. Chúng khắc phục nhược điểm “không tối ưu” của các phương pháp trước ở mức đại số, nhưng vẫn không phải nghiệm tối ưu theo sai số tái chiếu.

Bảng 3.1. Bốn họ lời giải dạng đóng: ý tưởng, ưu nhược, và vai trò trong pipeline marker.

Phương pháp	Ý tưởng	Điểm yếu	Vai trò cho marker
P3P	Định lý cosin, đa thức bậc bốn	Tối đa 4 nghiệm, cần điểm thứ tư để chọn	Hạt nhân RANSAC; không dùng làm bộ giải chính
DLT	Bỏ ràng buộc trực giao, giải tuyến tính	Kém chính xác nhất; không tận dụng K	Khởi tạo dự phòng; giá trị giáo dục
EPnP [19]	Bốn điểm điều khiển ảo, $O(n)$	Cực tiểu sai số đại số, không phải tái chiếu	Tốt khi nhiều điểm và không đồng phẳng
IPPE [13]	Khai thác cấu trúc phẳng, phân tích vi phân	Chỉ cho target phẳng	Lựa chọn mặc định cho marker vuông — cho cả hai nghiệm

Ghi chú trung thực: tôi không đưa vào chương này các con số so sánh tốc độ hay độ chính xác giữa các thuật toán, dù chúng có sẵn trong các bài báo gốc. Lý do là những con số ấy được đo trên các cấu hình rất khác nhau — số điểm, mức nhiễu, phân bố hình học — và trích chúng ra khỏi ngữ cảnh thì chúng không so sánh được với nhau. Điều tôi khẳng định ở đây chỉ là các *tính chất cấu trúc*: IPPE cho cả hai nghiệm của target phẳng, EPnP là $O(n)$, P3P cho tối đa bốn nghiệm. Ba điều ấy là tính chất toán học chứ không phải kết quả đo.

3.4 Tinh chỉnh: nghiệm thật sự

Đây là bước quyết định, và nó là bước cấp Jacobian cho A/05.

Hàm mục tiêu là sai số tái chiếu có trọng số:

$$C(\xi) = \frac{1}{2} \sum_{i=1}^N \mathbf{r}_i(\xi)^\top \Sigma_i^{-1} \mathbf{r}_i(\xi), \quad \mathbf{r}_i(\xi) = u_i - \hat{u}_i(\xi). \quad (3.1)$$

Đây là hàm hợp lý âm (negative log-likelihood) dưới giả thiết nhiễu Gauss độc lập trên toạ độ pixel, nên nghiệm của nó là ước lượng hợp lý cực đại — đó là lý do nó đáng làm, chứ không phải vì nó “chính xác hơn” một cách mơ hồ.

A. Tham số hoá xoay: vì sao phải dùng $\mathfrak{se}(3)$

Trước khi lấy đạo hàm, phải quyết định biểu diễn ξ thế nào. Đây là chỗ Q4 chỉ tới và nó là một lỗi cài đặt hay gặp.

Vấn đề: R có chín phần tử nhưng chỉ ba bậc tự do. Tối ưu trực tiếp trên chín phần tử với sáu ràng buộc là phiền; tối ưu trên góc Euler thì gặp **khóa gimbal** (*gimbal lock*) — ở một số tư thế, hai trong ba trục trùng nhau và Jacobian mất hạng. Triệu chứng đúng như Q4 mô tả: chạy tốt phần lớn thời gian rồi phân kỳ ở một số tư thế cụ thể, lặp đi lặp lại ở đúng những tư thế ấy.

Cách đúng là dùng một *tham số hoá cục bộ*: giữ ước lượng hiện tại R_k dưới dạng ma trận (hoặc quaternion), và tham số hoá *số gia* bằng một vectơ ba chiều ω qua ánh xạ mũ:

$$R_{k+1} = R_k \exp(\omega^\wedge),$$

với $\cdot^\wedge$ là toán tử đưa vectơ ba chiều thành ma trận phản đối xứng. Tại mỗi bước lặp ta tối ưu trên ω quanh không, nơi tham số hoá không có kỳ dị, rồi cập nhật và đặt lại. Đây là ý tưởng chuẩn của tối ưu trên đa tạp, và một tài liệu tham chiếu gọn cho nó là [21].

B. Bước Gauss–Newton

Tuyến tính hoá $\mathbf{r}_i$ quanh ξ_k :

$$\mathbf{r}_i(\xi_k + \delta) \approx \mathbf{r}_i(\xi_k) - J_i \delta, \quad J_i = \left. \frac{\partial \hat{u}_i}{\partial \xi} \right|_{\xi_k}.$$

Thay vào (3.1), lấy đạo hàm theo δ và cho bằng không:

$$\left(\sum_i J_i^\top \Sigma_i^{-1} J_i \right) \delta = \sum_i J_i^\top \Sigma_i^{-1} \mathbf{r}_i. \quad (3.2)$$

Đây là *phương trình chuẩn*, và ma trận bên trái là chính xác ma trận thông tin $\mathcal{I}$ mà A/05 đảo để ra hiệp phương sai. Điểm đáng nhấn mạnh: $\mathcal{I}$ không phải một thứ phải tính thêm — nó đã có sẵn như một sản phẩm phụ của mỗi bước lặp. Bỏ qua nó là vứt đi thông tin miễn phí.

Với sáu ẩn, hệ (3.2) là 6×6 — giải trong micro giây. Đây cũng là lý do PnP tinh chỉnh rẻ đến mức không có lý do gì để bỏ qua: vài chục bước lặp trên một hệ 6×6 là không đáng kể so với chi phí chạy detector.

LM thêm số hạng $\lambda \text{diag}(\mathcal{I})$ vào về trái, điều chỉnh λ theo mức độ thành công của từng bước. Nó bền hơn khi khởi tạo xa hoặc khi $\mathcal{I}$ kém điều kiện — tức đúng các tình huống mà A/02 mục 4 cảnh báo.

C. Jacobian của phép chiếu

Để hoàn chỉnh, đây là mảnh ghép cuối. Với điểm trong hệ camera $\mathbf{X}_c = (X, Y, Z)$, phép chiếu là $\hat{u} = fX/Z + c_x$, $\hat{v} = fY/Z + c_y$. Đạo hàm theo $\mathbf{X}_c$:

$$\frac{\partial \hat{u}}{\partial \mathbf{X}_c} = \frac{f}{Z} \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}.$$

Nhìn ma trận này một lát thì thấy ngay nguồn gốc của mọi kết luận ở A/05: hệ số chung là f/Z , giải thích vì sao sai số ngang là $Z\sigma/f$; và cột thứ ba — đạo hàm theo chiều sâu — mang thêm hệ số X/Z và Y/Z , tức *tỉ lệ với độ lệch khỏi trục quang*. Một điểm nằm đúng trên trục quang có $X = Y = 0$ nên đạo hàm theo Z bằng không: dịch nó ra xa hay lại gần không đổi hình chiếu chút nào. Đây chính là cơ chế sâu xa vì sao chiều sâu khó đo, và vì sao nó chỉ đo được nhờ các điểm *cách xa trục quang* — tức nhờ kích thước tag.

Phần còn lại của Jacobian là đạo hàm của $\mathbf{X}_c$ theo ξ , và nó có dạng chuẩn $[I \quad -\mathbf{X}_c^\wedge]$ với tham số hoá $\mathfrak{se}(3)$. Ghép lại bằng quy tắc chuỗi.

CẦN NHỚ

Ba lý do bắt buộc phải tinh chỉnh, không phải một.

Một, độ chính xác: lời giải dạng đóng cực tiểu sai số đại số, tinh chỉnh cực tiểu sai số tái chiếu — chỉ cái sau là ước lượng hợp lý cực đại.

Hai, và đây là lý do hay bị quên: tinh chỉnh sinh ra ma trận thông tin $\mathcal{I} = \sum J^\top \Sigma^{-1} J$ miễn phí. Không có nó thì không có hiệp phương sai để hợp nhất, không có cơ sở loại ngoại lai theo chi-square, và không có gì đưa vào ngân sách sai số.

Ba, chi phí gần như bằng không: một hệ 6×6 giải vài chục lần, không đáng kể so với detector.

Kết luận thực hành: `solvePnPRefineLM` hoặc tương đương phải luôn có trong pipeline, ngay sau lời giải dạng đóng.

3.5 PnP hợp nhất trên nhiều tag

Đây là mục quan trọng nhất của chương với bài toán docking, và nó trả lời Q3.

Tình huống: M tag, mỗi tag bốn góc, tọa độ 3D của tất cả $4M$ góc đã biết *trong cùng một hệ tọa độ dock*. Câu hỏi: tìm pose của camera so với hệ dock.

Cách đúng là chạy (3.1) với $N = 4M$, tức một bài toán tối ưu duy nhất trên toàn bộ tập góc. Không có gì đặc biệt về mặt thuật toán — vẫn là (3.2) với tổng chạy qua nhiều số hạng hơn. Cái đặc biệt nằm ở chỗ *thông tin hình học được sử dụng*.

Để thấy rõ, so sánh hai cách bằng ngôn ngữ của ma trận thông tin.

Cách hợp nhất cộng các ma trận thông tin trong cùng một hệ tọa độ:

$$\mathcal{I}_{\text{hợp nhất}} = \sum_{m=1}^M \sum_{i \in \text{tag } m} J_i^T \Sigma_i^{-1} J_i.$$

Các J_i ở đây được tính với tọa độ 3D thật của từng góc trong hệ dock, nên chúng *biết* rằng tag 1 nằm cách tag 2 một khoảng B .

Cách trung bình pose tính M pose riêng, mỗi pose có hiệp phương sai $\Sigma_m = \mathcal{I}_m^{-1}$ tính trong hệ của riêng tag ấy, rồi gộp. Ngay cả khi gộp đúng cách (trung bình có trọng số theo nghịch đảo hiệp phương sai, trên đa tạp), kết quả vẫn khác, vì phép nghịch đảo và phép tổng không giao hoán:

$$\left(\sum_m \mathcal{I}_m \right)^{-1} \neq \left(\sum_m \mathcal{I}_m^{-1} \right)^{-1} \dots$$

và quan trọng hơn cả bất đẳng thức đại số: khi tính $\mathcal{I}_m$ riêng cho từng tag, ta đã *vứt bỏ* ràng buộc rằng các tag gắn cứng với nhau. Ràng buộc ấy chính là thứ tạo ra cơ chế baseline của A/05 mục 6.

Lưu ý điểm này là trường hợp riêng của một nguyên lý tổng quát trong hiệu chỉnh chùm tia: *cộng thông tin, đừng cộng ước lượng* [22, §2]. Nó xuất hiện ở khắp nơi trong ước lượng trạng thái, và nó luôn nói cùng một điều: gộp ở tầng phần dư thì giữ được cấu trúc, gộp ở tầng kết quả thì mất.

CẠM BẮY

Có một điều kiện tiên quyết cho PnP hợp nhất mà nếu không thoả thì nó *tệ hơn* trung bình pose: *tọa độ 3D của các tag trong hệ dock phải đúng*.

Lý do: PnP hợp nhất tin vào mô hình constellation và ép một pose duy nhất giải thích mọi góc. Nếu mô hình sai — tag 2 thực ra cách tag 1 là 398 mm chứ không phải 400 mm như CAD — thì bộ giải phải thoả hiệp, và nó phân bổ sai lệch ấy vào pose theo một cách không kiểm soát được. Trung bình pose thì “rộng lượng” hơn: mỗi pose riêng vẫn đúng trong hệ của tag nó, và sai lệch constellation chỉ làm các pose tản mát.

Hệ quả: *PnP hợp nhất chỉ tốt hơn nếu bạn đã đo constellation cẩn thận*, và đó là lý do B/12 đặt bước hiệu chuẩn constellation bằng bundle adjustment ở mức bắt buộc chứ không phải tùy chọn.

May mắn là có một chỉ báo: nếu sai số tái chiếu sau khi hợp nhất lớn hơn hẳn so với khi giải từng tag riêng, thì mô hình constellation của bạn sai. Đây là một phép thử miễn phí và nó đáng chạy định kỳ.

3.6 Trọng số: không phải góc nào cũng như nhau

Công thức (3.1) có Σ_i cho từng góc, và phần lớn cài đặt đặt $\Sigma_i = \sigma^2 I$ như nhau cho mọi góc. Mục này trả lời Q5: giả định ấy sai theo chiều nào, và sửa thế nào.

Bốn nguồn làm Σ_i khác nhau giữa các góc.

Khoảng cách và kích thước biểu kiến. Một tag ở xa có kích thước nhỏ trên ảnh; các cạnh của nó ngắn hơn nên đường thẳng khớp lên chúng kém xác định hơn, nên góc kém chính xác hơn. Đây là nguồn lớn nhất trong bài toán docking vì các tag trong constellation có thể ở các khoảng cách rất khác nhau khi robot tới gần.

Góc nghiêng. Một tag nhìn rất chếch có các cạnh bị nén, và số pixel dọc theo cạnh ít đi, nên định vị kém hơn. Thêm nữa, cạnh bị nén có gradient thay đổi theo cách khác, làm phép khớp đường kém ổn định.

Vị trí trên ảnh. Méo còn sót lại lớn hơn ở rìa (A/01), và nó thể hiện như một thành phần thiên lệch — không phải nhiễu, nhưng nếu ta không mô hình hoá được nó thì cách thực dụng là tăng Σ cho các góc ở rìa để giảm ảnh hưởng của chúng.

Điều kiện quang học cục bộ. Một góc nằm trong vùng bị loá, hoặc bị mờ do độ sâu trường ảnh, hoặc có tương phản thấp, thì kém chính xác hơn (A/06, B/14).

Hai cách hiện thực hoá. Cách đơn giản là gán Σ_i theo một quy tắc heuristic — chẳng hạn tỉ lệ nghịch với kích thước tag trên ảnh — và điều đó đã tốt hơn nhiều so với trọng số đều. Cách đúng hơn là để chính detector ước lượng bất định của mỗi góc: bước khớp đường thẳng ở B/08 sinh ra một hiệp phương sai cho mỗi góc như sản phẩm phụ, và nó mang cả thông tin về *hướng* — một góc trên một cạnh dài và sắc nét được xác định tốt theo hướng vuông góc cạnh và kém hơn theo hướng dọc cạnh, tức Σ_i dị hướng.

Khung xử lý dị hướng đúng cách đã được đưa ra trong MLPnP [23], và ý tưởng của nó áp dụng thẳng vào đây: dùng hiệp phương sai thật của từng phép đo thay vì một hằng số. Lợi ích không chỉ là độ chính xác mà còn là *tính trung thực của hiệp phương sai đầu ra* — nếu Σ_i đầu vào đúng thì T^{-1} đầu ra mới là một ước lượng bất định đáng tin, và cả chuỗi hợp nhất ở B/11 phụ thuộc vào điều đó.

3.7 Loại ngoại lai

Bình phương tối thiểu rất nhạy với điểm ngoại lai: một góc sai 50 px đóng góp vào hàm mục tiêu gấp 250^2 lần một góc sai 0,2 px, nên nó kéo nghiệm về phía nó.

Trong bài toán marker, ngoại lai đến từ đâu? Không nhiều như trong bài toán đặc trưng tự nhiên — đó là một lợi thế lớn của marker. Nhưng vẫn có: một tag bị che một phần cho góc sai; một tag bị bắn hoặc mòn cho decode đúng nhưng góc lệch; một tag decode nhầm hướng cho bốn tương ứng xoay 90 độ; và một tag trong constellation bị lắp sai vị trí so với mô hình cho một sai lệch có hệ thống trông như ngoại lai.

Ba lớp công cụ, dùng theo thứ tự.

RANSAC ở mức tag, không ở mức góc. Vì bốn góc của một tag hoặc cùng đúng hoặc cùng sai, việc lấy mẫu ngẫu nhiên ở mức góc là lãng phí. Lấy mẫu ở mức tag: chọn ngẫu nhiên hai tag, giải PnP, đếm số tag phù hợp, lặp [16]. Rẻ hơn và đúng cấu trúc hơn.

Cổng chi-square trên phần dư. Sau khi hội tụ, phần dư chuẩn hoá $\mathbf{r}_i^\top \Sigma_i^{-1} \mathbf{r}_i$ theo lý thuyết có phân bố χ^2 hai bậc tự do. Loại các góc vượt quá một phân vị — chẳng hạn 99% — rồi giải lại. Đây là phép thử có cơ sở thống kê chứ không phải một ngưỡng tùy ý, và nó đòi Σ_i đúng — một lý do nữa để làm mục 6 cho tử tế.

Hàm mất mát bền (robust loss). Thay bình phương bằng Huber hoặc Cauchy, làm giảm ảnh hưởng của phần dư lớn mà không loại hẳn. Ưu điểm so với loại cứng: liên tục, nên không gây nhảy khi một điểm dao động quanh ngưỡng. Nhược điểm: nó làm hiệp phương sai đầu ra khó diễn giải hơn.

CẦN NHỚ

Có một phép kiểm nhất quán riêng cho constellation mà không hệ nào tự làm và nó rất đáng thêm: *giải PnP riêng cho từng tag, rồi so các pose với nhau.*

Nếu constellation đúng và mọi tag lành, các pose ấy phải nhất quán trong phạm vi hiệp phương sai của chúng. Nếu pose từ tag A và tag B lệch nhau quá ngưỡng, thì một trong ba điều đã sai: mô hình constellation sai (tag bị lắp lệch, hoặc CAD sai), một tag bị che hoặc bắn, hoặc một tag đang ở nghiệm ambiguity khác.

Phép kiểm này không thay thế PnP hợp nhất — nó chạy *song song* với nó như một giám sát. Chi phí thấp, và nó bắt được đúng loại lỗi mà sai số tái chiếu của bài toán hợp nhất có thể che giấu khi số tag lớn.

3.8 Giới hạn và chế độ hỏng

Giả thiết K và hệ số méo đúng. PnP coi chúng là hằng số đã biết. Nếu chúng sai thì PnP vẫn hội tụ, sai số tái chiếu vẫn nhỏ, và pose sai — không có tín hiệu nào. Đây là lý do A/01 nằm trước chương này và B/12 nằm ở mức xương sống.

Giả thiết tọa độ 3D đúng. Như mục 5 đã nêu, PnP hợp nhất khuếch đại hậu quả của sai số constellation. Chỉ báo: so sai số tái chiếu giữa hợp nhất và từng tag.

Giả thiết tương ứng đúng. PnP giả định biết góc nào ứng với điểm 3D nào. Với tag có mã thì decode giải quyết, nhưng decode sai hướng cho một chế độ hỏng thâm lặn: pose sai đúng 90 độ và sai số tái chiếu vẫn nhỏ vì hình vuông đối xứng bốn lần. Phép bắt: kiểm tính nhất quán với tag khác trong constellation.

Tối ưu cục bộ. Tinh chỉnh hội tụ về cực tiểu gần nhất, không phải cực tiểu toàn cục. Với target phẳng thì có đúng hai cực tiểu (A/04), và khởi tạo quyết định bạn rơi vào cái nào. Đây là lý do khởi tạo bằng IPPE — vốn tìm cả hai — tốt hơn khởi tạo bằng bất cứ thứ gì khác.

Giả thiết nhiễu Gauss. Hàm mục tiêu bình phương là hợp lý cực đại chỉ dưới giả thiết này. Với nhiễu có đuôi nặng (ngoại lai), nó không còn tối ưu — đó là lý do mục 7 tồn tại.

3.9 Thực hành

Pipeline khuyến nghị, năm bước, và nó là công thức tôi khuyên dùng làm mặc định.

Một, khử méo tọa độ góc (hoặc đưa méo vào hàm chiếu — A/01 mục 7).

Hai, khởi tạo bằng IPPE cho target phẳng, lấy cả hai nghiệm và tỉ số sai số tái chiếu ρ của chúng. Với constellation không đồng phẳng, dùng EPnP hoặc SQPnP.

Ba, tinh chỉnh LM trên toàn bộ tập góc của mọi tag nhìn thấy, với tọa độ 3D trong hệ dock và trọng số theo Σ_i từ mục 6.

Bốn, công chi-square trên phần dư, loại góc ngoại lai, giải lại nếu có loại.

Năm, xuất ra ba thứ chứ không phải một: pose, hiệp phương sai $\mathcal{I}^{-1}$, và ρ . Ba con số ấy là tối thiểu để tăng trên (B/11, B/13) làm việc có cơ sở.

Trong OpenCV, chuỗi ấy là `undistortPoints` $\rightarrow$ `solvePnPGeneric` với `SOLVEPNP_IPPE_SQUARE` $\rightarrow$ `solvePnPRefineLM`, cộng phần tính $\mathcal{I}$ tự viết (OpenCV không trả về nó, và đây là một thiếu sót đáng tiếc của API).

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Hai tầng của PnP	Dạng đóng (P3P, EPnP, IPPE, DLT) cực tiểu sai số <i>đại số</i> , không cần khởi tạo, nhanh. Tinh chỉnh phi tuyến cực tiểu sai số <i>tái chiếu</i> — đây mới là ước lượng hợp lý cực đại và là nghiệm cuối.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao P3P có tối đa 4 nghiệm	Khử hai trong ba phương trình định lý cosin cho một đa thức bậc bốn [17]. Cần điểm thứ tư để chọn — đó là lý do RANSAC cho PnP dùng tập con 4 điểm.
●●● 4 nghiệm P3P vs 2 nghiệm phẳng	Khác loại. 4 nghiệm P3P là do <i>thiếu dữ liệu</i> , biến mất khi thêm điểm. 2 nghiệm phẳng là do <i>đối xứng hình học</i> , không biến mất khi thêm điểm đồng phẳng (A/04).
●●● Ba lý do phải tinh chỉnh	Độ chính xác (hợp lý cực đại); ma trận thông tin $\mathcal{I} = \sum J^T \Sigma^{-1} J$ sinh ra <i>miễn phí</i> ; và chi phí gần bằng không (hệ 6×6).
●●● Phương trình chuẩn	$(\sum_i J_i^T \Sigma_i^{-1} J_i) \delta = \sum_i J_i^T \Sigma_i^{-1} \mathbf{r}_i$. Ma trận về trái chính là $\mathcal{I}$ mà A/05 đảo.
●●● Vì sao không dùng góc Euler	Khoá gimbal: ở một số tư thế hai trục trùng nhau, Jacobian mất hạng, bộ giải phân kỳ. Triệu chứng đặc trưng: hỏng lặp lại ở đúng vài tư thế cụ thể. Cách đúng: tham số hoá cục bộ $R_{k+1} = R_k \exp(\omega^\wedge)$ [21].
●●● Jacobian của phép chiếu	$\partial \hat{\mathbf{u}} / \partial \mathbf{X}_c = (f/Z) \begin{pmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{pmatrix}$. Hệ số chung f/Z giải thích $\delta x = Z \sigma / f$; cột thứ ba tỉ lệ với X/Z — điểm trên trục quang không cho thông tin chiều sâu nào.
●●● PnP hợp nhất vs trung bình pose	Hợp nhất cộng <i>thông tin</i> trong cùng hệ toạ độ, nên giữ được ràng buộc rằng các tag gắn cứng với nhau — tức giữ được cơ chế baseline. Trung bình pose cộng <i>ước lượng</i> , vứt bỏ ràng buộc ấy [22, §2].
●●● Điều kiện để hợp nhất tốt hơn	Toạ độ 3D constellation phải đúng. Nếu sai, hợp nhất <i>tệ hơn</i> trung bình vì nó ép một pose duy nhất giải thích mô hình sai. Chỉ báo: số sai số tái chiếu hợp nhất với từng tag riêng.
●● Bốn nguồn làm Σ_i khác nhau	Khoảng cách và kích thước biểu kiến (lớn nhất trong docking); góc nghiêng; vị trí trên ảnh (méo còn sót ở rìa); điều kiện quang học cục bộ.
●● Vì sao trọng số đúng quan trọng hơn độ chính xác	Vì $\mathcal{I}^{-1}$ chỉ là ước lượng bất định đáng tin nếu Σ_i đầu vào đúng, và cả chuỗi hợp nhất ở B/11 lẫn công chi-square ở mục 7 phụ thuộc vào tính đúng ấy [23].
●● RANSAC ở mức nào	Mức <i>tag</i> , không mức góc — vì bốn góc một tag cùng đúng hoặc cùng sai. Chọn ngẫu nhiên hai tag, giải, đếm tag phù hợp.
●● Công chi-square	Phần dư chuẩn hoá $\mathbf{r}^T \Sigma^{-1} \mathbf{r} \sim \chi_2^2$; loại góc vượt phân vị 99% rồi giải lại. Có cơ sở thống kê, nhưng <i>đòi</i> Σ đúng.
●● Phép kiểm nhất quán constellation	Giải PnP riêng từng tag rồi so các pose. Lệch quá ngưỡng $\Rightarrow$ constellation sai, tag bị che/bẩn, hoặc một tag đang ở nghiệm ambiguity khác. Chạy song song với hợp nhất như một giám sát.
● Chế độ hỏng thâm lặng nhất	Decode sai hướng: pose sai đúng 90° và sai số tái chiếu <i>vẫn nhỏ</i> vì hình vuông đối xứng bốn lần. Chỉ bắt được bằng kiểm nhất quán với tag khác.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenCV solvePnPGeneric	Trả về nhiều nghiệm kèm sai số tái chiếu từng nghiệm; hỗ trợ IPPE, EPnP, SQPnP, P3P	API quan trọng nhất của cả cây — xem D/17
OpenCV solvePnPRefineLM	Bước tinh chỉnh LM trên sai số tái chiếu	Bắt buộc có trong pipeline; rẽ
apriltag apriltag_pose.c	Cài IPPE cho tag vuông, tính cả hai nghiệm và tỉ số	Đọc để thấy mục 3 và A/04 thành code

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ceres [24], GTSAM	Tối ưu tổng quát với tham số hoá trên đa tạp và hàm mất mát bền	Dùng khi bài toán vượt quá một pose duy nhất — B/12
MLPnP [23]	PnP với hiệp phương sai dị hướng cho từng phép đo	Nguồn cho mục 6; ý tưởng áp thẳng vào trọng số góc marker

MINI PROJECT · So PnP hợp nhất với trung bình pose, và đo cái giá của constellation sai · 2 ngày

Mục tiêu	chuyển luận điểm trung tâm của mục 5 từ một lập luận thành một cặp đường cong, và đồng thời đo được ngưỡng sai số constellation mà trên đó hợp nhất bắt đầu tệ hơn.
Dữ liệu	mô phỏng, nối tiếp script của A/05 và A/04.
Các bước	dựng constellation bốn tag với baseline B , tọa độ 3D chân trị đã biết; chiếu, thêm nhiễu σ ; chạy hai đường: (i) PnP hợp nhất trên 16 góc, (ii) bốn PnP riêng rồi trung bình Karcher trên $SO(3)$ cộng trung bình có trọng số cho tịnh tiến; so sai số góc và sai số vị trí của hai cách qua 2000 lần chạy; lặp khi quét B . Sau đó phần thứ hai: cố ý làm sai mô hình constellation — dịch tọa độ 3D của một tag đi e mm so với chân trị — và quét e từ 0 tới 5 mm, xem cách nào xuống dốc nhanh hơn.
Xong khi	bạn có hai đồ thị. Đồ thị một cho thấy hợp nhất thẳng rõ về sai số góc và khoảng cách thẳng tăng theo B . Đồ thị hai cho thấy một giao điểm: với e vượt một ngưỡng nào đó, trung bình pose bắt đầu bền hơn. Con số ngưỡng ấy chính là yêu cầu độ chính xác cơ khí cho constellation của bạn. Kiểm tra chéo bắt buộc: ở $e = 0$ và B rất nhỏ, hai cách phải cho kết quả gần như nhau — nếu không thì phép trung bình Karcher của bạn có lỗi.
Nối với	B/09 và B/12 nhận con số ngưỡng này làm yêu cầu thiết kế; C/15 đưa nó vào bảng ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Bốn vì hệ ba phương trình định lý cosin, sau khi khử hai ẩn, cho một đa thức bậc bốn theo ẩn còn lại [17]; không phải hai vì hệ thực sự bậc bốn chứ không suy biến, và không phải tám vì các nghiệm ứng với điểm nằm sau camera bị vật lý loại. Ý nghĩa ở tầng code rất cụ thể: bạn *phải* có một điểm thứ tư để chọn trong bốn ứng viên, bằng cách chiếu nó qua từng ứng viên và giữ cái cho sai số tái chiếu nhỏ nhất. Đó là lý do mọi cài đặt RANSAC cho PnP lấy mẫu bốn điểm chứ không ba, dù ba mới là số tối thiểu để giải.
- A2. Vì EPnP cực tiểu hoá một sai số *đại số* — độ lệch trong hệ phương trình tuyến tính của bốn điểm điều khiển ảo — chứ không phải sai số tái chiếu tính bằng pixel [19]. Hai đại lượng ấy không tỉ lệ với nhau, nên nghiệm tối ưu theo cái này không tối ưu theo cái kia, và chênh lệch đáng kể khi nhiễu không nhỏ. Nhưng có một lý do thứ hai quan trọng không kém: EPnP không sinh ra ma trận thông tin. Không có $\mathcal{I}$ thì không có hiệp phương sai để hợp nhất ở B/11, không có cơ sở cho cổng chi-square, và không có gì đưa vào ngân sách sai số. Bước tinh chỉnh cho cả hai thứ với chi phí gần bằng không.
- A3. Khác nhau vì PnP hợp nhất *biết* rằng bốn tag gắn cứng với nhau ở những vị trí đã biết, còn trung bình pose thì không. Cụ thể: hợp nhất cộng các ma trận thông tin $J_i^\top \Sigma_i^{-1} J_i$ trong cùng một hệ tọa độ, với J_i tính từ tọa độ 3D thật của từng góc trong hệ dock — nên thông tin về hướng đến từ chênh lệch vị trí giữa các tag cách nhau B , tức cơ chế baseline của A/05. Trung bình pose tính hiệp phương sai riêng cho từng tag trong hệ của tag ấy, vớt bỏ ràng buộc liên tag, nên mỗi pose vẫn chịu hệ số $\sin \theta$ và trung bình chỉ giảm nhiễu theo $\sqrt{M}$. Cách đúng là hợp nhất — với một điều kiện: tọa độ 3D constellation phải đúng, nếu không thì hợp nhất lại tệ hơn.
- A4. Khoá gimbal. Góc Euler có kỳ dị ở một số tư thế nơi hai trục quay trùng nhau; tại đó ánh xạ từ ba tham số sang $SO(3)$ mất hạng, nên Jacobian mất hạng, nên phương trình chuẩn suy biến và bước cập nhật nổ tung. Triệu chứng đặc trưng và chẩn đoán được: nó hỏng *lặp lại ở đúng vài tư thế cụ thể* chứ không ngẫu nhiên — nếu bạn thấy phân kỳ luôn xảy ra quanh cùng một giá trị pitch thì gần như chắc chắn là chuyện này. Cách sửa đúng không phải thêm chính quy hoá

hay đổi ngưỡng mà là đổi tham số hoá: giữ R dưới dạng ma trận hoặc quaternion, tham số hoá số *gia* bằng vectơ ba chiều qua ánh xạ mũ, và đặt lại sau mỗi bước — tức tối ưu quanh gốc của $se(3)$ nơi không có kỳ dị [21].

- A5. Bạn đang giả định rằng mọi góc có cùng độ tin cậy, và giả định ấy sai theo chiều *nguy hiểm*: các góc kém nhất được cho cùng tiếng nói với các góc tốt nhất, nên chúng kéo nghiệm về phía chúng nhiều hơn mức đáng có. Cụ thể trong tình huống mô tả: tag ở xa có kích thước nhỏ trên ảnh nên cạnh ngắn nên định vị góc kém, và tag nghiêng mạnh có cạnh bị nén nên càng kém — cả hai hiệu ứng cộng lại có thể làm σ của nó lớn gấp nhiều lần tag gần. Hậu quả thứ hai, và nó tệ hơn hậu quả thứ nhất: hiệp phương sai $\mathcal{I}^{-1}$ mà bạn xuất ra sẽ *sai*, vì nó được tính từ Σ_i giả định. Nó sẽ báo một bất định nhỏ hơn thực tế, và mọi tầng phía trên dùng con số ấy làm trọng số sẽ bị lừa theo.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đứng giữa một cánh đồng và nhìn thấy vài cái cột mà bạn biết chính xác chúng ở đâu. Bạn không đo được khoảng cách tới chúng, chỉ đo được hướng: cột này ở bên trái một chút, cột kia ở bên phải nhiều hơn. Câu hỏi là bạn đang đứng ở đâu và quay mặt về hướng nào.

Một cái cột thì chưa nói được gì. Ba cái thì gần đủ — nhưng chỉ gần, vì hoá ra vẫn còn tới bốn chỗ mà từ đó ba cái cột trông đúng như vậy. Phải có cái cột thứ tư mới loại được ba chỗ giả.

Với phép đo hoàn hảo thì xong ở đó. Nhưng phép đo không hoàn hảo: bạn đo hướng lệch đi một chút, nên không có chỗ nào trên cánh đồng làm mọi hướng khớp cùng lúc. Bài toán đổi thành: tìm chỗ mà tổng độ lệch nhỏ nhất.

Và ở đây có hai cách làm, thường bị nhầm là một. Cách thứ nhất là dùng một công thức có sẵn để ra ngay một chỗ khá tốt — nhanh, nhưng công thức ấy tối thiểu hoá một thứ tiện tính chứ không phải chính cái tổng độ lệch mà bạn quan tâm. Cách thứ hai là đứng ở chỗ ấy rồi dịch từng chút theo hướng làm tổng lệch giảm, cho tới khi hết giảm được. Cách thứ hai mới cho câu trả lời đúng, và nó rẻ — vài chục bước tính toán nhỏ.

Nhưng cái quý nhất của cách thứ hai không phải câu trả lời. Khi bạn dừng lại, bạn biết thêm một thứ: mặt đất quanh chỗ bạn đứng *dốc* thế nào. Dốc đứng theo mọi hướng nghĩa là bạn rất chắc chắn — dịch đi một tí là tổng lệch tăng ngay. Có một hướng mà đi dọc theo nó gần như phẳng nghĩa là theo hướng ấy bạn hầu như không biết gì. Cái độ dốc đó chính là thước đo độ tin cậy, và nó có sẵn trong phép tính — ai bỏ bước dịch chuyển thì cũng mất luôn nó, và mất nó thì không còn gì để nói mình chắc chắn tới đâu.

Còn một chuyện nữa, về mấy cái cột. Nếu bạn biết chúng gắn cứng với nhau trên cùng một cái khung — cột này cách cột kia đúng bốn mươi phân — thì bạn nên dùng thông tin đó. Cách sai là tính riêng vị trí theo từng cột rồi lấy trung bình bốn kết quả; làm thế là vứt bỏ hiểu biết về cái khung. Cách đúng là để cả bốn cột cùng kéo về một lời giải duy nhất, vì khi ấy chính khoảng cách giữa chúng trở thành thước đo góc.

Chỗ không rút gọn được: cái giá của chuyện dùng khung là bạn phải *tin* vào khung. Nếu cái khung thật ra không đúng như bản vẽ — cột lệch đi vài milimét — thì cách dùng khung lại tệ hơn cách tính riêng, vì nó ép mọi thứ khớp với một mô hình sai. Ranh giới giữa hai chế độ ấy thì phải đo, không suy ra được.

Một câu: Công thức cho bạn chỗ gần đúng, việc dịch từng chút cho bạn chỗ đúng và cho bạn biết mình chắc tới đâu.

3.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — PnP hợp nhất trên toàn bộ tập góc tốt hơn trung bình pose từng tag, và khoảng cách thẳng tăng theo baseline — bị bác bỏ nếu mô phỏng ở mini project cho thấy hai cách cho cùng độ tản mát góc trên constellation trải rộng, với toạ độ 3D chính xác. Đây là mệnh đề tôi dẫn bằng lập luận thông tin (cửa a) và đối chiếu với nguyên lý tổng quát trong [22, §2] (cửa c một phần), và nó *chưa* qua cửa (b). Nó là mệnh đề có hậu quả kiến trúc lớn nhất trong chương nên nên kiểm sớm.

Mệnh đề thứ hai — *hợp nhất trở nên tệ hơn trung bình khi sai số constellation vượt một ngưỡng* — là một dự đoán định tính của tôi dựa trên cơ chế ở mục 5. Nó bị bác bỏ nếu mô phỏng cho thấy hợp nhất vẫn thắng ở mọi mức sai số constellation cho tới khi cả hai đều vô dụng. Nếu vậy thì lời cảnh báo ở mục 5 là quá mạnh và nên được nới.

Mệnh đề thứ ba — *tinh chỉnh cải thiện độ chính xác đáng kể so với lời giải dạng đóng* — bị bác bỏ nếu mô phỏng cho thấy chênh lệch nhỏ hơn nhiều so với σ ở mức nhiễu thực tế. Điều đáng nói: kể cả nếu mệnh đề này bị bác bỏ, lời khuyên “luôn tinh chỉnh” vẫn đứng vững, vì lý do thứ hai — ma trận thông tin miễn phí — không phụ thuộc vào nó.

Cuối cùng, một mệnh đề mà tôi đã cố ý *không* phát biểu: chương này không xếp hạng các thuật toán PnP theo độ chính xác, vì tôi không có dữ liệu so sánh được. Nếu bạn cần xếp hạng ấy cho hệ của mình thì nó phải đo trên chính cấu hình của bạn — và mini project cung cấp khung để làm việc đó.

3.11 Kết nối

Chương này là trục của Phần A: nó nhận mô hình từ hai chương trước và cấp Jacobian cho hai chương sau.

Nối lên trên. [A/01-mô-hình-camera-va-distortion](#) cấp hàm chiếu $\hat{u}(\xi)$ và cảnh báo rằng PnP coi K cùng hệ số méo là hằng số đúng — nếu chúng sai thì PnP hội tụ về một pose sai mà không có tín hiệu. [A/02-homography-va-hình-học-phang](#) cấp một cách khởi tạo, và cũng cấp lý do vì sao IPPE là lựa chọn tốt hơn phân rã H cho target phẳng.

Nối xuống dưới trong Phần A. [A/04-pose-ambiguity-target-phang](#) là hệ quả trực tiếp của mục 8: tối ưu là cực bộ, có hai cực tiểu, và khởi tạo quyết định bạn rơi vào cái nào. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) nhận *đúng* ma trận $\mathcal{I}$ từ phương trình chuẩn (3.2) và toàn bộ chương ấy chỉ làm một việc là đọc ý nghĩa vật lý của $\mathcal{I}^{-1}$; Jacobian ở mục 4C là chỗ nguồn gốc của bốn quan hệ tỉ lệ rõ nhất.

Nối xuống Phần B. [B/08-corner-refinement](#) cấp Σ_i mà mục 6 cần — không chỉ độ lớn mà cả hướng, vì hiệp phương sai của một góc từ giao hai đường thẳng vốn dị hướng. [B/09-multi-marker-constellation](#) là chương biến mục 5 thành thiết kế: nó trả lời đặt tag ở đâu để $\mathcal{I}$ có điều kiện tốt nhất. [B/12-calibration-cho-docking](#) nhận cảnh báo ở mục 5 làm lý do bắt buộc cho bước đo constellation bằng bundle adjustment. [B/11-uoc-luong-theo-thoi-gian](#) nhận $\mathcal{I}^{-1}$ làm trọng số khi hợp nhất theo thời gian — và nó chỉ làm được điều đó nếu bước tinh chỉnh đã chạy.

Cuối cùng, một nối sang [B/13-docking-control](#): mục 9 khuyến nghị xuất ra ba thứ (pose, hiệp phương sai, ρ) chứ không phải một, và lý do đầy đủ của khuyến nghị ấy nằm ở chương điều khiển — một bộ điều khiển docking cần biết không chỉ “mục tiêu ở đâu” mà cả “tôi tin điều đó tới mức nào” và “có phải tôi đang nhìn nhầm nghiệm không”.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao ba điểm cho bốn nghiệm chứ không phải hai, và vì sao đó là một loại đa nghiệm hoàn toàn khác với hai nghiệm của target phẳng?
2. Nêu ba lý do bắt buộc phải chạy tinh chỉnh sau lời giải dạng đóng, và chỉ ra lý do nào vẫn đứng vững kể cả khi cải thiện độ chính xác là không đáng kể.
3. Trong Jacobian của phép chiếu, cột thứ ba tỉ lệ với X/Z và Y/Z . Điều đó nói gì về khả năng đo chiều sâu của một điểm nằm trên trục quang, và nó nối với công thức nào ở [A/05](#)?
4. Một đồng nghiệp trung bình bốn pose từ bốn tag. Giải thích chính xác thông tin gì bị vứt bỏ, và nêu điều kiện mà dưới đó cách làm của họ lại *đúng hơn* cách hợp nhất.

5. Bộ giải của bạn phân kỳ ở đúng một vài tư thế và chạy tốt ở mọi tư thế khác. Nêu nguyên nhân khả dĩ nhất, cơ chế của nó, và cách sửa đúng.
6. Vì sao tổng chi-square trên phần dư đòi Σ_i đúng, và chuyện gì xảy ra với tỉ lệ loại nhầm nếu bạn dùng một σ chung quá nhỏ?
7. Decode sai hướng cho pose sai 90° mà sai số tái chiếu vẫn nhỏ. Vì sao, và cơ chế nào trong chương này phát hiện được nó?

Nguồn

- [1] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tính chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, “Propagating Covariance in Computer Vision,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, “Statistical Optimization for Geometric Computation: Theory and Practice,” 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., “Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera,” in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., “Rolling Shutter Camera Calibration,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, “Introduction to Fourier Optics,” 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

Pose ambiguity của target phẳng (Planar Pose Ambiguity)

PHỤ THUỘC A/02-homography-va-hình-hoc-phang (homography của một mặt phẳng và phép phân rã nó thành pose – chỗ hai nghiệm xuất hiện lần đầu); A/03-ppnp-va-uoc-luong-tu-the (IPPE và sai số tái chiếu – chương này dùng liên tục cả hai).

TÓM TẮT

Chương này giải thích hiện tượng mà gần như ai làm marker cũng gặp và ít người gọi đúng tên: pose nhảy qua lại giữa hai giá trị dù camera và tag gần như đứng yên. Nguyên nhân không phải nhiễu, không phải bug, và không chữa được bằng bộ lọc: với một target *phẳng*, bài toán PnP có *đúng hai* nghiệm cục bộ, và cả hai đều giải thích ảnh gần như tốt ngang nhau. Hai nghiệm ấy là ảnh gương của nhau qua một mặt phẳng, chúng có cùng vị trí tâm gần đúng nhưng hướng nghiêng ngược dấu, và khoảng cách giữa chúng theo góc có thể lên tới hàng chục độ. Ambiguity tệ nhất đúng ở tư thế fronto-parallel — tức đúng tư thế của một robot tiếp cận dock thẳng trục. Nếu chỉ nhớ một điều: *đây là một tính chất của hình học phẳng, không phải một khiếm khuyết của thuật toán*, nên cách chữa triệt để là phá tính phẳng chứ không phải lọc mạnh hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **target phẳng** — tập điểm điều khiển 3D cùng nằm trên một mặt phẳng. Một tag vuông đơn lẻ là target phẳng; bốn tag dán trên cùng tấm bảng cũng vẫn là target phẳng, dù có bao nhiêu điểm đi nữa. Đây là điểm quan trọng nhất của định nghĩa: *thêm điểm đồng phẳng không phá được tính phẳng*.
- **cực tiểu cục bộ (local minimum)** — một điểm mà hàm mục tiêu nhỏ hơn mọi điểm lân cận, nhưng không nhất thiết nhỏ nhất toàn cục. Bộ giải phi tuyến dừng ở cực tiểu cục bộ gần nhất với điểm khởi đầu, và nó không có cách nào biết còn cực tiểu nào khác ở xa.
- **sai số tái chiếu (reprojection error)** — tổng bình phương khoảng cách giữa góc đo được và góc dự đoán bởi một pose giả định, tính bằng pixel. Đây là hàm mục tiêu của PnP tinh chỉnh và là đại lượng duy nhất bộ giải “nhìn thấy”.
- **tỉ số ambiguity** — tỉ số giữa sai số tái chiếu của nghiệm tốt thứ hai và nghiệm tốt nhất, $\rho = e_2/e_1 \geq 1$. Gần 1 nghĩa là hai nghiệm giải thích ảnh tốt ngang nhau và không phân biệt được; lớn (chẳng hạn trên 3–5) nghĩa là nghiệm thứ nhất thắng rõ ràng.
- **IPPE — Infinitesimal Plane-based Pose Estimation**, thuật toán giải PnP cho target phẳng dạng đóng, và điểm đáng giá nhất của nó là nó trả về *cả hai* nghiệm kèm sai số tái chiếu của từng nghiệm [13].
- **đồng phẳng và không đồng phẳng** — tập điểm gọi là không đồng phẳng nếu không tồn tại một mặt phẳng chứa tất cả. Với tập điểm không đồng phẳng, bài toán PnP có nghiệm duy nhất — đây là kết quả trung tâm và là lối thoát của cả chương.

CÂU HỎI MỞ ĐẦU

- Q1. Camera và tag của bạn đứng yên hoàn toàn trên bàn. Pose trả về vẫn nhảy qua lại giữa hai giá trị cách nhau 20° vài lần mỗi giây. Nếu không phải nhiễu và không phải bug, thì đó là gì — và vì sao lấy trung bình hai giá trị ấy lại là việc tệ nhất có thể làm?
- Q2. Bạn dán thêm ba tag nữa lên cùng tấm bảng, tổng cộng bốn tag và mười sáu góc thay vì bốn. Bài toán ambiguity được cải thiện bao nhiêu, và vì sao câu trả lời là “gần như không”?
- Q3. Hai nghiệm của một target phẳng có quan hệ hình học rất cụ thể với nhau, không phải hai giá trị ngẫu nhiên. Quan hệ ấy là gì, và biết nó giúp gì cho việc chẩn đoán?
- Q4. Thuật toán nào cũng phải chọn một trong hai nghiệm. Nếu nó chọn bằng cách lấy nghiệm có sai số tái chiếu nhỏ hơn, thì trong trường hợp nào cách chọn ấy tệ hơn cả việc tung đồng xu?
- Q5. Có một cách khử ambiguity triệt để, không cần lọc, không cần thông tin thời gian, không cần cảm biến khác — và nó không nằm ở tầng phần mềm. Cách ấy là gì, và vì sao nó triệt để trong khi mọi cách khác chỉ là giảm thiểu?

4.1 Đặt vấn đề

Có một triệu chứng rất đặc trưng, và nếu bạn đã làm marker thì gần như chắc chắn đã thấy nó.

Bạn đặt camera trên một cái giá, dán một tag lên tường, không ai chạm vào gì cả. Bạn in pose ra màn hình. Vị trí tâm tag thì ổn định — nó dao động vài phần mười milimét, đúng như [A/05](#) dự đoán. Nhưng góc thì không ổn định chút nào: nó nhảy giữa, chẳng hạn, $+12^\circ$ và -8° theo trục pitch, vài lần mỗi giây, một cách hoàn toàn ngẫu nhiên. Bạn thử tag khác, thử detector khác, thử camera khác. Triệu chứng không đổi.

Phản ứng đầu tiên gần như luôn là “nhiều quá lớn”, và nó dẫn tới một chuỗi việc làm không giúp gì: thêm ánh sáng, tăng độ phân giải, dùng subpixel tốt hơn, thêm bộ lọc thông thấp. Ba việc đầu làm nhiều giảm thật nhưng *tần suất nhảy vẫn nguyên* — chỉ có điều mỗi giá trị trong hai giá trị trở nên ổn định hơn. Việc thứ tư thì tệ nhất trong bốn: lọc thông thấp lấy trung bình hai giá trị và cho ra một giá trị thứ ba mà *không* phải nghiệm nào cả — một pose sai hoàn toàn nhưng trông rất mượt.

Nguyên nhân đã được biết từ lâu và nó không nằm ở nhiễu. Với một target phẳng, bài toán tìm pose từ ảnh có *hai* nghiệm cục bộ, và hiện tượng này được nhận diện ít nhất từ giữa những năm 1990 trong công trình của Oberkampf và cộng sự [25], rồi được phân tích hoàn chỉnh bởi Schweighofer và Pinz [26], và cuối cùng được đưa vào một thuật toán dạng đóng trả về cả hai nghiệm bởi Collins và Bartoli [13]. Ba nhóm ấy độc lập nhau và cách nhau ba thập niên, nên kết quả này thuộc loại đã được kiểm chứng kỹ — đây là một trong số ít khẳng định trong cây mà tôi tự tin ở mức cao.

Điều đáng nói là vì sao nó vẫn làm nhiều người bất ngờ. Lý do nằm ở chỗ hiện tượng này *không xuất hiện trong các tình huống mà người ta hay thử đầu tiên*. Khi bạn cầm tag trong tay và xoay qua xoay lại để thử, tag thường nghiêng khá nhiều, và ở góc nghiêng lớn thì một nghiệm thẳng rõ ràng nên bạn không thấy vấn đề. Vấn đề chỉ nổi lên khi tag ở tư thế gần vuông góc với trục quang — tức đúng khi bạn gắn nó lên một cái dock và cho robot tiếp cận thẳng trục. Nói cách khác: *hiện tượng này ẩn mình trong giai đoạn thử nghiệm và xuất hiện đúng lúc triển khai*.

Cách hiển nhiên hơn để chữa — và cách hầu như ai cũng thử — là chọn nghiệm có sai số tái chiếu nhỏ hơn. Cách này đúng về nguyên tắc và nó là cách mọi thư viện đang làm. Chỗ nó hỏng là khi hai sai số gần bằng nhau: lúc ấy việc “chọn cái nhỏ hơn” trở thành việc chọn theo nhiễu, và nó tệ hơn tung đồng xu ở một điểm — tung đồng xu ít nhất không giả vờ là một quyết định có cơ sở. Chương này dành mục 5 cho việc *đo* xem ta đang ở tình huống nào, và mục 6 cho việc thoát ra khỏi tình huống ấy.

TRỰC GIÁC

Cầm một tấm bìa hình chữ nhật và giơ trước mắt, hơi nghiêng đi một chút. Nhắm một mắt lại, và nhìn kỹ hình dạng bạn thấy: một hình thang, cạnh trên hơi ngắn hơn cạnh dưới.

Bây giờ nghiêng tấm bìa theo *chiều ngược lại*, cùng một góc. Bạn lại thấy một hình thang, và nếu góc đúng bằng nhau thì hình thang lần này là ảnh lật của hình thang lần trước. Với một mắt, và với một tấm bìa đối xứng như hình vuông của một cái tag, *hai hình thang ấy trông giống nhau*.

Đó là toàn bộ vấn đề. Mắt bạn — và camera — không có cách nào phân biệt “nghiêng ra xa ở trên” với “nghiêng ra xa ở dưới” chỉ từ hình dạng phẳng của bóng chiếu. Với người thật thì còn có bóng đổ, có tô bóng, có thị sai hai mắt; camera đơn nhìn một cái tag đen trắng thì không có gì trong số đó.

Bây giờ để ý một điều quan trọng: nếu bạn nghiêng tấm bìa *rất nhiều*, hai hình thang bắt đầu khác nhau rõ, vì cạnh gần và cạnh xa chênh nhau nhiều và phối cảnh không còn đối xứng nữa. Còn nếu tấm bìa gần như vuông góc với hướng nhìn, hai hình thang gần như trùng khít — và lúc đó bạn hoàn toàn không phân biệt được.

Cuối cùng, lối thoát: dựng một cái que vuông góc lên mặt tấm bìa. Bây giờ hai tư thế không còn giống nhau nữa, vì cái que chỉ về hai hướng khác hẳn. Cái que ấy chính là điểm *không đồng phẳng*, và đó là toàn bộ nội dung của mục 6.

4.2 Vì sao đúng hai nghiệm: hình học của phép lật

Trực giác trên đủ để tin, nhưng để dùng được thì cần biết hai nghiệm ấy quan hệ với nhau thế nào một cách chính xác.

Xét một target phẳng và một pose thật (R, t) đặt nó trong hệ camera. Gọi $\mathbf{n}$ là pháp tuyến của mặt phẳng tag trong hệ camera, và gọi $\mathbf{c}$ là hướng từ tâm camera tới tâm tag. Hai vectơ ấy xác định một mặt phẳng — gọi là *mặt phẳng lật*, chứa trục nhìn tới tâm tag và pháp tuyến của tag.

Nghiệm thứ hai, gọi là (R', t') , thu được bằng cách *lật* target quanh trục nằm trong mặt phẳng tag và vuông góc với mặt phẳng lật. Nói cách khác: pháp tuyến $\mathbf{n}$ được phản xạ gương qua đường nhìn $\mathbf{c}$. Nếu pháp tuyến thật nghiêng một góc α so với đường nhìn về một phía, thì pháp tuyến của nghiệm thứ hai nghiêng đúng góc α về phía bên kia [26, §III].

Từ mô tả ấy rút ra ngay ba hệ quả rất dùng được.

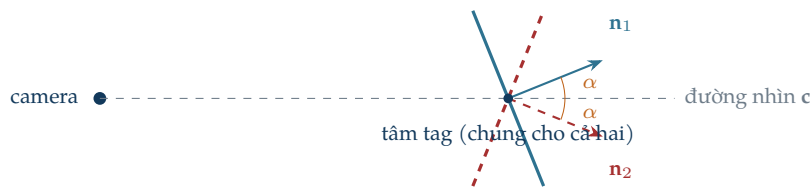
Hệ quả một: hai nghiệm cách nhau 2α về góc. Nếu tag nghiêng 10° so với fronto-parallel, hai nghiệm cách nhau 20° . Nếu tag nghiêng 30° , hai nghiệm cách nhau 60° — rất xa nhau, và cũng rất dễ phân biệt. Ngược lại, nếu tag gần fronto-parallel ($\alpha \rightarrow 0$) thì hai nghiệm *hội tụ về nhau* và đồng thời trở nên không phân biệt được. Đây là một quan hệ đáng chú ý vì nó đi theo hai chiều ngược nhau: ở góc nghiêng lớn, hai nghiệm xa nhau nhưng dễ phân biệt; ở góc nghiêng nhỏ, chúng gần nhau nhưng không phân biệt được.

Hệ quả hai: vị trí tâm gần như giống nhau ở cả hai nghiệm. Phép lật giữ nguyên (tới xấp xỉ bậc nhất) khoảng cách từ camera tới tâm tag, vì nó chỉ đổi hướng của mặt phẳng chứ không đổi vị trí tâm. Đây là lý do triệu chứng quan sát được là “vị trí ổn định, góc nhảy” chứ không phải cả sáu tham số cùng nhảy. Nó cũng là một manh mối chẩn đoán rất tốt: nếu bạn thấy góc nhảy mà vị trí đứng yên, thì gần như chắc chắn đó là ambiguity chứ không phải nhiễu chung.

Hệ quả ba, và đây là hệ quả thực dụng nhất: lấy trung bình hai nghiệm là vô nghĩa. Trung bình của hai xoay cách nhau 2α là một xoay ở chính giữa, tức xoay ứng với $\alpha = 0$, tức fronto-parallel. Nói cách khác, một bộ lọc thông thấp trên đầu ra pose sẽ luôn kéo bạn về phía kết luận “tag đang vuông

góc với camera”, bất kể tag thực sự nghiêng bao nhiêu. Đó không phải làm mượt; đó là tạo ra một thiên lệch có hệ thống hướng về một giá trị cụ thể và sai.

Hình 4.1 vẽ cấu hình này.



Hai nghiệm là **ảnh gương của nhau qua đường nhìn**. Chúng cách nhau 2α về góc và có *cùng* vị trí tâm tới xấp xỉ bậc nhất — nên triệu chứng là “góc nhảy, vị trí yên”. Khi $\alpha \rightarrow 0$ (fronto-parallel) hai nghiệm hội tụ *và* trở nên không phân biệt được: trung bình của chúng luôn là $\alpha = 0$, một giá trị sai có hệ thống.

Hình 4.1. Quan hệ hình học giữa hai nghiệm của một target phẳng, theo mô tả trong [26, §III]. Pháp tuyến của nghiệm thứ hai là ảnh phản xạ của pháp tuyến thật qua đường nhìn tới tâm tag. Hệ quả thực dụng quan trọng nhất nằm ở dòng cuối: mọi phép làm mượt trên đầu ra pose đều kéo kết quả về phía fronto-parallel, tức tạo ra một thiên lệch chứ không phải giảm nhiễu.

4.3 Vì sao thêm điểm đồng phẳng không cứu được

Đây là mục ngắn nhất của chương và cũng là mục hay bị bỏ qua nhất, nên nó đáng đứng riêng.

Phản xạ tự nhiên khi gặp ambiguity là thêm dữ liệu: dán thêm tag, dùng tag có nhiều điểm hơn, chuyển sang ChArUco board với hàng chục góc chessboard. Trực giác đằng sau là “nhiều ràng buộc hơn thì nghiệm xác định hơn”, và trực giác ấy đúng cho *nhiều* nhưng sai cho *ambiguity*.

Lý do nằm ở chỗ phép lật ở mục 2 là một phép biến đổi tác động lên *cả mặt phẳng*, không lên từng điểm riêng lẻ. Khi ta lật target, mọi điểm trên mặt phẳng đều được lật cùng nhau, và hình chiếu của toàn bộ tập điểm sau khi lật vẫn khớp với ảnh gần như tốt như trước. Thêm một điểm thứ năm, thứ mười, thứ trăm — miễn là nó nằm trên cùng mặt phẳng — thì nó cũng bị lật cùng, và nó cũng khớp. Số điểm không phá được một phép đối xứng của hình học.

Điều thêm điểm đồng phẳng *có* làm là giảm nhiễu của mỗi nghiệm. Với nhiều điểm hơn, cả hai nghiệm đều được xác định chính xác hơn, và sai số tái chiếu của cả hai đều nhỏ đi. Nhưng *tỉ* số giữa hai sai số ấy — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn có hai nghiệm sắc nét hơn thay vì hai nghiệm nhoè, và bạn vẫn phải chọn một trong hai.

CẦN NHỚ

Số điểm không phá được ambiguity; chỉ có *tính không đồng phẳng* mới phá được.

Cụ thể: bốn tag dán trên cùng một tấm bảng phẳng vẫn là một target phẳng và vẫn có hai nghiệm. Hai tag dán trên hai mặt phẳng nghiêng với nhau thì không — và hai tag đã đủ. Đây là lý do vì sao lời khuyên ở 00-map không phải “dùng nhiều tag” mà là “dùng constellation *không đồng phẳng*”. Hai lời khuyên nghe gần giống nhau và chúng khác nhau về bản chất.

4.4 Khi nào ambiguity tệ nhất

Bốn yếu tố làm ambiguity nặng thêm, và biết chúng là biết cách tránh.

Góc nghiêng nhỏ (fronto-parallel). Đây là yếu tố mạnh nhất. Khi $\alpha \rightarrow 0$, hai nghiệm hội tụ và tỉ số sai số tái chiếu của chúng tiến về 1. Về mặt cơ chế, đây chính là hiện tượng mà A/05 mục 5 mô tả từ phía khác: độ nhạy của ảnh theo góc nghiêng tỉ lệ với $\sin \alpha$, nên khi α nhỏ thì hai giá trị góc rất khác nhau cho hai ảnh rất giống nhau. Hai chương nói cùng một sự thật: chương 5 nói “hiệp phương sai lớn”, chương này nói “có hai cực tiểu gần nhau”. Trên thực tế cả hai đều đúng và chúng là hai mặt của cùng một ma trận thông tin suy biến.

Tag nhỏ trên ảnh. Tag càng nhỏ thì chênh lệch sai số tái chiếu giữa hai nghiệm càng nhỏ so với nhiễu, vì chênh lệch ấy tỉ lệ với kích thước tag còn nhiễu thì không. Đây là lý do ambiguity nặng lên khi robot còn ở xa và nhẹ đi khi tới gần.

Trường nhìn hẹp, tức tiêu cự dài. Với ống kính tele, phép chiếu tiến gần tới phép chiếu trực giao (orthographic), và dưới phép chiếu trực giao thì hai nghiệm *trùng khít hoàn toàn* — ambiguity trở thành tuyệt đối chứ không chỉ khó phân biệt. Ống kính góc rộng cho nhiều biến dạng phối cảnh hơn, nên phân biệt tốt hơn. Đây là một đánh đổi đáng biết: tele cho f lớn nên sai số ngang nhỏ (A/05), nhưng nó làm ambiguity nặng hơn.

Tag nằm gần tâm ảnh. Một tag nằm ở rìa ảnh được nhìn dưới một góc chéo so với trục quang, và cái chéo ấy tự nó tạo ra một chút biến dạng phối cảnh phụ trội giúp phân biệt. Tag nằm chính giữa ảnh thì không có lợi thế đó. Hiệu ứng này yếu hơn ba yếu tố trên nhưng nó có thật, và nó gợi ý một mẹo bố trí: đừng đặt tag ở chính giữa trường nhìn nếu có thể tránh.

Bảng 4.1. Bốn yếu tố làm ambiguity nặng thêm, cơ chế, và cách giảm nhẹ ở tầng thiết kế.

Yếu tố	Cơ chế	Cách giảm nhẹ
Góc nghiêng nhỏ	Độ nhạy $\propto \sin \alpha$; hai nghiệm hội tụ khi $\alpha \rightarrow 0$	Nghiêng tag trong constellation; hoặc cho đường tiếp cận chéo
Tag nhỏ trên ảnh	Chênh lệch reprojection tỉ lệ với w , nhiễu thì không	Tag lớn hơn ở cự ly xa; nested multi-scale (B/09)
Trường nhìn hẹp (tele)	Tiến tới chiếu trực giao, nơi hai nghiệm trùng khít	Ống kính rộng hơn; chấp nhận đánh đổi với f nhỏ hơn
Tag ở tâm ảnh	Không có biến dạng phối cảnh phụ trội từ vị trí chéo	Bố trí tag lệch khỏi tâm trường nhìn

4.5 Đo ambiguity: tỉ số sai số tái chiếu

Nếu ambiguity không chứa được bằng phần mềm thì ít nhất nó phải *đo* được bằng phần mềm, và may mắn là có một đại lượng rất tự nhiên cho việc đó.

IPPE trả về hai nghiệm kèm sai số tái chiếu của từng nghiệm [13]. Định nghĩa tỉ số ambiguity:

$$\rho = \frac{e_2}{e_1}, \quad e_1 \leq e_2,$$

với e_1 là sai số tái chiếu của nghiệm tốt nhất và e_2 của nghiệm còn lại. Theo định nghĩa $\rho \geq 1$.

Đọc ρ như sau. Khi ρ lớn — chẳng hạn $\rho > 3$ — nghiệm thứ nhất giải thích ảnh tốt hơn hẳn và ta tin nó. Khi $\rho \approx 1$, hai nghiệm giải thích ảnh tốt ngang nhau, và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu; kết quả sẽ nhảy từ khung này sang khung khác đúng như triệu chứng ở mục 1.

Ngưỡng đặt ở đâu thì đúng? Câu trả lời trung thực là *tôi không có một con số có cơ sở lý thuyết*, và tôi ghi rõ điều đó ở đây thay vì đưa ra một con số trông thuyết phục. Cái có cơ sở là cách *dẫn ra* ngưỡng cho hệ của bạn: chạy Monte Carlo (C/16), với mỗi tư thế ghi lại cả ρ lẫn việc nghiệm được chọn có đúng là nghiệm thật không, rồi vẽ tỉ lệ chọn sai theo ρ . Ngưỡng của bạn là giá trị

ρ mà dưới nó tỉ lệ chọn sai vượt mức bạn chấp nhận được. Ngưỡng ấy phụ thuộc vào σ , vào w , vào số điểm — nên nó phải là ngưỡng của hệ bạn chứ không phải một hằng số chép từ đâu đó.

A. Ví dụ số: chênh lệch reprojection lớn cỡ nào

Để thấy các con số ở bậc nào, làm một ước lượng thô. Giả sử tag 120 px trên ảnh (ví dụ chuẩn của cây, A/05 mục 4), nghiêng $\alpha = 5^\circ$, và $\sigma = 0,2$ px.

Nghiệm sai lệch khỏi nghiệm thật một góc $2\alpha = 10^\circ$. Sai lệch góc ấy gây ra sai lệch trong hình chiếu ở bậc: các điểm ở mép tag lệch khỏi vị trí đúng chừng $(w/2)(1 - \cos 2\alpha)$ theo chiều vuông góc với trục lật, tức $60 \times (1 - \cos 10^\circ) = 60 \times 0,0152 = 0,91$ px. Nhưng phần lớn sai lệch ấy được hấp thụ bằng cách điều chỉnh các tham số pose khác (khoảng cách, vị trí ngang), nên phần dư thực sự còn lại nhỏ hơn nhiều — đây chính là lý do nghiệm thứ hai tồn tại như một cực tiểu chứ không bị loại ngay.

Ý nghĩa của phép ước lượng này không nằm ở con số cụ thể mà ở bậc: *chênh lệch reprojection giữa hai nghiệm ở góc nghiêng nhỏ nằm cùng bậc với nhiễu định vị góc*. Khi hai đại lượng cùng bậc thì phân biệt chúng là việc bất khả thi theo nghĩa thống kê, không phải theo nghĩa “chưa đủ thông minh”. (Tôi suy ra ước lượng bậc này từ hình học; nó chưa được kiểm bằng mô phỏng — đã ghi vào 99-chua-biet.md.)

CẠM BẮY

Ba sai lầm phổ biến khi xử lý ambiguity, xếp theo mức độ tai hại tăng dần.

Nhệ: chỉ lấy nghiệm thứ nhất mà không nhìn ρ . Hệ thống chạy đúng phần lớn thời gian và sai một cách bất ngờ ở đúng những tư thế quan trọng nhất.

Nặng: lọc thông thấp trên góc để làm mượt. Như mục 2 đã chỉ ra, trung bình hai nghiệm luôn là fronto-parallel, nên bộ lọc biến một đại lượng *nhảy nhưng đôi khi đúng* thành một đại lượng *mượt và sai có hệ thống*. Tệ hơn nữa, nó xoá mất triệu chứng nên bạn không còn biết mình đang có vấn đề.

Rất nặng: chọn nghiệm theo tiêu chí “gần với pose khung trước”. Nghe rất hợp lý và nó là một kỹ thuật hợp lệ (B/10 mục về nhất quán thời gian), nhưng nếu dùng một mình thì nó có một chế độ hỏng chết người: một khi đã chọn nhầm nghiệm một lần, tiêu chí “gần khung trước” sẽ giữ bạn ở nghiệm sai đó mãi mãi. Bạn có một hệ thống rất ổn định quanh một giá trị sai, và không có tín hiệu nào báo động.

4.6 Lối thoát triệt để: phá tính đồng phẳng

Mọi thứ ở trên là giảm thiểu. Mục này là cách khử.

Kết quả then chốt: *với một tập điểm điều khiển không đồng phẳng, bài toán PnP có nghiệm duy nhất* (bỏ qua các trường hợp suy biến rất đặc biệt và với đủ điểm). Lý do đã nằm sẵn trong hình học của mục 2: phép lật là một phép đối xứng của mặt phẳng, và nó chỉ bảo toàn hình chiếu vì mọi điểm cùng nằm trên mặt phẳng ấy. Một điểm nằm ngoài mặt phẳng bị phép lật đưa tới một vị trí khác hẳn, hình chiếu của nó lệch đi rõ rệt, và sai số tái chiếu của nghiệm thứ hai nhảy vọt. Nghiệm thứ hai không còn là một cực tiểu nữa — nó bị loại bởi chính dữ liệu.

Điều đáng chú ý là *không cần nhiều* tính không đồng phẳng. Về mặt cấu trúc, chỉ cần một điểm nằm ngoài mặt phẳng là đủ để phá đối xứng. Về mặt thực hành, ta cần độ lệch khỏi mặt phẳng đủ lớn để chênh lệch reprojection vượt hẳn nhiễu — và đây là một câu hỏi định lượng mà B/09 trả lời bằng thiết kế cụ thể, còn C/16 trả lời bằng mô phỏng.

Ba cách hiện thực hoá, xếp theo độ dễ thi công.

Cách một, nghiêng các tag. Giữ tất cả tag trên cùng một tấm bảng nhưng gắn mỗi tag lên một nêm nghiêng $15\text{--}30^\circ$ theo các hướng khác nhau. Rẻ, và nó vừa phá đồng phẳng vừa giải quyết vấn đề $\sin \alpha$ của A/05: mỗi tag bây giờ có góc nghiêng riêng khác không so với trục quang, nên chính nó cũng có độ nhạy góc tốt hơn.

Cách hai, đặt tag ở các độ sâu khác nhau. Một số tag trên mặt phẳng dock, một số trên các gờ nhô ra 5–10 cm. Thi công dễ nếu dock vốn đã có cấu trúc ba chiều, và nó cho baseline theo chiều sâu — một hướng mà cách một không cho.

Cách ba, target ba chiều có chủ ý. Một cấu trúc dạng góc (hai hoặc ba mặt phẳng vuông góc nhau, mỗi mặt mang tag). Đây là cách mạnh nhất về mặt hình học nhưng nó chiếm không gian và dễ bị che khuất một phần khi robot tới gần.

CẦN NHỚ

Ambiguity là tính chất của *hình học phẳng*, không phải của thuật toán. Ba hệ quả:
 Một, không thuật toán nào khử được nó khi target còn phẳng — chỉ chọn được nghiệm, và việc chọn có xác suất sai khác không.
 Hai, cách khử duy nhất triệt để là phá tính đồng phẳng, và việc đó xảy ra ở xương cơ khí chứ không trong trình biên dịch.
 Ba, chi phí của việc khử rất thấp so với lợi ích: một nêm nghiêng 20° dưới mỗi tag là một chi tiết gia công tầm thường, và nó đồng thời giải quyết cả bài toán ambiguity lẫn bài toán độ nhạy góc.

4.7 Giới hạn và chế độ hỏng

Ba giới hạn của cách hiểu trình bày trong chương này.

“Hai nghiệm” là kết quả cho target phẳng dưới mô hình pinhole không nhiễu. Trong thực tế có nhiễu, và khi nhiễu lớn thì bề mặt hàm mục tiêu có thể có nhiều hơn hai cực tiểu cục bộ nông. Điều này hiếm và nó không đối bức tranh lớn, nhưng nó có nghĩa là một bộ giải lặp khởi đầu từ một điểm tôi có thể dừng ở chỗ không phải nghiệm nào trong hai nghiệm chính. Đây là lý do nên dùng một lời giải dạng đóng (IPPE, P3P) để khởi tạo thay vì khởi tạo tùy tiện — A/03 bàn kỹ.

Phép lật ở mục 2 là mô tả xấp xỉ. Quan hệ “ảnh gương qua đường nhìn” đúng chính xác trong giới hạn chiếu trục giao và là xấp xỉ tốt dưới chiếu phối cảnh với tag nhỏ so với khoảng cách. Với tag lớn chiếm phần lớn trường nhìn, quan hệ giữa hai nghiệm phức tạp hơn, và hai nghiệm cũng khác nhau ở cả vị trí tâm chứ không chỉ ở hướng. Trong bối cảnh docking thì xấp xỉ này ổn, vì tag hiếm khi chiếm quá một phần tư trường nhìn.

Không đồng phẳng khử ambiguity nhưng không khử được kém điều kiện. Một constellation “không đồng phẳng” với độ lệch khỏi mặt phẳng chỉ 2 mm về mặt kỹ thuật là không đồng phẳng, nhưng chênh lệch reprojection mà 2 mm ấy tạo ra có thể nhỏ hơn nhiễu, và khi đó bạn vẫn có hai cực tiểu gần như bằng nhau trong thực tế. Ranh giới giữa “đủ không đồng phẳng” và “không đủ” là một câu hỏi định lượng phụ thuộc σ , w và Z ; tôi không có công thức cho nó và tôi khuyên đo bằng mô phỏng — đây là một mục trong 99-chua-biet.md.

4.8 Thực hành

Bốn việc cụ thể, theo thứ tự nên làm.

Một, luôn tính cả hai nghiệm và luôn ghi ρ vào log. Dùng solvePnPGeneric với SOLVEPNP_IPPE_SQUARE cho tag vuông, hoặc đọc trực tiếp apriltag_pose.c nếu dùng thư viện gốc — nó đã tính sẵn cả hai. Chi phí gần như bằng không và giá trị chẩn đoán rất cao.

Hai, từ chối cập nhật khi ρ dưới ngưỡng thay vì trả về một nghiệm không đáng tin. Một hệ thống nói “tôi không biết” hữu ích hơn nhiều so với một hệ thống đoán và không báo. B/10 bàn chiến lược cụ thể.

Ba, đừng lọc thông thấp trên góc cho tới khi đã khử ambiguity. Nếu buộc phải làm mượt, hãy làm mượt sau khi đã chọn nghiệm một cách tin cậy, và hãy làm mượt trên đại lượng có ý nghĩa (B/11 về trung bình trên đa tạp), không phải trên góc Euler.

Bốn, và quan trọng nhất, hãy đưa yêu cầu “không đồng phẳng” vào bản vẽ cơ khí ngay từ đầu. Thêm một nệm nghiêng vào bản vẽ trước khi gia công tốn vài phút; thêm nó sau khi đã gia công một trăm cái dock thì tốn một trăm lần chi phí ấy.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao target phẳng có đúng hai nghiệm	Vì phép lật target quanh trục trong mặt phẳng (phản xạ pháp tuyến qua đường nhìn) bảo toàn hình chiếu tới xấp xỉ bậc nhất. Đã được ba nhóm độc lập xác nhận [13], [25], [26].
••• Quan hệ giữa hai nghiệm	Ảnh gương qua đường nhìn; cách nhau 2α về góc với α là góc nghiêng thật; cùng vị trí tâm tới xấp xỉ bậc nhất.
••• Triệu chứng đặc trưng	Góc nhảy giữa hai giá trị, vị trí đứng yên. Nếu thấy cả sáu tham số cùng nhảy thì đó là nhiễu chứ không phải ambiguity.
••• Vì sao lọc thông thấp là sai	Trung bình hai nghiệm cách nhau 2α luôn ra $\alpha = 0$, tức fronto-parallel. Bộ lọc biến một đại lượng nhảy-nhưng-đôi-khi-đúng thành mượt-và-sai-có-hệ-thống, đồng thời xoá mất triệu chứng.
••• Thêm điểm đồng phẳng cứu được không?	Không. Phép lật tác động lên cả mặt phẳng nên mọi điểm đồng phẳng đều bị lật cùng và đều vẫn khớp. Nhiều điểm làm mỗi nghiệm sắc nét hơn nhưng ρ gần như không đổi.
••• Điều kiện khử triệt để	Tập điểm không đồng phẳng $\Rightarrow$ nghiệm duy nhất. Một điểm ngoài mặt phẳng là đủ về cấu trúc; thực hành cần độ lệch đủ lớn để chênh lệch reprojection vượt nhiễu.
••• Bốn yếu tố làm ambiguity nặng	Góc nghiêng nhỏ (mạnh nhất); tag nhỏ trên ảnh; trường nhìn hẹp (tele, tiến tới chiều trục giao); tag nằm ở tâm ảnh.
••• Tỉ số ambiguity	$\rho = e_2/e_1 \geq 1$, từ solvePnPGeneric hoặc IPPE. ρ gần 1 nghĩa là không phân biệt được và việc chọn nghiệm nhỏ hơn là chọn theo nhiễu.
•• Ngưỡng ρ nên đặt ở đâu	Không có hằng số phổ quát. Phải dẫn ra cho hệ của mình bằng Monte Carlo: vẽ tỉ lệ chọn sai theo ρ , lấy ngưỡng ở mức sai chấp nhận được. Phụ thuộc σ , w , số điểm.
•• Vì sao ống kính tele làm nặng thêm	Tele tiến gần tới chiều trục giao, mà dưới chiều trục giao hai nghiệm trùng khít hoàn toàn. Đánh đổi đáng biết: tele cho f lớn nên δx nhỏ, nhưng ambiguity nặng hơn.
•• Chế độ hồng của “chọn theo khung trước”	Một khi chọn nhầm, tiêu chí ấy giữ bạn ở nghiệm sai vĩnh viễn. Hệ thống rất ổn định quanh một giá trị sai và không có tín hiệu báo động. Phải kết hợp với ρ .
•• Quan hệ với chương 5	Cùng một sự thật ở hai mức: chương 5 nói hiệp phương sai theo hướng nghiêng lớn, chương này nói có hai cực tiểu gần nhau. Cả hai đều là hệ quả của $\sin \alpha \rightarrow 0$.
• Ba cách phá đồng phẳng	Nệm nghiêng dưới mỗi tag (rẻ nhất, giải quyết luôn bài toán $\sin \alpha$); tag ở các độ sâu khác nhau (cho baseline theo chiều sâu); target dạng góc nhiều mặt phẳng (mạnh nhất, dễ bị che khuất).

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag <code>apriltag_pose.c</code>	Cài IPPE, tính cả hai nghiệm và sai số tái chiếu của từng nghiệm; hàm <code>estimate_tag_pose</code> trả về tỉ số	Chỗ tốt nhất để đọc chương này thành code; vài trăm dòng
OpenCV <code>SOLVEPNP_IPPE_SQUARE</code>	Cài [13] cho trường hợp riêng tag vuông; dùng với <code>solvePnPGeneric</code> để lấy cả hai nghiệm	API duy nhất cho bạn ρ trực tiếp
S _{Tag} [27]	Thêm một đường tròn ngoại tiếp vào thiết kế tag để ổn định pose — tức thêm thông tin hình học thay vì đổi thuật toán	Cùng triết lý với chương này, giải quyết ở tầng thiết kế marker
ChArUco board	Nhiều góc chessboard nhưng <i>vẫn đồng phẳng</i> — cải thiện σ , không cải thiện ρ	Minh hoạ trực tiếp cho mục 3; đừng nhầm hai loại cải thiện
tagslam [28]	Map nhiều tag ở các mặt phẳng khác nhau trong môi trường, rồi giải hợp nhất	Không đồng phẳng ở quy mô phòng; mức tin C

MINI PROJECT · Nhìn tận mắt hai nghiệm, và đo ngưỡng ρ cho hệ của mình · 1–2 ngày

Mục tiêu	chuyển ambiguity từ một khái niệm thành một đại lượng bạn đo được, và có một ngưỡng ρ có cơ sở thay vì một con số chép từ internet.
Dữ liệu	mô phỏng hoàn toàn, nối tiếp script của A/05 . Không cần phần cứng.
Các bước	sinh một tag vuông ở góc nghiêng α đã biết; chiếu, thêm nhiễu σ ; gọi <code>solvePnPGeneric</code> với <code>SOLVEPNP_IPPE_SQUARE</code> và lấy <i>cả hai</i> nghiệm cùng sai số tái chiếu; ghi lại ρ , và ghi lại nghiệm nào gần chân trị hơn; lặp 2000 lần cho mỗi α , quét α từ 0° tới 45° ; vẽ hai đường: ρ trung vị theo α , và tỉ lệ chọn sai theo α ; cuối cùng vẽ tỉ lệ chọn sai theo ρ (gộp mọi α) — đó chính là đường cong cho bạn ngưỡng.
Xong khi	bạn có một con số ngưỡng ρ^* cho hệ của mình kèm câu phát biểu dạng “với $\rho > \rho^*$, tỉ lệ chọn nhầm nghiệm dưới 1%”. Kiểm tra chéo bắt buộc: lặp lại toàn bộ với constellation không đồng phẳng (thêm một tag nghiêng 20°) và xác nhận rằng ρ bây giờ lớn ở <i>mọi</i> α — nếu không thì độ lệch khỏi mặt phẳng của bạn chưa đủ, và bạn vừa đo được nó cần bao nhiêu.
Nối với	B/10 dùng trực tiếp ngưỡng này; B/09 dùng kết quả kiểm chéo để chọn góc nghiêng cho constellation.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- Đó là pose ambiguity: bài toán PnP với target phẳng có đúng hai nghiệm cục bộ, cả hai giải thích ảnh gần như tốt ngang nhau, và nhiễu quyết định nghiệm nào có sai số tái chiếu nhỏ hơn ở từng khung. Lấy trung bình là việc tệ nhất vì hai nghiệm là ảnh gương của nhau qua đường nhìn, nên trung bình của chúng luôn rơi đúng vào tư thế fronto-parallel — một giá trị không phải nghiệm nào cả và sai một cách có hệ thống. Tệ hơn nữa, nó xoá mất triệu chứng nhảy, nên bạn mất luôn dấu hiệu duy nhất cho biết mình đang có vấn đề.
- Gần như không, và đây là điểm hay bị hiểu nhầm nhất. Phép lật sinh ra nghiệm thứ hai tác động lên *cả mặt phẳng*, nên mọi điểm nằm trên mặt phẳng ấy đều bị lật cùng nhau và đều vẫn khớp với ảnh. Mười sáu góc đồng phẳng cho hai nghiệm sắc nét hơn bốn góc đồng phẳng, nhưng *tỉ số ρ* giữa hai sai số tái chiếu — đại lượng quyết định việc phân biệt — gần như không đổi. Bạn cần tính không đồng phẳng, không phải số lượng: hai tag trên hai mặt phẳng nghiêng nhau thắng bốn tag trên một mặt phẳng.
- Nghiệm thứ hai là ảnh gương của nghiệm thật qua đường nhìn tới tâm tag: nếu pháp tuyến thật nghiêng α về một phía thì pháp tuyến nghiệm hai nghiêng α về phía kia, nên hai nghiệm cách nhau 2α về góc và có cùng vị trí tâm tới xấp xỉ bậc nhất [26, §III]. Biết quan hệ này giúp

ba việc: nhận diện triệu chứng (góc nhảy, vị trí yên — khác hẳn với nhiễu chung làm cả sáu tham số nhảy); dự đoán biên độ nhảy (bằng hai lần góc nghiêng, nên biên độ nhảy nhỏ lại là dấu hiệu *xấu* vì nó nghĩa là bạn đang ở gần fronto-parallel); và hiểu vì sao trung bình luôn ra fronto-parallel.

- A4. Khi $\rho \approx 1$, tức khi hai sai số tái chiếu gần bằng nhau. Lúc ấy cái quyết định nghiệm nào nhỏ hơn là nhiễu chứ không phải hình học, nên chọn theo ϵ nhỏ hơn về bản chất là tung đồng xu. Nó tệ hơn tung đồng xu ở một điểm quan trọng: tung đồng xu không giả vờ là một quyết định có cơ sở, còn “chọn nghiệm có residual nhỏ hơn” thì nghe rất khoa học và không ai nghĩ tới việc kiểm lại. Cách sửa không phải đổi tiêu chí chọn mà là *đo ρ và từ chối trả lời khi nó quá thấp*.
- A5. Phá tính đồng phẳng của tập điểm — nghiêng các tag, đặt chúng ở các độ sâu khác nhau, hoặc dùng target dạng góc. Nó triệt để vì nó phá chính phép đối xứng sinh ra nghiệm thứ hai: điểm nằm ngoài mặt phẳng bị phép lật đưa tới vị trí khác hẳn, hình chiếu của nó lệch rõ rệt, nên nghiệm thứ hai không còn là cực tiểu nữa mà bị dữ liệu loại bỏ. Mọi cách khác — ngưỡng ρ , nhất quán thời gian, fuse IMU — chỉ giúp *chọn đúng* trong hai nghiệm vẫn đang tồn tại, nên chúng luôn có một xác suất sai khác không. Và điểm đáng nói về kinh tế: cách triệt để này được thi công ở xương cơ khí với chi phí gần như bằng không, trong khi các cách giảm thiểu tiêu tốn công sức phần mềm vô hạn.

NÓI LẠI THẬT ĐƠN GIẢN

Có một trò ảo giác thị giác quen thuộc: hình vẽ một cái hộp dây thép, và nhìn một lúc thì nó tự lật — lúc thì mặt này ở trước, lúc thì mặt kia. Bạn không điều khiển được, và không có câu trả lời đúng, vì bản thân hình vẽ chứa đúng hai cách hiểu và cả hai đều khớp hoàn hảo với những gì trên giấy. Một cái tag phẳng nhìn qua một camera rơi đúng vào tình huống ấy. Từ hình dạng méo của cái tag trên ảnh, có đúng hai cách nghiêng nó trong không gian để cho ra hình dạng đó — nghiêng về phía này, hoặc nghiêng về phía kia. Máy tính cũng không có câu trả lời đúng, và nó cũng lật qua lật lại, chỉ có điều nó lật mấy chục lần mỗi giây thay vì mấy giây một lần.

Điều tệ nhất bạn có thể làm là lấy trung bình hai cách hiểu. Trung bình của “nghiêng sang trái” và “nghiêng sang phải” là “không nghiêng gì cả” — một câu trả lời trơn tru, ổn định, và luôn luôn sai. Nhưng nó lại là điều đầu tiên ai cũng làm, vì con số ngừng nhảy và mọi thứ trông đã ổn.

Có hai cách xử lý đúng, và chúng rất khác nhau về bản chất. Cách thứ nhất là dạy máy tính nói “tôi không chắc” — đo xem hai cách hiểu khớp với ảnh tốt ngang nhau tới mức nào, và khi chúng ngang nhau thì im lặng thay vì đoán. Cách này luôn dùng được nhưng nó không giải quyết gì, nó chỉ trung thực.

Cách thứ hai là làm cho câu hỏi hết mơ hồ. Với cái hộp dây thép, chỉ cần vẽ thêm một chỗ mà một cạnh che khuất cạnh kia — chỉ một chỗ thôi — và ảo giác biến mất vĩnh viễn, vì bây giờ chỉ còn một cách hiểu khớp. Với cái tag, việc tương đương là làm cho nó đừng phẳng nữa: kẻ nghiêng vài cái tag, hoặc đặt một cái nhô ra trước những cái kia. Và đây là điều đáng nhớ nhất của cả chương: việc ấy được làm bằng một cái nẹp nhôm và bốn con vít, không phải bằng một thuật toán.

Chỗ không rút gọn được: ranh giới giữa “đủ không đồng phẳng” và “chưa đủ” thì không có công thức. Nó phụ thuộc vào nhiễu của hệ bạn, vào kích thước tag trên ảnh, vào khoảng cách — nên nó phải đo, không suy ra được. May là đo bằng mô phỏng rẻ và làm được trong một buổi.

Một câu: *Cái tag phẳng của bạn luôn có hai câu trả lời; chọn đúng là một trò may rủi, còn đặt nó nghiêng đi thì chỉ còn một câu.*

4.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *target phẳng có đúng hai nghiệm cục bộ, và tập điểm không đồng phẳng có nghiệm duy nhất* — là mệnh đề tôi tin nhất trong cả cây, vì nó đã qua cửa (c) với ba nguồn độc lập cách nhau ba thập niên [13], [25], [26], và cửa (a) bằng lập luận đối xứng. Nó bị bác bỏ nếu ai đó trình bày một cấu hình target phẳng không suy biến mà bài toán PnP chỉ có một cực tiểu, hoặc

ngược lại một tập điểm không đồng phẳng đủ tổng quát mà vẫn có hai nghiệm phân biệt được với sai số tái chiếu bằng nhau.

Mệnh đề thứ hai — *trung bình hai nghiệm luôn rơi vào fronto-parallel* — bị bác bỏ nếu mô phỏng cho thấy trung bình của hai nghiệm IPPE trên một chuỗi khung hình hội tụ về một giá trị khác $\alpha = 0$ một cách có hệ thống. Đây là mệnh đề tôi dẫn từ hình học của mục 2 (cửa a) và nó *chưa* qua cửa (b) — một chỗ nọ kiểm chứng, tuy nhỏ vì nó là hệ quả trực tiếp của quan hệ gương.

Mệnh đề thứ ba — *thêm điểm đồng phẳng không cải thiện ρ* — là mệnh đề dễ kiểm nhất và cũng là mệnh đề có giá trị thực hành cao nhất, vì nếu nó sai thì lời khuyên trung tâm của chương (phá đồng phẳng thay vì thêm tag) sẽ sai theo. Nó bị bác bỏ nếu mô phỏng cho thấy ρ tăng đáng kể khi đi từ 4 lên 64 góc đồng phẳng ở cùng một góc nghiêng. Mini project ở trên chính là phép kiểm ấy, và nó nên được chạy sớm.

Cuối cùng, một mệnh đề mềm hơn: chương này khẳng định rằng *ambiguity là nguồn lỗi xoay lớn hơn nhiều trong bài toán docking điển hình*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã khử hoàn toàn ambiguity bằng constellation không đồng phẳng, sai số xoay vẫn không cải thiện đáng kể — khi ấy nút thắt nằm ở chỗ khác (nhiều khả năng là sai số cơ khí của constellation, B/12). (Tôi phát biểu mệnh đề này dựa trên cấu trúc của vấn đề chứ không dựa trên thống kê ca hòng thực tế; tôi không có số liệu về tỉ lệ.)

4.10 Kết nối

Chương này là chương bệnh lý của Phần A, và nó nối vào cây ở nhiều chỗ hơn độ dài của nó gợi ý.

Nối lên trên. [A/02-homography-va-hinh-hoc-phang](#) là nơi hai nghiệm xuất hiện lần đầu dưới dạng đại số: phép phân rã homography thành pose vốn đã cho nhiều nghiệm, và chương này giải thích ý nghĩa hình học của chúng. [A/03-pnp-va-uoc-luong-tu-the](#) cho IPPE và sai số tái chiếu, tức cả hai công cụ mà chương này dùng để đo ambiguity; nó cũng là nơi giải thích vì sao phải khởi tạo bằng lời giải dạng đóng thay vì tùy tiện.

Nối ngang, và đây là liên kết quan trọng nhất. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) là chương song sinh: cùng một sự kiện hình học — độ nhạy tỉ lệ với $\sin \alpha$ — được nhìn từ hai phía. Chương 5 nói “ma trận thông tin gần suy biến theo hướng nghiêng, nên hiệp phương sai lớn”; chương này nói “có hai cực tiểu gần nhau, nên bộ giải nhảy giữa chúng”. Cả hai đều đúng và chúng mô tả cùng một bệnh. Một phân tích chỉ dùng chương 5 sẽ kết luận rằng sai số xoay là nhiều và có thể lấy trung bình — một kết luận sai và tốn kém. Đó là lý do hai chương phải đọc cùng nhau.

Nối xuống dưới. [B/09-multi-marker-constellation](#) là chương biến mục 6 thành thiết kế cụ thể: nghiêng bao nhiêu độ, đặt tag ở đâu, độ lệch khỏi mặt phẳng cần bao nhiêu. [B/10-xu-ly-ambiguity](#) nhận toàn bộ mục 5 và mục 8: nó là chương về những gì phải làm ở tầng code khi bạn *chưa* khử được ambiguity bằng hình học — ngưỡng, từ chối cập nhật, nhất quán thời gian, fuse IMU. [B/11-uoc-luong-theo-thoi-gian](#) nhận cảnh báo ở mục 5: mọi kỹ thuật làm mượt trong chương ấy chỉ hợp lệ *sau khi* nghiệm đã được chọn tin cậy, và chương này là lý do của điều kiện ấy.

Cuối cùng, một nối sang Phần C. [C/15-ngan-sach-sai-so](#) phải xử lý ambiguity theo một cách khác với mọi dòng khác trong ngân sách: nó không phải một nguồn sai số cộng vào mà là một *lỗi thô* phá hỏng toàn bộ ngân sách khi nó xảy ra. Ngân sách sai số giả định bạn đang ở gần nghiệm đúng; nếu bạn đang ở nghiệm sai thì mọi con số milimét trong bảng đều vô nghĩa. Đó là lý do việc khử ambiguity phải là điều kiện tiên quyết chứ không phải một dòng trong bảng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao thêm mười hai góc đồng phẳng không cải thiện tỉ số ρ trong khi nó rõ ràng cải thiện σ — hãy trả lời bằng cách chỉ ra phép biến đổi nào đang được bảo toàn?
2. Một đồng nghiệp thấy pose nhảy và đề xuất dùng bộ lọc Kalman. Nêu chính xác chuyện gì sẽ xảy ra với đầu ra, và vì sao kết quả trông tốt hơn nhưng thực chất tệ hơn.
3. Hai nghiệm cách nhau 2α . Vì sao biên độ nhảy *nhỏ* lại là dấu hiệu xấu chứ không phải dấu hiệu tốt?
4. Bạn dùng ống kính tele để có f lớn, theo lời khuyên của chương 5. Điều đó ảnh hưởng thế nào tới ambiguity, và bạn cân bằng hai yêu cầu ngược nhau ấy bằng cách nào?
5. Bạn đã phá tính đồng phẳng bằng cách đặt một tag nhô ra trước 3 mm so với các tag khác. Vì sao điều đó có thể vẫn không đủ, và bạn kiểm bằng cách nào mà không cần dựng phần cứng?
6. Tiêu chí “chọn nghiệm gần pose khung trước” có một chế độ hỏng rất đặc trưng. Mô tả nó, và nêu một cơ chế phát hiện được nó từ trong dữ liệu đo.
7. Vì sao ambiguity không thể được coi là một dòng trong bảng ngân sách sai số, mà phải được coi là điều kiện tiên quyết?

Nguồn

- [1] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, “Propagating Covariance in Computer Vision,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, “Statistical Optimization for Geometric Computation: Theory and Practice,” 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., “Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera,” in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., “Rolling Shutter Camera Calibration,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, “Introduction to Fourier Optics,” 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

Lan truyền sai số từ pixel sang pose (Error Propagation)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (ma trận K , và vì sao tiêu cự f tính bằng pixel là đại lượng duy nhất của camera xuất hiện trong mọi công thức dưới đây); A/03-pnp-va-uoc-luong-tu-the (PnP, sai số tái chiếu, Jacobian của phép chiếu — chương này lấy đúng Jacobian ấy rồi đảo nó); A/04-pose-ambiguity-target-phang (vì sao hướng xoay là hướng yếu — chương này định lượng cái yếu đó).

TÓM TẮT

Chương này biến một câu hỏi mơ hồ — “hệ của tôi chính xác cỡ nào” — thành bốn phép nhân chia làm được trên giấy trong ba mươi giây. Xuất phát điểm là một công thức duy nhất, $\Sigma_{\text{pose}} \approx (J^T \Sigma_{\text{px}}^{-1} J)^{-1}$, và toàn bộ phần còn lại của chương là việc đọc ý nghĩa vật lý của ma trận ấy trong bốn trường hợp cụ thể. Bốn kết quả: sai số ngang $\delta x \approx Z\sigma/f$, sai số dọc trục $\delta Z \approx Z\sigma/w$, độ nhạy xoay của một tag phẳng tỉ lệ với sin của góc nghiêng, và sai số xoay từ một constellation có baseline B là $\delta\theta \approx \delta x/B$. Bốn quan hệ ấy ngắn đủ để thuộc lòng và chúng là công cụ chẩn đoán hằng ngày. Nếu chỉ nhớ một điều: *sai số ngang chia cho f , sai số sâu chia cho w , và $w \ll f$ — nên chiều sâu luôn tệ hơn chiều ngang chừng f/w lần; còn xoay thì không nằm trong danh sách ấy vì nó tuân theo một quy luật khác hẳn, và đó là lý do nó là nút thắt.*

KIẾN THỨC CẦN CÓ TRƯỚC

- **tiêu cự tính bằng pixel (f)** — phần tử f_x hoặc f_y của ma trận K . Nó không phải chiều dài vật lý của ống kính mà là tỉ số giữa tiêu cự vật lý và kích thước một pixel trên cảm biến. Với một camera 1280×960 và trường nhìn ngang khoảng 60°, f cỡ 1100 px. Đây là đại lượng duy nhất của camera xuất hiện trong công thức sai số ngang.
- **kích thước tag trên ảnh (w)** — cạnh của tag đo bằng pixel trên ảnh, $w \approx fW/Z$ với W là cạnh thật của tag và Z là khoảng cách. Nó là đại lượng quyết định sai số theo chiều sâu, và nó nhỏ hơn f rất nhiều trong mọi tình huống thực tế.
- **nhieu định vị góc (σ)** — độ lệch chuẩn của vị trí một góc marker trên ảnh, tính bằng pixel. Đây là đầu vào duy nhất mang tính “chất lượng thuật toán” trong cả chương; mọi thứ khác là hình học.
- **Jacobian của sai số tái chiếu (J)** — ma trận đạo hàm riêng của tọa độ ảnh dự đoán theo sáu tham số pose, $J = \partial u / \partial \xi$. Mỗi góc marker đóng góp hai hàng (một cho u , một cho v).
- **ma trận thông tin Fisher** — $\mathcal{I} = J^T \Sigma_{\text{px}}^{-1} J$. Nó đo lượng thông tin mà tập phép đo mang về pose; nghịch đảo của nó là cận dưới cho hiệp phương sai của mọi bộ ước lượng không chệch [29].
- **baseline của constellation (B)** — khoảng cách giữa hai marker xa nhau nhất trong tập marker, đo trong không gian 3D. Đây là đại lượng hình học có đòn bẩy lớn nhất trong toàn cây, và nó không phụ thuộc vào bất kỳ lựa chọn phần mềm nào.
- **fronto-parallel** — tư thế mà mặt phẳng tag song song với mặt phẳng ảnh, tức camera nhìn thẳng vuông góc vào tag. Đây là tư thế trực quan nhất và là tư thế *tệ nhất* cho việc ước lượng góc nghiêng — một nghịch lý mà mục 5 giải thích.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn tăng độ phân giải camera lên gấp đôi, giữ nguyên ống kính và mọi thứ khác. Sai số vị trí của bạn cải thiện bao nhiêu lần, và vì sao câu trả lời không phải “gấp đôi”?
- Q2. Một tag 100 mm ở cách camera nửa mét cho sai số dọc trục khoảng 0,8 mm. Nếu bạn dùng tag 200 mm thay vào, sai số đó thành bao nhiêu — và quan trọng hơn, nếu bạn thay bằng hai tag 100 mm đặt cách nhau 200 mm thì được gì mà tag lớn không cho?
- Q3. Vì sao sai số theo chiều sâu luôn tệ hơn sai số theo chiều ngang, và tỉ số giữa chúng bằng đại lượng nào — hãy trả lời bằng một phân số của hai đại lượng đã có tên trong chương này.
- Q4. Robot của bạn tiếp cận dock thẳng trục, nên camera nhìn gần vuông góc vào tấm marker. Đó là tư thế trực quan nhất và là tư thế *tệ nhất* cho việc đo góc. Cơ chế hình học nào tạo ra nghịch lý đó?
- Q5. Ngân sách của bạn cho phép 0,5°. Có hai cách tiêu tiền: mua camera tốt hơn để giảm σ từ 0,2 px xuống 0,05 px, hoặc làm tấm marker rộng thêm 30 cm. Cách nào cho nhiều độ hơn trên mỗi đồng, và vì sao câu trả lời gần như không phụ thuộc vào con số cụ thể?

5.1 Đặt vấn đề

Có một câu hỏi mà mọi người làm marker đều phải trả lời và gần như không ai trả lời được trong lần đầu được hỏi: *hệ của bạn chính xác tới đâu?*

Câu trả lời thường gặp là một con số thực nghiệm — “tôi đo được khoảng 3 mm” — và nó có hai vấn đề. Vấn đề thứ nhất là nó chỉ đúng cho đúng cấu hình đã đo: đổi khoảng cách, đổi kích thước tag, đổi ống kính, và con số ấy không nói gì nữa. Vấn đề thứ hai nghiêm trọng hơn: nó không cho biết *nên sửa gì*. Một con số ở đầu ra là tổng hợp của nhiều nguyên nhân, và nếu không tách được các thành phần thì mọi nỗ lực cải thiện đều là thử và sai.

Bài toán gốc ở đây không mới và cũng không riêng của thị giác. Nó là bài toán *lan truyền bất định* mà ngành đo lường đã hình thức hoá từ lâu [30]: có một đại lượng đo được trực tiếp với bất định đã biết, có một mô hình biến nó thành đại lượng ta quan tâm, và câu hỏi là bất định của đại lượng đích bằng bao nhiêu. Trong trường hợp của ta, đại lượng đo trực tiếp là vị trí các góc marker trên ảnh với bất định σ pixel, mô hình là phép chiếu phối cảnh, và đại lượng đích là sáu tham số pose.

Điều làm bài toán này đáng dành một chương riêng không phải độ khó toán học — công thức lan truyền là kiến thức chuẩn [31], [32] — mà là chuyện sau đây. Khi ta viết công thức lan truyền ra và đọc nó theo từng thành phần, ta phát hiện rằng *sáu tham số pose không được sinh ra bình đẳng*. Hai tham số tịnh tiến ngang có bất định nhỏ, tham số tịnh tiến dọc trục có bất định lớn hơn chừng f/w lần, và các tham số xoay — ít nhất là hai trong ba — tuân theo một quy luật hoàn toàn khác, một quy luật trong đó *góc nhìn* xuất hiện như một hệ số nhân và có thể tiến về không. Sự bất bình đẳng ấy là toàn bộ nội dung thực dụng của chương.

Cách hiển nhiên hơn — và cách gần như ai cũng thử trước — là đo. Dựng hệ, chạy, so với thước, ra một con số. Cách này không sai và C/16 dành hẳn cho nó. Nhưng nó có một nhược điểm chí mạng ở giai đoạn thiết kế: nó đòi hệ phải tồn tại. Khi bạn đang phải quyết định đặt hai tag hay bốn tag, cách nhau 30 cm hay 50 cm, có nghiêng hay không, thì không có gì để đo cả. Quyết định ấy phải được đưa ra *trước*, và nó tốn tiền gia công nếu sai. Chương này tồn tại để những quyết định đó được đưa ra trên giấy.

Có một cách hiển nhiên thứ hai, tinh vi hơn và cũng phổ biến hơn ở người có kinh nghiệm: chạy Monte Carlo. Thêm nhiễu vào góc, chạy PnP, xem phân bố. Cách này *tốt* và C/16 khuyến khích nó mạnh mẽ. Nhưng nó vẫn không thay thế được chương này, vì một lý do về bản chất của hiểu biết: Monte Carlo cho bạn biết *kết quả là bao nhiêu* chứ không cho biết *vì sao*. Nó không nói cho bạn rằng sai số sâu tệ hơn sai số ngang đúng f/w lần, nên nó không gợi ý rằng tăng w là đòn bẩy. Công thức giải tích và mô phỏng số phục vụ hai chức năng khác nhau, và cách dùng đúng là

dùng cả hai: công thức để hiểu và để chọn hướng, mô phỏng để kiểm và để bắt những chỗ công thức bỏ sót.

TRỰC GIÁC

Hình dung việc ước lượng pose như việc định vị bằng cách nhìn một vật đã biết kích thước. Để biết vật đó nằm *lệch trái phải* bao nhiêu, bạn nhìn xem nó nằm ở đâu trên khung nhìn. Một sai lệch nhỏ trên khung nhìn quy ra một sai lệch nhỏ trong không gian, và hệ số quy đổi là Z/f — càng xa thì mỗi pixel càng “đáng giá” nhiều milimét. Đây là phép đo trực tiếp và nó tốt.

Để biết vật đó ở *xa gần* bao nhiêu, bạn không có cách nhìn trực tiếp. Bạn phải suy từ việc nó *trông to cỡ nào*. Và ở đây có một sự khuếch đại: để biết khoảng cách chính xác 1%, bạn phải đo được kích thước biểu kiến chính xác 1%; nhưng kích thước biểu kiến chỉ có w pixel, nên 1% của nó là $w/100$ pixel — một đại lượng nhỏ hơn nhiều so với $f/100$. Đây là lý do chiều sâu luôn tệ hơn, và tỉ số tệ hơn đúng bằng f/w .

Để biết vật đó *ngiênêng* bao nhiêu, bạn phải nhìn vào sự *biến dạng phối cảnh*: cạnh xa trông ngắn hơn cạnh gần. Nhưng nếu vật đang nhìn thẳng vào bạn thì không có cạnh nào xa hơn cạnh nào, biến dạng bằng không, và một chút nghiêng gây ra một chút biến dạng *bậc hai* chứ không phải bậc một. Đó chính là lý do gần vuông góc là tệ nhất: đạo hàm của thứ bạn quan sát theo thứ bạn muốn biết đi qua không.

Và đây là lối thoát: nếu thay vì một vật, bạn nhìn *hai* vật cách nhau một khoảng B đã biết, thì góc quay của cặp ấy không còn phải suy từ biến dạng phối cảnh nữa. Nó suy từ chênh lệch vị trí ngang giữa hai vật — một phép đo thuộc loại *tốt*. Đó là toàn bộ lý do constellation tồn tại.

5.2 Công thức gốc: nghịch đảo của ma trận thông tin

Mọi thứ trong chương này chảy ra từ một công thức, nên đáng dẫn nó một lần cho cẩn thận.

Bài toán PnP tinh chỉnh (A/03) cực tiểu hoá tổng bình phương sai số tái chiếu có trọng số:

$$C(\xi) = \frac{1}{2} \sum_{i=1}^N (u_i - \hat{u}_i(\xi))^T \Sigma_i^{-1} (u_i - \hat{u}_i(\xi)),$$

với $\xi \in \mathbb{R}^6$ là tham số hoá pose (ba cho tịnh tiến, ba cho xoay — xem A/03 về việc dùng $\mathfrak{se}(3)$ để tránh kỳ dị), u_i là góc đo được, $\hat{u}_i(\xi)$ là góc dự đoán, và Σ_i là hiệp phương sai nhiễu của góc thứ i .

Khai triển quanh nghiệm $\hat{\xi}$ tới bậc hai và bỏ số hạng chứa phần dư nhân đạo hàm bậc hai — hợp lệ khi phần dư nhỏ, tức khi mô hình đúng và nhiễu nhỏ, và đây chính là giả thiết Gauss–Newton:

$$C(\hat{\xi} + \delta) \approx C(\hat{\xi}) + \frac{1}{2} \delta^T \mathcal{I} \delta, \quad \mathcal{I} = \sum_{i=1}^N J_i^T \Sigma_i^{-1} J_i, \quad J_i = \left. \frac{\partial \hat{u}_i}{\partial \xi} \right|_{\hat{\xi}}.$$

Vì sao bước bỏ số hạng kia hợp lệ? Vì đạo hàm bậc hai của C có dạng $\sum J^T \Sigma^{-1} J - \sum r^T \Sigma^{-1} \partial^2 \hat{u} / \partial \xi^2$, và số hạng thứ hai tỉ lệ với phần dư r . Khi mô hình đúng, phần dư là nhiễu có trung bình không, nên số hạng ấy triệt tiêu theo kỳ vọng. Khi mô hình *sai* — méo ảnh chưa khử sạch, constellation sai so với CAD — phần dư có thành phần hệ thống và số hạng ấy không triệt tiêu. Đây là một ghi chú không hàn lâm chút nào: nó có nghĩa là *công thức hiệp phương sai dưới đây tự nó giả định mô hình đúng*, và nó sẽ báo một hiệp phương sai đẹp ngay cả khi bạn đang đo quanh một giá trị sai. C/16 quay lại chuyện này bằng phép thử tính nhất quán.

Với giả thiết ấy, hiệp phương sai của pose ước lượng là

$$\Sigma_{\text{pose}} \approx \mathcal{I}^{-1} = \left(\sum_i J_i^T \Sigma_i^{-1} J_i \right)^{-1}. \quad (5.1)$$

Đây là kết quả chuẩn của lý thuyết ước lượng — nó chính là cận Cramér–Rao mà bộ ước lượng hợp lý cực đại đạt được tiệm cận [29, chương 3], và trong ngữ cảnh thị giác nó được viết ra tường minh bởi [31]. Điều đáng nói là nó đúng cho *mọi* bộ ước lượng không chệch: không thuật toán nào làm tốt hơn được, nên nếu $\mathcal{I}^{-1}$ nói bạn không đạt 0,5° thì đổi thuật toán không cứu được.

Trường hợp riêng hay dùng: nếu mọi góc có cùng nhiễu đẳng hướng $\Sigma_i = \sigma^2 I_2$, thì

$$\Sigma_{\text{pose}} \approx \sigma^2 \left(\sum_i J_i^T J_i \right)^{-1}.$$

Dạng này tách bạch hai thứ một cách rất sáng sủa, và sự tách bạch ấy là thông điệp trung tâm của chương: σ^2 là *chất lượng đo*, còn $(\sum J^T J)^{-1}$ là *hình học*. Chất lượng đo là thứ bạn mua bằng detector tốt hơn, quang học tốt hơn, trung bình nhiều khung. Hình học là thứ bạn mua bằng cách đặt các tag ở đâu. Và vì hình học vào công thức dưới dạng một ma trận nghịch đảo còn σ chỉ là một hệ số nhân vô hướng, *hình học có thể làm sai số lớn lên vô hạn còn σ thì không* — một ma trận gần suy biến có nghịch đảo lớn tùy ý.

CẦN NHỚ

Công thức (5.1) chia bài toán làm hai nửa không cân xứng: σ^2 (chất lượng đo) là một *hệ số nhân*, còn hình học là một *ma trận nghịch đảo*. Cải thiện σ gấp bốn lần thì sai số giảm bốn lần, không hơn. Sửa một hình học suy biến thì sai số giảm từ vô hạn xuống hữu hạn. Hệ quả thực hành: khi sai số của bạn lớn một cách khó hiểu, câu hỏi đầu tiên không phải “detector của tôi có kém không” mà là “ma trận $\sum J^T J$ của tôi có gần suy biến theo hướng nào không”.

5.3 Hai quan hệ tịnh tiến: chia cho f , chia cho w

Hai trường hợp đơn giản nhất, và chúng đơn giản đến mức dẫn được bằng vi phân một dòng mỗi cái.

A. Sai số ngang: chia cho tiêu cự

Phương trình chiếu cho tọa độ ngang là $u = f X / Z + c_x$. Giữ Z cố định và vi phân theo X :

$$\frac{\partial u}{\partial X} = \frac{f}{Z} \implies \delta X = \frac{Z}{f} \delta u.$$

Với $\delta u = \sigma$ là nhiễu định vị góc, ta được quan hệ thứ nhất:

$$\boxed{\delta x \approx \frac{Z \sigma}{f}} \quad (5.2)$$

Nếu có N góc và nhiễu của chúng độc lập, con số này còn chia thêm cho $\sqrt{N}$ — nhưng tôi cố ý không đưa $\sqrt{N}$ vào công thức đóng khung, vì trong thực tế nhiễu góc trên cùng một tag *không* độc lập (cùng chịu một điều kiện chiếu sáng, cùng một mức mờ, cùng một sai số méo cục bộ), nên $\sqrt{N}$ là một lời hứa thường không được giữ. Dùng công thức không có $\sqrt{N}$ là cách ước lượng thận trọng, và ngân sách sai số nên thận trọng.

Ba điều đáng đọc ra từ (5.2). Thứ nhất, sai số ngang *tuyến tính theo khoảng cách*: gần gấp đôi thì chính xác gấp đôi. Đây là lý do vì sao chiến lược coarse-to-fine ở B/13 hoạt động — bạn không cần chính xác ở xa, và khi tới gần thì chính xác tự đến. Thứ hai, đại lượng camera duy nhất xuất hiện là f tính bằng pixel. Thứ ba, và đây là điều quan trọng nhất: *không có kích thước tag trong công thức này*. Tag lớn hay nhỏ không ảnh hưởng tới sai số ngang, miễn là vẫn phát hiện được và σ không đổi. Đây là một sự thật phản trực giác mà tôi khuyên nên kiểm lại bằng Monte Carlo khi mới gặp, vì nó mâu thuẫn với cảm giác “tag lớn thì chính xác hơn”. Cảm giác ấy đúng — nhưng nó đúng cho chiều sâu và cho góc, không đúng cho chiều ngang.

B. Sai số dọc trục: chia cho kích thước tag trên ảnh

Đây là chỗ xuất hiện sự bất bình đẳng đầu tiên giữa các trục. Chiều sâu không được đo trực tiếp; nó được suy từ kích thước biểu kiến. Với một tag cạnh thật W ở khoảng cách Z , kích thước trên ảnh là $w = fW/Z$. Vi phân theo Z :

$$\frac{\partial w}{\partial Z} = -\frac{fW}{Z^2} = -\frac{w}{Z} \implies \delta Z = \frac{Z}{w} \delta w.$$

Nhiều của *kích thước biểu kiến* δw đến từ nhiều của hai đầu mút, nên nó cỡ σ (chính xác hơn là $\sqrt{2}\sigma$ nếu hai đầu độc lập; tôi giữ σ cho gọn và vì lý do thận trọng đã nói). Ta được quan hệ thứ hai:

$$\delta Z \approx \frac{Z \sigma}{w} = \frac{Z^2 \sigma}{f W} \quad (5.3)$$

Dạng thứ hai của công thức — viết theo W và Z thay vì w — đáng nhớ riêng, vì nó cho thấy sai số sâu là *bậc hai theo khoảng cách*. Xa gấp đôi thì tệ gấp bốn, không phải gấp đôi.

So sánh hai công thức cho ngay tỉ số:

$$\frac{\delta Z}{\delta x} \approx \frac{f}{w} = \frac{Z}{W}.$$

Đây là câu trả lời cho Q3, và nó có một cách phát biểu rất dễ nhớ: *sai số sâu tệ hơn sai số ngang đúng bằng tỉ số giữa khoảng cách và kích thước tag*. Tag 100 mm ở 500 mm thì sai số sâu tệ hơn 5 lần. Tag 100 mm ở 3 m thì tệ hơn 30 lần. Con số ấy giải thích vì sao mọi hệ thống marker đều thấy rằng thành phần khoảng cách là thành phần tệ nhất của tịnh tiến, và nó cũng giải thích vì sao đây là một trong số ít chỗ mà *làm tag to hơn thật sự có tác dụng*.

I 5.4 Ví dụ số chuẩn của cây

Đây là ví dụ chạy xuyên suốt cả cây, và nó là ví dụ nhỏ nhất tính tay được theo yêu cầu của learning-rules.md mục 4. Giả sử:

$$f = 600 \text{ px}, \quad Z = 0,5 \text{ m}, \quad W = 100 \text{ mm}, \quad \sigma = 0,2 \text{ px}.$$

Kích thước tag trên ảnh: $w = fW/Z = 600 \times 0,1/0,5 = 120 \text{ px}$.

Sai số ngang theo (5.2):

$$\delta x = \frac{0,5 \times 0,2}{600} = \frac{0,1}{600} = 1,67 \times 10^{-4} \text{ m} = \mathbf{0,17 \text{ mm}}.$$

Sai số dọc trục theo (5.3):

$$\delta Z = \frac{0,5 \times 0,2}{120} = \frac{0,1}{120} = 8,3 \times 10^{-4} \text{ m} = \mathbf{0,83 \text{ mm}}.$$

Kiểm chéo bằng tỉ số: $\delta Z/\delta x = 0,83/0,17 = 5$, và $Z/W = 500/100 = 5$. Khớp. Đây là cửa kiểm chứng (b) cho cả hai công thức: chúng nhất quán với nhau ở một quan hệ độc lập với cách dẫn ra chúng.

Đọc kết quả so với ngân sách 5 mm: cả hai con số đều *thoải mái*. Tình tiến không phải vấn đề, và nó không phải vấn đề với một biên độ lớn — ta còn dư gần một bậc độ lớn. Đây là kết luận quan trọng nhất của ví dụ, và nó nên làm người đọc ngạc nhiên, vì nó nói rằng phần lớn công sức mà các dự án bỏ vào việc cải thiện độ chính xác vị trí là công sức tiêu vào chỗ đã dư.

CẠM BẮY

Con số $\sigma = 0,2$ px trong ví dụ trên là **bậc độ lớn kinh nghiệm**, không phải số đo của một thí nghiệm có điều kiện xác định. Nó nằm trong khoảng mà người ta thường quan sát được với góc AprilTag trong điều kiện tốt, và B/08 bàn kỹ hơn về khoảng này cùng với các phương pháp tinh chỉnh khác.

Hệ quả bắt buộc: *mọi con số milimét sinh ra từ nó cũng chỉ là bậc độ lớn*. Chúng đủ tốt để so sánh các phương án thiết kế với nhau, đủ tốt để biết mình dư hay thiếu một bậc, và *không* đủ tốt để nghiệm thu. Trước khi nói “hệ của tôi đạt 5 mm”, bạn phải đo σ của chính hệ mình (C/16) và chạy lại phép tính với con số ấy.

Cách sai lầm phổ biến nhất là lấy con số 0,17 mm ở trên rồi báo cáo nó như độ chính xác của hệ thống. Nó không phải độ chính xác; nó là *thành phần đóng góp của nhiều góc vào độ lặp lại*, một dòng trong một bảng ngân sách còn có nhiều dòng khác — và các dòng khác thường lớn hơn nó nhiều lần.

5.5 Sai số xoay từ một tag phẳng: quy luật hoàn toàn khác

Hai mục trước có cùng cấu trúc: một phép vi phân, một hệ số quy đổi, một công thức sạch. Mục này không như thế, và chính sự khác biệt ấy là điều cần hiểu.

Xét một tag phẳng và hỏi: nghiêng nó đi một góc nhỏ $\delta\theta$ quanh một trục nằm *trong* mặt phẳng tag thì hình chiếu trên ảnh đổi bao nhiêu? Bản chất nằm ở một quan sát hình học đơn giản.

Nghiêng một tấm phẳng quanh trục nằm trong mặt phẳng của nó làm các điểm ở hai mép dịch chuyển theo chiều *sâu* theo hai hướng ngược nhau: mép này lùi ra, mép kia tiến vào, với biên độ $\pm(W/2)\sin\delta\theta$. Sự dịch chuyển theo chiều sâu ấy được camera nhìn thấy qua thay đổi của kích thước biểu kiến cục bộ — tức qua chính cơ chế *yếu* của mục 3B. Ngược lại, xoay tấm phẳng quanh trục *vuông góc* với nó (tức xoay trong mặt phẳng ảnh, quanh trục quang) làm mọi điểm dịch chuyển theo chiều *ngang* với biên độ $(W/2)\delta\theta$, và cái đó được nhìn thấy qua cơ chế *tốt* của mục 3A.

Đó là sự bất đối xứng cốt lõi, và nó cho ngay hai kết quả rất khác nhau. Với xoay trong mặt phẳng ảnh:

$$\delta\theta_{\text{trong mặt phẳng}} \approx \frac{\delta x}{W/2} = \frac{2Z\sigma}{fW} = \frac{2\sigma}{w},$$

tức nó tốt, và nó chỉ phụ thuộc vào kích thước tag trên ảnh. Với tag 120 px và $\sigma = 0,2$ px, con số này là $3,3 \times 10^{-3}$ rad = $0,19^\circ$ — vừa đủ trong ngân sách, và nó là thành phần xoay *đễ nhất*.

Với xoay ngoài mặt phẳng — tức nghiêng tag — thì phải qua cơ chế sâu, và ở đây xuất hiện hệ số sin:

$$\delta\theta_{\text{ngoài mặt phẳng}} \gtrsim \frac{\delta Z}{(W/2)} \sim \frac{2Z\sigma}{wW}, \quad \text{và độ nhạy} \propto \sin\theta \quad (5.4)$$

Tôi viết $\gtrsim$ và $\sim$ chứ không viết dấu bằng, vì con số chính xác phụ thuộc vào phân bố các góc và vào tham số hoá; đây là một *quan hệ tỉ lệ* chứ không phải một đẳng thức, và nó được dùng đúng

như vậy (tôi suy ra từ hình học cơ bản; cấu trúc phù hợp với phân tích trong [26] và [13] nhưng tôi chưa đổi chiều từng hệ số).

Điều then chốt nằm ở mệnh đề thứ hai: *độ nhạy tỉ lệ với $\sin \theta$* . Khi $\theta \rightarrow 0$, tức khi tag ở tư thế fronto-parallel, độ nhạy tiến về không. Nghĩa là: nghiêng tag thêm một chút không làm ảnh đổi tới bậc một — nó chỉ đổi tới bậc hai. Và một đại lượng mà phép đo chỉ phản ứng ở bậc hai là một đại lượng gần như không quan sát được.

TRỰC GIÁC

Vì sao fronto-parallel lại là tệ nhất, nói bằng hình ảnh và bằng số.

Cầm một tấm bìa hình vuông và giơ thẳng trước mặt, vuông góc với hướng nhìn. Bây giờ nghiêng nó đi 1° . Hình vuông có trông khác đi không? Gần như không — nó vẫn là một hình vuông, chỉ ngắn lại theo một chiều đúng $\cos 1^\circ = 0,99985$, tức 0,015%. Với tag 120 px, đó là 0,018 px — chìm hoàn toàn dưới nhiễu 0,2 px.

Bây giờ nghiêng tấm bìa đi 45° rồi nghiêng thêm 1° nữa. Lần này hình dạng đổi rõ: chiều ngắn lại từ $\cos 45^\circ = 0,7071$ xuống $\cos 46^\circ = 0,6947$, tức 1,8% — trên tag 120 px là 2,2 px, hoàn toàn đo được.

Cùng một lượng nghiêng, hai mức đổi ảnh khác nhau hơn trăm lần. Đó là toàn bộ nội dung của $\sin \theta$: đạo hàm của $\cos$ là $-\sin$, và $\sin$ bằng không tại không.

Bài học thực hành theo ngay: *đường tiếp cận vuông góc là đường tệ nhất cho việc đo góc*. Nếu có thể chọn, hãy tiếp cận chếch, hoặc — tốt hơn vì không đụng tới quỹ đạo robot — nghiêng chính các tag đi so với mặt phẳng dock.

5.6 Sai số xoay từ constellation: chia cho baseline

Mục trước kết thúc ở một tin xấu. Mục này là lối thoát, và lối thoát ấy đơn giản đến mức gần như thất vọng.

Giả sử thay vì một tag, ta có hai tag cách nhau một khoảng B đã biết trong hệ dock, cả hai đều nhìn thấy được. Bây giờ hỏi: cả cụm quay đi một góc $\delta\theta$ quanh một trục vuông góc với đường nối hai tag thì chuyện gì xảy ra? Tag thứ nhất dịch chuyển một đoạn $\approx (B/2)\delta\theta$ theo một hướng, tag thứ hai dịch một đoạn như thế theo hướng ngược lại. Nếu trục quay được chọn sao cho dịch chuyển ấy nằm theo *chiều ngang* của ảnh — và với constellation trải rộng thì luôn có thành phần như vậy — thì ta đo nó qua cơ chế *tốt* của mục 3A, không qua cơ chế *sâu*.

Vì vị trí của mỗi tag được xác định với sai số δx , và góc được suy từ chênh lệch vị trí của hai tag chia cho khoảng cách giữa chúng:

$$\delta\theta \approx \frac{\delta x}{B} = \frac{Z\sigma}{fB} \quad (5.5)$$

Đây là quan hệ thứ tư và là quan hệ có giá trị thực dụng cao nhất trong cả cây.

A. Ví dụ số, tiếp nối ví dụ ở mục 4

Giữ nguyên $f = 600$ px, $Z = 0,5$ m, $\sigma = 0,2$ px, nên $\delta x = 0,167$ mm như đã tính. Đặt baseline $B = 0,4$ m:

$$\delta\theta \approx \frac{0,167 \text{ mm}}{400 \text{ mm}} = 4,2 \times 10^{-4} \text{ rad} = 4,2 \times 10^{-4} \times \frac{180}{\pi} = \mathbf{0,024^\circ}.$$

So với ngân sách $0,5^\circ$: ta dư *hơn hai mươi lần*. Đặt cạnh con số “dễ dàng vượt $1\text{--}2^\circ$ ” của một tag phẳng nhìn vuông góc, chênh lệch giữa hai phương án là gần hai bậc độ lớn — và phương án

tốt hơn không đòi một dòng code nào khác, chỉ đòi đặt hai tấm tag cách nhau 40 cm thay vì một tấm.

Đây là câu trả lời cho Q5, và lý do câu trả lời không phụ thuộc con số cụ thể nằm ở cấu trúc của hai công thức. Giảm σ bốn lần thì cả δx lẫn $\delta \theta$ giảm bốn lần — tuyến tính, và có trần: σ không xuống dưới giới hạn quang học được (A/06). Tầng B thì $\delta \theta$ giảm tuyến tính theo B — cũng tuyến tính, nhưng B rẻ và gần như không có trần trong phạm vi kích thước một cái dock. Hơn nữa, và đây mới là điểm quyết định: tầng B đồng thời cho phép *phá tính đồng phẳng* nếu bạn đặt hai tag ở hai độ sâu khác nhau, và cái đó khử luôn bài toán hai nghiệm của A/04 — một lợi ích mà giảm σ hoàn toàn không cho.

CẦN NHỚ

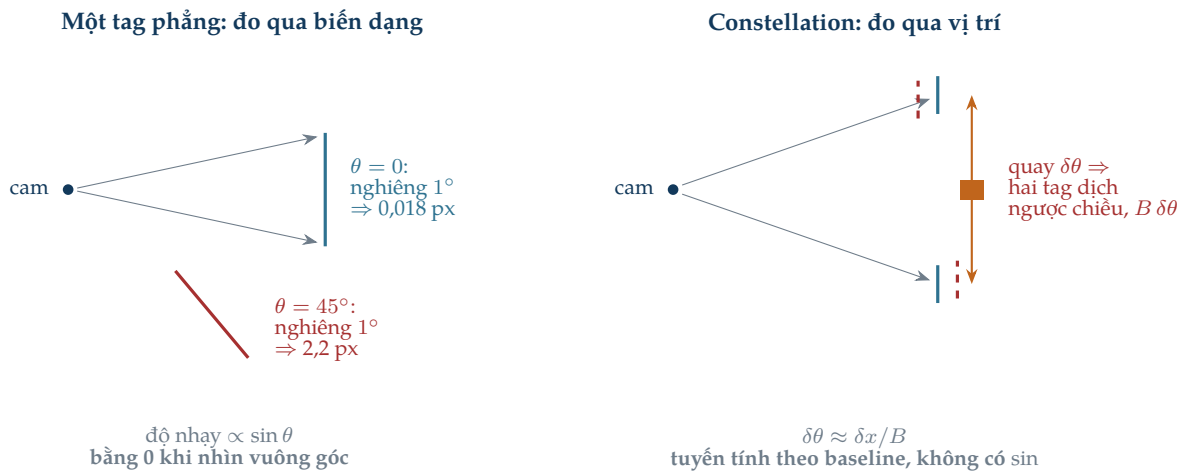
Bốn quan hệ, thuộc lòng cả bốn:

$$\delta x \approx \frac{Z\sigma}{f}, \quad \delta Z \approx \frac{Z\sigma}{w}, \quad \delta \theta_{\text{tag phẳng}} \sim \frac{1}{\sin \theta} \cdot \frac{Z\sigma}{wW}, \quad \delta \theta_{\text{constellation}} \approx \frac{Z\sigma}{fB}.$$

Đọc chúng cạnh nhau thì thấy ngay cấu trúc: tử số luôn là $Z\sigma$; cái quyết định là *mẫu số*. Mẫu số càng lớn càng tốt, và bốn mẫu số là $f, w, wW \sin \theta, fB$.

Kết luận rút ra trong một câu: *muốn xoay tốt thì tăng B và phá đồng phẳng, không phải tăng số tag đồng phẳng*. Thêm một tag thứ ba nằm giữa hai tag cũ hầu như không cải thiện gì vì nó không tăng B ; đẩy một tag ra xa thêm 20 cm thì cải thiện thấy rõ.

Hình 5.1 vẽ hai cơ chế cạnh nhau, và nó là hình nên nhớ cho cả chương.



Hình 5.1. Hai cơ chế ước lượng góc. Bên trái, một tag phẳng: góc nghiêng chỉ lộ ra qua biến dạng phối cảnh, mà biến dạng ấy có đạo hàm tỉ lệ với $\sin \theta$ — bằng không đúng ở tư thế fronto-parallel, tức đúng tư thế của một robot tiếp cận thẳng trục. Hai con số trong hình là ví dụ số học cho tag 120 px (tính ở mục 5), không phải số đo. Bên phải, một constellation baseline B : góc lộ ra qua chênh lệch *vị trí* của hai tag, một đại lượng đo bằng cơ chế tốt, và độ nhảy không có hệ số sin nào cả. Đây là toàn bộ lý do vì sao lời khuyên “tăng baseline” thắng lời khuyên “dùng detector tốt hơn”.

5.7 Vì sao trung bình pose từ từng tag là sai

Công thức (5.5) mang một hệ quả mà nhiều người bỏ lỡ, và nó đủ quan trọng để có mục riêng.

Cách tự nhiên nhất khi nhìn thấy bốn tag là: chạy PnP cho từng tag, được bốn pose, rồi lấy trung bình. Cách này *hỏng*, và nó hỏng theo hai tầng.

Tầng thứ nhất là tầng thống kê, và nó tương đối dễ thấy. Pose sống trên $SE(3)$ chứ không phải trên không gian vectơ, nên trung bình số học của các ma trận xoay không phải một ma trận xoay, và trung bình của quaternion phụ thuộc vào dấu ta gán cho từng quaternion (vì q và $-q$ biểu diễn cùng một xoay). Có cách làm đúng — trung bình Karcher trên $SO(3)$ [33], và B/11 bàn kỹ — nhưng ngay cả khi làm đúng, tầng thứ hai vẫn còn.

Tầng thứ hai là tầng thông tin, và nó mới là tầng quyết định. Hãy nhìn kỹ vào việc ta vừa vứt đi cái gì. Pose của từng tag riêng lẻ có hiệp phương sai rất *dị hướng*: chính xác cỡ 0,17 mm theo hai chiều ngang, 0,83 mm theo chiều sâu, và tệ tới vài độ theo hướng nghiêng. Trung bình bốn đại lượng như vậy cho ra một đại lượng có hiệp phương sai vẫn *dị hướng* như thế, chỉ nhỏ đi $\sqrt{4} = 2$ lần theo mỗi hướng — nên thành phần nghiêng vẫn tệ, chỉ tệ ít hơn hai lần.

Trong khi đó, PnP hợp nhất trên toàn bộ tập góc *biết* rằng bốn tag ấy gắn cứng với nhau ở những vị trí đã biết. Nó không đo bốn pose rồi gộp; nó đo *một* pose bằng cách dùng toàn bộ $4 \times 4 = 16$ góc, và vì các góc ấy trải trên một vùng rộng B , thông tin về hướng đến từ cơ chế baseline chứ không từ cơ chế biến dạng. Cải thiện không phải $\sqrt{4}$ lần mà là cỡ B/W lần, nhưng quan trọng hơn là nó *đổi cơ chế*, và đổi cơ chế thì thoát khỏi hệ số $\sin \theta$.

Nói bằng ngôn ngữ của công thức (5.1): trung bình pose là việc cộng các Σ_i , còn PnP hợp nhất là việc cộng các $\mathcal{I}_i = J_i^T \Sigma_i^{-1} J_i$ trong cùng một hệ toạ độ. Hai phép cộng ấy không giao hoán với phép nghịch đảo, và sự không giao hoán ấy chính là chỗ thông tin hình học nằm.

CẠM BẮY

Trung bình pose từ từng tag không chỉ kém tối ưu — nó còn *che giấu* vấn đề. Bốn pose từ bốn tag đồng phẳng sẽ có thành phần nghiêng nhảy loạn, nhưng trung bình của chúng trông mượt hơn nhiều và độ lệch chuẩn mẫu của bốn giá trị ấy trông nhỏ. Bạn sẽ báo cáo một độ lặp lại tốt cho một đại lượng vẫn đang sai có hệ thống.

Phép thử phân biệt: đừng nhìn độ tán mát giữa các tag, hãy nhìn *sai số tái chiếu*. PnP hợp nhất cho một con số sai số tái chiếu duy nhất, và nếu constellation của bạn sai so với mô hình thì con số ấy lớn lên và bạn thấy. Trung bình pose không có đại lượng tương đương, nên nó không có cách nào báo động.

5.8 Cộng các nguồn sai số: khi nào bình phương, khi nào cộng thẳng

Chương này tính đóng góp của nhiều góc. Ngân sách thật (C/15) có nhiều dòng hơn: sai số constellation so với mô hình, sai số intrinsic, sai số hand-eye, sai số bám của bộ điều khiển. Câu hỏi cộng chúng thế nào không tầm thường và trả lời sai thì ngân sách vô nghĩa.

Quy tắc có hai nhánh, và ranh giới giữa chúng là *ngẫu nhiên hay hệ thống*.

Các nguồn **ngẫu nhiên** (*random*) — độc lập, trung bình không, đổi từ khung này sang khung khác — cộng theo bình phương:

$$\sigma_{\text{tổng}} = \sqrt{\sigma_1^2 + \sigma_2^2 + \dots}$$

Nhiều định vị góc thuộc loại này. Hệ quả quan trọng của phép cộng bình phương: *nguồn nhỏ gần như không đóng góp gì*. Nếu $\sigma_1 = 3$ mm và $\sigma_2 = 1$ mm thì tổng là $\sqrt{9 + 1} = 3,16$ mm — nguồn thứ hai thêm 5%, dù nó bằng một phần ba nguồn thứ nhất. Đây là cơ sở định lượng cho luận điểm “mất xích yếu nhất quyết định” ở 00-map, và nó nói rằng tối ưu hoá một nguồn đã nhỏ là lãng phí.

Các nguồn **hệ thống** (*systematic*) — không đổi hoặc đổi chậm, có dấu xác định — cộng *thẳng* trong trường hợp xấu nhất:

$$e_{\text{tổng}} \leq e_1 + e_2 + \dots$$

Sai số kích thước tag, sai số constellation so với CAD, sai số hand-eye thuộc loại này. Chúng không trung bình mất đi khi bạn lấy nhiều khung — lấy một nghìn khung thì chúng vẫn nguyên vẹn. Đây là lý do vì sao 1 mm sai số cơ khí nguy hiểm hơn nhiều so với 1 mm nhiễu: nhiễu chia cho $\sqrt{N}$, sai số hệ thống thì không.

Và đây là chỗ luận điểm teach-by-showing của B/13 có cơ sở định lượng. Nếu ta điều khiển theo *sai lệch tương đối* so với một pose mục tiêu đã ghi lại, thì mọi nguồn hệ thống xuất hiện ở cả hai vế và *trừ đi nhau*. Ngân sách thu hẹp lại chỉ còn các nguồn ngẫu nhiên — tức chỉ còn những nguồn chia được cho $\sqrt{N}$. Đó là lý do kỹ thuật ấy không phải một mẹo vặt mà là một thay đổi bậc độ lớn trong bài toán.

5.9 Giới hạn và chế độ hỏng

Bốn công thức của chương đứng trên một tập giả thiết, và chúng hỏng khi giả thiết bị vi phạm. Bốn chỗ hỏng dưới đây đều xảy ra thật.

Giả thiết nhiễu Gauss, độc lập, đẳng hướng, trung bình không. Vi phạm phổ biến nhất là *tương quan*: bốn góc của cùng một tag chịu chung một mức mờ, một hướng chiếu sáng, một sai số méo cục bộ, nên nhiễu của chúng tương quan mạnh. Hậu quả: hiệp phương sai thật lớn hơn công thức dự đoán, và cải thiện theo $\sqrt{N}$ không xảy ra. Vi phạm thứ hai là *trung bình khác không*, tức thiên lệch — motion blur đẩy góc về một phía, bias tâm ellipse đẩy tâm theo một hướng phụ thuộc tư thế (A/06). Công thức (5.1) hoàn toàn mù với thiên lệch: nó mô tả độ tản mát quanh *giá trị mà bộ ước lượng hội tụ tới*, chứ không nói giá trị ấy có đúng không.

Giả thiết mô hình đúng. Đã bàn ở mục 2: nếu intrinsic sai hoặc constellation sai, phần dư mang thành phần hệ thống, khai triển Gauss–Newton không còn hợp lệ, và hiệp phương sai báo về nhỏ hơn sự thật. Đây là chế độ hỏng nguy hiểm nhất vì nó đi theo chiều sai lầm: hệ thống trở nên *tự tin hơn* khi nó sai nhiều hơn.

Giả thiết góc nhỏ trong các quan hệ tỉ lệ. Công thức (5.5) dùng xấp xỉ $\sin \delta\theta \approx \delta\theta$ — luôn đúng cho sai số cỡ phần nghìn radian, nên đây không phải giới hạn thực tế. Nhưng (5.4) thì khác: nó là quan hệ tỉ lệ chứ không phải đẳng thức, và hệ số của nó phụ thuộc vào chi tiết phân bố góc mà tôi không dẫn ra. Dùng nó để *so sánh* hai phương án thì được; dùng nó để dự đoán một con số tuyệt đối thì không. Với con số tuyệt đối, hãy chạy Monte Carlo (C/16).

Giả thiết tuyến tính hoá quanh nghiệm đúng. Toàn bộ phân tích là *cục bộ*. Nó không nói gì về khả năng bộ giải hội tụ vào một cực tiểu cục bộ khác — và với target phẳng thì có đúng một cực tiểu khác như thế, chính là nghiệm thứ hai của A/04. Hiệp phương sai tính quanh nghiệm sai vẫn trông đẹp. Đây là lý do chương 4 và chương này phải đọc cùng nhau: chương này định giá nhiễu, chương kia cảnh báo về lỗi thô, và một ngân sách chỉ tính nhiễu là một ngân sách bỏ sót chi phí lớn nhất.

Bảng 5.1. Bốn giả thiết của chương, chỗ hỏng, và dấu hiệu nhận ra.

Giả thiết	Vi phạm điển hình	Dấu hiệu nhận ra
Nhiều độc lập giữa các góc	Cùng tag chịu chung blur, chiếu sáng, méo cục bộ	Lấy trung bình N khung không cải thiện theo $1/\sqrt{N}$; độ tản mát bão hoà sớm
Nhiều trung bình không	Motion blur, bias tâm ellipse, rolling shutter	Sai số đổi dấu khi đổi chiều chuyển động; sai số phụ thuộc tư thế một cách trơn
Mô hình đúng	Intrinsic sai, constellation lệch CAD, méo chưa khử sạch	Sai số tái chiếu có <i>cấu trúc</i> theo vị trí ảnh, không giống nhiễu trắng
Tuyến tính hoá cục bộ	Hội tụ vào nghiệm thứ hai của target phẳng	Pose nhảy giữa hai giá trị; tỉ số sai số tái chiếu của hai nghiệm gần 1

5.10 Thực hành: dùng bốn quan hệ này trong một buổi chiều

Bốn công thức trên có một cách dùng rất cụ thể ở giai đoạn thiết kế, và nó gồm năm bước làm được trên giấy.

Bước một, viết ra bốn con số của bài toán: tiêu cự f tính bằng pixel (từ trường nhìn và độ phân giải, hoặc từ hiệu chuẩn nếu đã có), khoảng cách cuối Z ở thời điểm cần chính xác nhất, kích thước tag W dự kiến, và σ — dùng 0,2 px làm giá trị mặc định thận trọng nếu chưa đo được. *Bước hai*, tính $w = fW/Z$; nếu w nhỏ hơn khoảng 60 px thì dừng lại và xem lại thiết kế, vì decode sẽ bắt đầu không tin cậy ($B/07$) và mọi phép tính sau đó là vô nghĩa. *Bước ba*, tính δx và δZ ; gần như chắc chắn chúng sẽ nhỏ so với ngân sách, và đó là tin tốt cần được ghi nhận chứ không phải bỏ qua. *Bước bốn*, tính $\delta \theta$ cho baseline dự kiến và so với ngân sách góc; đây là dòng quyết định. *Bước năm*, nếu không đạt, giải ngược công thức (5.5) theo B để biết baseline tối thiểu cần bao nhiêu — và đó là con số bạn mang sang cho người thiết kế cơ khí.

Bước năm đáng nhấn mạnh vì nó là chỗ chương này trả lại giá trị lớn nhất. Giải ngược:

$$B_{\min} = \frac{Z \sigma}{f \delta \theta_{\text{cho phép}}}.$$

Với ví dụ của cây ($Z = 0,5$ m, $\sigma = 0,2$ px, $f = 600$ px) và ngân sách góc $0,5^\circ = 8,73 \times 10^{-3}$ rad, ta được $B_{\min} = 0,1/(600 \times 8,73 \times 10^{-3}) = 0,019$ m, tức 19 mm. Con số ấy nhỏ đến mức đáng ngờ, và nó đáng được đọc cho đúng: nó nói rằng *nếu* cơ chế baseline hoạt động thuần tuý và không có nguồn sai số nào khác, thì một baseline 2 cm đã đủ. Thực tế không như vậy, vì các nguồn hệ thống ở mục 8 — đặc biệt là sai số của chính constellation — chiếm phần lớn ngân sách. Cách dùng đúng của con số này là: *nhiều góc không phải ràng buộc cho baseline; ràng buộc thật đến từ độ chính xác cơ khí của tâm marker*, và C/15 làm rõ chuyện đó bằng một bảng đầy đủ.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Công thức gốc của cả chương	$\Sigma_{\text{pose}} \approx (\sum_i J_i^T \Sigma_i^{-1} J_i)^{-1}$ — nghịch đảo ma trận thông tin Fisher [29], [31]. Nó là cận dưới cho <i>mọi</i> bộ ước lượng không chệch, nên đổi thuật toán không vượt qua được nó.
••• Sai số ngang	$\delta x \approx Z \sigma / f$. Tuyến tính theo khoảng cách. Không phụ thuộc kích thước tag — đây là chỗ phản trực giác nhất trong bốn công thức.
••• Sai số dọc trục	$\delta Z \approx Z \sigma / w = Z^2 \sigma / (fW)$. Bậc hai theo khoảng cách. Tệ hơn sai số ngang đúng $f/w = Z/W$ lần.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Tỉ số sâu trên ngang	Z/W — khoảng cách chia kích thước tag. Tag 100 mm ở 500 mm: tệ hơn 5 lần. Ở 3 m: tệ hơn 30 lần.
●●● Xoay từ một tag phẳng	Độ nhảy $\propto \sin \theta$ với θ là góc nghiêng. Bảng không ở fronto-parallel, tức đúng tư thế của robot tiếp cận thẳng trục. Đây là nguồn lỗi xoay số một.
●●● Xoay từ constellation	$\delta\theta \approx \delta x/B = Z\sigma/(fB)$. Không có hệ số sin nào. Cải thiện tuyến tính theo baseline.
●●● Ví dụ số chuẩn của cây	$f = 600$ px, $Z = 0,5$ m, $W = 100$ mm, $\sigma = 0,2$ px $\Rightarrow w = 120$ px, $\delta x = 0,17$ mm, $\delta Z = 0,83$ mm, và với $B = 0,4$ m thì $\delta\theta = 0,024^\circ$. Tất cả tính tay được.
●●● Kết luận một câu của chương	Translation gần như luôn đạt; rotation là nút thắt; và cách chữa nút thắt là hình học (baseline, phá đồng phẳng) chứ không phải thuật toán.
●● Vì sao trung bình pose là sai	Hai tầng: pose sống trên $SE(3)$ nên trung bình số học không hợp lệ [33]; và quan trọng hơn, nó vứt bỏ ràng buộc hình học giữa các tag, tức vứt bỏ đúng cơ chế baseline.
●● Cộng sai số ngẫu nhiên	Theo bình phương: $\sqrt{\sigma_1^2 + \sigma_2^2}$. Hệ quả: nguồn bằng 1/3 nguồn lớn nhất chỉ thêm 5% vào tổng — nên tối ưu nguồn nhỏ là lãng phí.
●●● Cộng sai số hệ thống	Cộng thẳng trong trường hợp xấu nhất, và <i>không</i> chia cho $\sqrt{N}$ khi lấy trung bình nhiều khung. Đây là lý do 1 mm cơ khí nguy hiểm hơn 1 mm nhiễu.
●● Baseline tối thiểu	$B_{\min} = Z\sigma/(f\delta\theta_{\text{cho phép}})$. Với ví dụ chuẩn và ngân sách $0,5^\circ$: 19 mm — nhỏ, và điều đó nói rằng ràng buộc thật không đến từ nhiễu góc mà từ cơ khí.
●● Chế độ hồng nguy hiểm nhất	Mô hình sai (intrinsic, constellation): phần dư có thành phần hệ thống, hiệp phương sai báo về <i>nhỏ hơn</i> sự thật. Hệ thống tự tin hơn khi nó sai nhiều hơn.
●● Dấu hiệu mô hình sai	Sai số tái chiếu có <i>cấu trúc</i> theo vị trí ảnh — vẽ residual theo toạ độ ảnh, nếu thấy vân thì mô hình sai chứ không phải nhiễu lớn [6].
● Vì sao không đưa $\sqrt{N}$ vào công thức	Vì nhiễu giữa các góc cùng một tag tương quan mạnh (chung blur, chung chiếu sáng, chung méo cục bộ), nên $\sqrt{N}$ là lời hứa thường không được giữ. Bỏ nó ra là cách ước lượng thận trọng.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag (apriltag_pose.c)	Tính sai số tái chiếu cho hai nghiệm IPPE — đo trực tiếp thứ mà mục 9 gọi là cảnh báo lỗi thô	Chỗ lý thuyết chương này gặp code; đọc kèm A/04
OpenCV solvePnPGeneric	Trả về nhiều nghiệm kèm sai số tái chiếu từng nghiệm	API duy nhất cho bạn đại lượng cần để đo ambiguity
Hiệu chuẩn bằng target phẳng [2]	Dùng target rộng và nhiều tư thế để ma trận thông tin không suy biến theo hướng nào — đúng nguyên lý baseline của mục 6	Hiệu chuẩn và docking là cùng một bài toán hình học
tag slam [28]	Bài toán hợp nhất trên toàn bộ tag thay vì trung bình pose từng tag	Hiện thân của mục 7; mức tin C, dùng làm gợi ý kiến trúc
Đánh giá quỹ đạo [34]	Cùng tư duy: chọn sai phép căn chỉnh thì con số sai số vô nghĩa	Song song về phương pháp luận, khác lĩnh vực

MINI PROJECT · Kiểm bốn quan hệ tỉ lệ bằng Monte Carlo, không cần phần cứng · 1–2 ngày

Mục tiêu	xác nhận bằng số rằng bốn công thức của chương đúng tới hệ số, và tự tay nhìn thấy hệ số $\sin \theta$ làm sai số xoay nổ tung khi tag về fronto-parallel.
Dữ liệu	tự sinh hoàn toàn. Không cần camera, không cần tag thật. Một hàm chiếu điểm với K đã biết, một tập góc 3D, một bộ sinh nhiễu Gauss.
Các bước	dựng một tag vuông cạnh W với bốn góc trong hệ dock; đặt nó ở khoảng cách Z , chiếu ra ảnh bằng K ; thêm nhiễu Gauss σ vào từng tọa độ góc; chạy solvePnPGeneric; lặp 2000 lần và lấy độ lệch chuẩn của từng thành phần pose; lập toàn bộ quy trình khi quét Z , quét W , quét σ , và quét góc nghiêng θ từ 0° tới 60° ; cuối cùng thay một tag bằng hai tag cách nhau B và quét B .
Xong khi	bạn có bốn đồ thị log-log, và độ dốc của chúng khớp với bốn công thức: δx dốc +1 theo Z , δZ dốc +2 theo Z , $\delta \theta$ dốc -1 theo B , và $\delta \theta_{\text{tag}}$ tăng vọt khi $\theta \rightarrow 0$. Kiểm tra chéo bắt buộc: với θ nhỏ, hãy <i>cũng</i> kiểm xem solvePnPGeneric có trả về hai nghiệm với sai số tái chiếu gần bằng nhau không — nếu có thì phần lớn độ tán mát bạn đang đo là lỗi thô do chọn nhầm nghiệm chứ không phải nhiễu, và hai thứ đó phải được báo cáo riêng.
Nối với	C/16 — script này chính là hạt nhân của mini project ở đó; A/04 cho phần hai nghiệm.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Không phải gấp đôi trong thực tế, dù công thức nói vậy. Tăng độ phân giải gấp đôi với cùng ống kính làm f tính bằng pixel tăng gấp đôi, nên $\delta x = Z\sigma/f$ giảm một nửa — nếu σ tính bằng pixel giữ nguyên. Nó thường không giữ nguyên: pixel nhỏ hơn thu ít photon hơn nên tỉ số tín hiệu trên nhiễu kém đi, và quang học có thể không phân giải nổi mức chi tiết mới (giới hạn MTF, A/06). Nguyên tắc chung: f là đại lượng hình học thật, còn σ là đại lượng phụ thuộc toàn bộ chuỗi quang học và thuật toán, nên đừng cải thiện một cái mà quên kiểm cái kia.
- A2. Tag 200 mm cho $w = 240$ px, nên $\delta Z = 0,5 \times 0,2/240 = 0,42$ mm — cải thiện đúng hai lần, tuyến tính theo W . Nhưng hai tag 100 mm cách nhau 200 mm cho một thứ mà tag lớn không cho: một baseline. Sai số xoay của nó là $\delta x/B = 0,167/200 = 8,3 \times 10^{-4}$ rad = $0,048^\circ$, trong khi tag đơn 200 mm vẫn chịu hệ số $\sin \theta$ nên ở fronto-parallel thì tệ hơn hàng chục lần. Nói ngắn: tag lớn mua cho bạn chiều sâu, constellation mua cho bạn góc — và góc mới là nút thắt.
- A3. Bằng f/w , tương đương Z/W . Lý do cơ chế: chiều ngang được đo *trực tiếp* qua vị trí trên ảnh với hệ số quy đổi Z/f ; chiều sâu được đo *gián tiếp* qua kích thước biểu kiến với hệ số quy đổi Z/w . Vì w luôn nhỏ hơn f rất nhiều (tag chiếm một phần nhỏ của trường nhìn), chiều sâu luôn tệ hơn. Cách nhớ: bạn đo vị trí bằng cả khung hình làm thước, còn đo khoảng cách chỉ bằng chính cái tag làm thước.
- A4. Vì góc nghiêng chỉ lộ ra qua *biến dạng phối cảnh*, và biến dạng ấy tỉ lệ với $\cos \theta$ — một hàm có đạo hàm $-\sin \theta$, bằng không tại $\theta = 0$. Cụ thể: nghiêng 1° từ tư thế vuông góc làm cạnh ngắn lại 0,015%, tức 0,018 px trên tag 120 px, chìm dưới nhiễu. Nghiêng 1° từ tư thế 45° làm cạnh ngắn lại 1,8%, tức 2,2 px, đo được thoải mái. Cùng một lượng nghiêng, hai mức tín hiệu cách nhau hơn trăm lần. Lối thoát có hai: tiếp cận chệch có chủ ý, hoặc — tốt hơn vì không đụng tới quỹ đạo robot — nghiêng chính các tag trong constellation.
- A5. Baseline, và chênh lệch là lớn chứ không sát sao. Giảm σ bốn lần (0,2 xuống 0,05 px — một cải thiện rất khó, đòi chuyển sang ChArUco cộng quang học tốt cộng trung bình nhiều khung) làm $\delta \theta$ giảm đúng bốn lần. Thêm 30 cm baseline lên một constellation đang có 10 cm làm $\delta \theta$ giảm bốn lần bằng một việc duy nhất là bắt vít hai tấm tag xa nhau hơn. Lý do câu trả lời không phụ thuộc con số cụ thể: cả hai đều tuyến tính, nên tỉ lệ giá trên hiệu quả quyết định, và baseline gần như miễn phí trong phạm vi kích thước một cái dock trong khi σ chạm trần vật lý rất nhanh. Thêm nữa, baseline lớn hơn thường đi kèm khả năng đặt tag ở hai độ sâu khác nhau, và cái đó khử luôn bài toán hai nghiệm — một lợi ích mà giảm σ hoàn toàn không cho.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng bạn đứng ở một đầu sân bóng và có người hỏi về một cái biển hiệu ở đầu kia.

Biển nằm lệch trái phải bao nhiêu? Dễ. Bạn nhìn nó nằm ở đâu so với khung hình trong mắt mình, rồi quy ra. Nhìn càng rõ thì trả lời càng chính xác, và độ chính xác tỉ lệ thuận với việc bạn nhìn rõ tới đâu.

Biển cách bao xa? Khó hơn nhiều, vì bạn không nhìn thẳng được khoảng cách. Bạn phải suy: “biển ấy cao chừng một mét, mà trông nó nhỏ thế này, vậy nó phải xa chừng ấy.” Vấn đề là bạn đang lấy chính cái biển làm thước đo, mà cái biển thì nhỏ. Đo bằng một cái thước nhỏ thì sai nhiều hơn đo bằng một cái thước lớn — đó là toàn bộ lý do khoảng cách khó hơn vị trí, và nó khó hơn đúng bằng tỉ số giữa cái sân và cái biển.

Biển đang xoay nghiêng bao nhiêu độ? Đây mới là chỗ thú vị, và nó không khó hơn một chút mà khó hơn theo một kiểu khác hẳn. Nếu cái biển đang quay mặt thẳng về phía bạn, thì nghiêng nó đi một độ trông *y* hết. Bạn không phát hiện được, không phải vì mắt bạn kém mà vì thật sự không có gì để phát hiện. Còn nếu cái biển đang nghiêng sẵn bốn mươi lăm độ, thì nghiêng thêm một độ nhìn thấy rõ ngay. Cùng một câu hỏi, cùng một con mắt, hai câu trả lời khác nhau hàng trăm lần — và cái quyết định là *tư thế của cái biển*, không phải chất lượng của mắt bạn.

Giờ đến mẹo. Giả sử thay vì một cái biển, ở đầu sân có *hai* cái cột cách nhau mười mét, và bạn biết chúng cách nhau đúng mười mét. Cả cụm quay đi một độ thì sao? Cột này dịch sang trái, cột kia dịch sang phải, mỗi cột dịch gần một tấc. Mà việc nhìn xem một cái cột dịch sang trái hay phải thì bạn giỏi — đó là câu hỏi loại thứ nhất, câu hỏi dễ. Bạn vừa biến một câu hỏi khó thành một câu hỏi dễ, chỉ bằng cách nhìn hai vật thay vì một.

Và đây là điều đáng nhớ nhất: cách làm ấy không đòi mắt tinh hơn. Nó đòi hai cái cột đặt xa nhau. Đặt chúng cách nhau hai mươi mét thay vì mười thì bạn đo góc chính xác gấp đôi — vẫn với đúng con mắt ấy.

Chỗ không rút gọn được: ranh giới giữa “nhiều” và “sai số hệ thống” không nhìn ra được từ một lần đo. Cả hai đều làm con số lệch khỏi sự thật; chỉ có điều một loại thì lấy trung bình nhiều lần sẽ hết, loại kia thì không. Phân biệt chúng đòi hoặc một chuẩn bên ngoài để so, hoặc một phép thử thay đổi điều kiện có chủ ý — và đó là việc phải làm bằng tay, không có công thức nào thay được.

Một câu: Sai số chia cho tiêu cự thì nhỏ, chia cho kích thước tag thì lớn hơn, còn góc thì đừng chia cho gì cả — hãy chia cho baseline.

5.11 Điều gì chứng minh cách hiểu này sai

Theo quy tắc của kho, một chương phải nói rõ điều gì sẽ bác bỏ nó. Với chương này, các mệnh đề bác bỏ được rất cụ thể — và may mắn là chúng đều kiểm được bằng script Monte Carlo trong một buổi.

Mệnh đề chính — $\delta\theta$ từ constellation tỉ lệ nghịch với B , không có hệ số $\sin\theta$ — bị bác bỏ nếu một mô phỏng cẩn thận cho thấy sai số xoay của một constellation không đồng phẳng *vẫn* tăng vọt khi camera về tư thế fronto-parallel so với mặt phẳng trung bình của constellation. Tôi đã dẫn mệnh đề này bằng hình học góc nhỏ (cửa a) nhưng *chưa* chạy mô phỏng, nên nó chưa qua cửa (b) — và đây là chỗ nợ kiểm chứng lớn nhất của chương, đã ghi vào 99-chua-biet.md.

Mệnh đề thứ hai — $\delta Z/\delta x = Z/W$ — bị bác bỏ nếu đo được một tỉ số khác đáng kể trong mô phỏng với nhiễu Gauss độc lập. Đây là mệnh đề tôi tin nhất vì nó đã qua hai cửa: tự dẫn bằng vi phân, và kiểm nhất quán bằng ví dụ số ở mục 4 (hai công thức độc lập cho cùng tỉ số 5).

Mệnh đề thứ ba — PnP hợp nhất tốt hơn trung bình pose, và tốt hơn theo hệ số cỡ B/W chứ không phải $\sqrt{N}$ — bị bác bỏ nếu mô phỏng cho thấy hai cách cho cùng độ tản mát góc trên một constellation trải rộng. Đây là mệnh đề dễ kiểm nhất và nên kiểm sớm nhất, vì nó ảnh hưởng trực tiếp tới kiến trúc phần mềm.

Cuối cùng, một mệnh đề mềm hơn nhưng đáng nêu: chương này khẳng định rằng *trong bài toán docking, thành phần xoay luôn là ràng buộc chặt hơn thành phần tịnh tiến*. Điều đó bị bác bỏ nếu trong

một hệ thống thật, sai lệch cuối cùng bị chi phối bởi thành phần tịnh tiến — và điều ấy hoàn toàn có thể xảy ra nếu sai số hand-eye hoặc sai số bám của bộ điều khiển lớn, vì hai nguồn ấy không nằm trong phân tích của chương này. Nói cách khác, kết luận của chương đúng trong phạm vi *chuỗi quang học tới pose*, và C/15 là chương kiểm xem nó còn đúng khi tính cả chuỗi.

5.12 Kết nối

Chương này là bản lề của cả cây: nó nhận công cụ từ ba chương trước và cấp số liệu cho mọi chương sau.

Nối lên trên trước. [A/01-mo-hinh-camera-va-distortion](#) cho f , đại lượng camera duy nhất trong mọi công thức ở đây — và nó cũng cảnh báo rằng nếu méo chưa khử sạch thì giả thiết “mô hình đúng” ở mục 2 bị vi phạm và hiệp phương sai báo về là sai. [A/03-pnp-va-uoc-luong-tu-the](#) cho Jacobian J ; chương này chỉ làm một việc là đảo $J^T J$ và đọc ý nghĩa của các phần tử. [A/04-pose-ambiguity-target-phang](#) là chương song sinh: nó mô tả lỗi thô, chương này mô tả nhiễu, và một phân tích chỉ có một trong hai là một phân tích thiếu. Trên thực tế hai chương ấy nói cùng một sự thật ở hai mức: mục 5 ở đây nói độ nhạy tiến về không khi fronto-parallel, còn chương 4 nói rằng khi độ nhạy về không thì hai nghiệm trở nên không phân biệt được. Đó là cùng một hiện tượng nhìn từ hai phía.

Nối xuống dưới, theo thứ tự cấp bách. C/15-ngan-sach-sai-so là chương nhận trực tiếp và đầy đủ nhất: nó lấy bốn công thức này, gán số thật, cộng thêm các dòng hệ thống theo quy tắc ở mục 8, và ra bảng ngân sách. Nếu bạn chỉ đọc hai chương trong cả cây thì hãy đọc chương này và chương 15. B/09-multi-marker-constellation biến mục 6 thành hướng dẫn thiết kế cụ thể: đặt bao nhiêu tag, ở đâu, nghiêng bao nhiêu. B/08-corner-refinement là chương quyết định σ , tức đại lượng duy nhất ở đây mà phần mềm còn ảnh hưởng được. B/11-uoc-luong-theo-thoi-gian khai thác chỗ mà mục 8 để ngỏ: nguồn ngẫu nhiên chia cho $\sqrt{N}$, nên trung bình nhiều khung là cách rẻ nhất để giảm σ — và nó cũng là chương giải thích vì sao cách đó *không* giúp gì cho các nguồn hệ thống.

Cuối cùng, một nối ngang quan trọng nhất về mặt thực hành. B/13-docking-control lấy mục 8 làm nền lý thuyết cho luận điểm teach-by-showing: nếu điều khiển theo sai lệch tương đối, mọi dòng *hệ thống* trong ngân sách triệt tiêu và chỉ còn các dòng *ngẫu nhiên*, tức chỉ còn những dòng chia được cho $\sqrt{N}$. Chương này cung cấp lý do định lượng cho việc vì sao mẹo ấy không phải một mẹo nhỏ mà là một thay đổi bậc độ lớn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao sai số ngang không phụ thuộc kích thước tag còn sai số dọc thì phụ thuộc — hãy trả lời bằng cách chỉ ra hai đại lượng đóng vai trò “thước đo” trong hai trường hợp?
2. Một đồng nghiệp đề xuất tăng số tag đồng phẳng từ bốn lên mười sáu để cải thiện độ chính xác góc. Cải thiện kỳ vọng là bao nhiêu lần, và vì sao nó không giải quyết được vấn đề gốc?
3. Bạn đo được sai số xoay thực tế lớn hơn công thức dự đoán mười lần. Nêu ba nguyên nhân hoàn toàn khác nhau, và với mỗi nguyên nhân nêu một phép đo phân biệt nó với hai nguyên nhân kia.
4. Công thức $\Sigma \approx (J^T \Sigma_{\text{px}}^{-1} J)^{-1}$ bỏ đi một số hạng trong đạo hàm bậc hai của hàm mục tiêu. Số hạng đó là gì, khi nào nó không triệt tiêu, và hậu quả đi theo chiều an toàn hay chiều nguy hiểm?
5. Bạn lấy trung bình 100 khung hình khi robot đứng yên. Nguồn sai số nào giảm 10 lần, nguồn nào không giảm chút nào, và làm sao phân biệt hai loại đó chỉ bằng dữ liệu đo chứ không cần chân trị?

6. Với $Z = 2$ m thay vì 0,5 m và mọi thứ khác giữ nguyên, bốn con số của ví dụ chuẩn thành bao nhiêu? Tính nhầm, rồi nói xem con số nào vượt ngân sách trước.
7. Vì sao $B_{\min} = 19$ mm tính ở mục 10 là một con số đúng nhưng dùng sai cách thì nguy hiểm — điều gì nó không tính tới?

Nguồn

- [1] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, “Propagating Covariance in Computer Vision,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, “Statistical Optimization for Geometric Computation: Theory and Practice,” 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., “Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera,” in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., “Rolling Shutter Camera Calibration,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, “Introduction to Fourier Optics,” 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

Hiệu ứng vật lý và quang học (Physical and Optical Effects)

PHỤ THUỘC A/01-mo-hinh-camera-va-distortion (mô hình camera — chương này liệt kê những gì mô hình ấy bỏ qua); A/05-lan-truyen-sai-so-pixel-sang-pose (phân loại nguồn ngẫu nhiên và hệ thống — chương này chủ yếu nói về loại thứ hai).

TÓM TẮT

Năm chương trước dựng một chuỗi toán học sạch: điểm 3D chiếu thành pixel, pixel cho pose, nhiều pixel cho hiệp phương sai pose. Chuỗi ấy giả định camera là một thiết bị hình học lý tưởng chụp một cảnh đứng yên trong điều kiện lý tưởng. Chương này liệt kê những chỗ giả định ấy sai, và nó được tổ chức theo một tiêu chí duy nhất: *hiệu ứng nào gây nhiễu và hiệu ứng nào gây thiên lệch*. Sự phân biệt ấy quyết định cách xử lý, vì nhiễu giảm theo $1/\sqrt{N}$ khi lấy trung bình còn thiên lệch thì không. Sáu hiệu ứng được bàn: rolling shutter, motion blur, lấy mẫu và răng cưa, độ sâu trường ảnh và MTF, bias tâm ellipse, và đặc trưng quang học của vật liệu marker. Nếu chỉ nhớ một điều: *gần như tất cả chúng gây thiên lệch chứ không gây nhiễu* — và vì thế lời khuyên rẻ nhất của cả chương là dừng hẳn robot lại trước khi đo, vì bốn trong sáu hiệu ứng biến mất khi vận tốc bằng không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **rolling shutter** — cách đọc cảm biến theo từng hàng nối tiếp nhau thay vì cùng lúc. Hàng cuối được phơi sáng muộn hơn hàng đầu một khoảng bằng t_{line} nhân số hàng, nên một vật chuyển động xuất hiện *méo* chứ không chỉ mờ.
- **global shutter** — cách đọc mà mọi pixel bắt đầu và kết thúc phơi sáng cùng lúc. Đắt hơn, độ nhạy thường thấp hơn một chút, và — với bài toán đo lường khi đang chuyển động — gần như bắt buộc.
- **MTF** — *modulation transfer function*: hàm mô tả mức tương phản mà hệ quang học còn giữ được ở từng tần số không gian. MTF thấp ở tần số cao nghĩa là các chi tiết nhỏ bị nhòe, và cạnh tag là một chi tiết tần số cao.
- **đĩa Airy** — điểm sáng nhỏ nhất mà một hệ quang học hoàn hảo tạo ra được, giới hạn bởi nhiễu xạ chứ không bởi chất lượng chế tạo. Đường kính $\approx 2,44\lambda N$ với λ là bước sóng và N là số f (f-number).
- **độ sâu trường ảnh (depth of field)** — khoảng khoảng cách mà trong đó ảnh còn đủ sắc nét theo một tiêu chí cho trước. Với docking, tag ở gần và tag ở xa có thể không cùng nằm trong khoảng ấy, và đó là một ràng buộc thiết kế thật.
- **răng cưa (aliasing)** — hiện tượng khi tần số không gian của cảnh vượt quá một nửa tần số lấy mẫu của cảm biến; chi tiết nhỏ hơn hai pixel không chỉ mất đi mà còn xuất hiện lại dưới dạng mẫu hình giả.
- **retroreflective** — vật liệu phản xạ ánh sáng ngược về phía nguồn thay vì tán xạ đều. Với đèn gắn cạnh camera, nó cho tương phản rất cao và gần như độc lập với ánh sáng môi trường.

CÂU HỎI MỞ ĐẦU

- Q1. Robot của bạn chạy 0,3 m/s và camera phơi sáng 20 ms. Vệt nhoè trên ảnh dài bao nhiêu pixel ở khoảng cách nửa mét, và vì sao con số ấy gây *thiên lệch* chứ không chỉ làm mờ?
- Q2. Cùng một hệ thống, đo khi robot đang tiến và đo khi robot đang lùi qua cùng một điểm, cho hai kết quả lệch nhau có hệ thống theo hai chiều ngược nhau. Hiệu ứng nào giải thích được điều đó, và vì sao nó là một dấu hiệu chẩn đoán rất mạnh?
- Q3. Bạn tăng độ phân giải camera lên gấp bốn và sai số không cải thiện chút nào. Nêu hai cơ chế quang học hoàn toàn khác nhau có thể giải thích, và một phép thử phân biệt chúng.
- Q4. Trong sáu hiệu ứng của chương này, bốn cái biến mất khi robot dừng hẳn. Vì sao đó là lập luận mạnh nhất ủng hộ chiến lược stop-and-go, và hai cái còn lại là gì?
- Q5. Bạn khép khẩu để tăng độ sâu trường ảnh, vì cần cả tag gần lẫn tag xa đều nét. Vì sao khép quá thì ảnh lại nhoè *hơn*, và giới hạn vật lý nào đang chặn bạn?

6.1 Đặt vấn đề

Năm chương trước xây một chuỗi lập luận sạch, và độ sạch ấy có được nhờ một loạt giả định không bao giờ được nói ra: camera chụp tức thời, cảnh đứng yên trong suốt thời gian phơi sáng, ống kính phân giải được mọi chi tiết cần thiết, cảm biến lấy mẫu đủ dày, và bề mặt marker phản xạ ánh sáng một cách hiền lành. Chương này gom tất cả những giả định ấy lại và hỏi chúng sai ở đâu.

Có một lý do cụ thể để làm việc này trong một chương riêng thay vì rải rác. Lý do ấy là: *gần như mọi hiệu ứng vật lý trong danh sách đều gây thiên lệch chứ không gây nhiễu*, và A/05 mục 8 đã chỉ ra rằng hai loại ấy có số phận hoàn toàn khác nhau trong một ngân sách sai số. Nhiễu chia cho $\sqrt{N}$ khi lấy trung bình; thiên lệch thì không chia cho gì cả. Một hệ thống đã tối ưu tốt phần nhiều — detector tốt, subpixel tốt, trung bình nhiều khung — sẽ thấy ngân sách của mình bị chi phối hoàn toàn bởi các dòng trong chương này.

Điều làm chúng khó chịu hơn nữa là chúng *phụ thuộc điều kiện vận hành*. Sai số méo ảnh ở A/01 ít ra là cố định: đo một lần, bù một lần. Còn motion blur phụ thuộc vận tốc, rolling shutter phụ thuộc vận tốc, độ nhoè do lấy nét phụ thuộc khoảng cách, loá phụ thuộc góc chiếu sáng. Chúng đổi trong khi robot đang chạy, và chúng đổi theo những cách tương quan với chính đại lượng ta đang đo — đó là kịch bản tệ nhất cho một phép đo, vì sai số tương quan với đại lượng đo không tách ra được bằng bất kỳ phép thống kê nào.

Cách xử lý hiển nhiên là mô hình hoá từng hiệu ứng rồi bù. Cách này tồn tại và nó hoạt động: có mô hình rolling shutter dùng để *ước lượng* cả pose lẫn vận tốc từ một ảnh duy nhất [35], có cách đo t_{line} bằng một target đã biết [36], có mô hình bù bias tâm ellipse bằng conic [11]. Nhưng với bài toán docking thì có một cách rẻ hơn nhiều và nó là kết luận trung tâm của chương: *bốn trong sáu hiệu ứng biến mất khi vận tốc bằng không*. Dừng robot lại một phần tư giây, chụp ba mươi khung, lấy trung bình, rồi đi tiếp — và bạn vừa xoá bốn dòng khỏi bảng ngân sách mà không cần một dòng code mô hình hoá nào. Đó là cơ sở vật lý của chiến lược stop-and-go ở B/13, và nó là một ví dụ điển hình của luận điểm trung tâm của cây: bài toán này thắng bằng thiết kế chứ không bằng thuật toán.

TRỰC GIÁC

Hình dung sự khác nhau giữa nhiễu và thiên lệch bằng hình ảnh bắn bia. Nhiễu là các phát đạn tản mát quanh tâm bia. Bắn nhiều phát rồi lấy trung bình thì trung bình tiến về tâm — càng nhiều phát càng gần. Đây là tình huống mà mọi công cụ thống kê hoạt động: lấy mẫu nhiều hơn thì biết chắc hơn.

Thiên lệch là các phát đạn tản mát quanh một điểm *lệch khỏi tâm*. Bắn một nghìn phát rồi lấy trung bình thì bạn có một ước lượng rất chính xác của cái điểm lệch ấy. Bạn biết chắc hơn về một con số sai. Và — đây là phần tệ nhất — độ tản mát của một nghìn phát nói cho bạn biết bạn “chắc chắn” tới đâu, nên bạn sẽ báo cáo một khoảng tin cậy rất hẹp quanh một giá trị sai.

Phần lớn nội dung chương này thuộc loại thứ hai. Ống kính làm cạnh nhoè theo một cách nhất quán, nên góc lệch theo cùng một hướng ở mọi khung. Cảm biến đọc hàng dưới muộn hơn hàng trên, nên khi robot chạy thì tag bị xé theo cùng một kiểu ở mọi khung. Bề mặt marker loá về phía đèn, nên cạnh phía ấy bị ăn mất theo cùng một cách.

Và bây giờ là mẹo. Với ba trong số đó, cái quyết định hướng lệch là *vận tốc*. Đảo chiều chuyển động thì lệch đổi dấu. Đứng yên thì lệch bằng không. Nghĩa là có một nút bấm xoá được chúng, và nút ấy tên là “dừng lại”.

6.2 Rolling shutter: tag bị xé, không chỉ bị mờ

Hiệu ứng tệ nhất trong danh sách khi robot đang chuyển động, và cũng là hiệu ứng dễ phòng nhất — bằng cách mua đúng camera.

Cảm biến rolling shutter đọc từng hàng nối tiếp. Hàng thứ k được phơi sáng muộn hơn hàng đầu một khoảng $k t_{\text{line}}$, với t_{line} là thời gian đọc một hàng — thường cỡ hàng chục micro giây. Với cảm biến 960 hàng và $t_{\text{line}} = 20 \mu\text{s}$ (*bậc độ lớn kinh nghiệm, không phải số đo của một cảm biến cụ thể*), độ trễ giữa hàng đầu và hàng cuối là $960 \times 20 \mu\text{s} = 19,2 \text{ ms}$.

Hậu quả: nếu cảnh chuyển động trong khoảng thời gian ấy, các hàng khác nhau của ảnh ghi lại cảnh ở các thời điểm khác nhau. Một tag vuông chuyển động ngang trở thành một hình bình hành; một tag đang quay trở thành một hình méo phi tuyến.

Điều quan trọng là *đây không phải mờ*. Mờ làm cạnh nhoè nhưng giữ đúng vị trí trung bình; rolling shutter làm cạnh *dịch chỗ* theo một hàm của toạ độ dọc. Detector vẫn tìm thấy một tứ giác đẹp, decode vẫn thành công, sai số tái chiếu vẫn nhỏ — và pose thì sai. Đây là một chế độ hỏng hoàn toàn im lặng, đúng loại nguy hiểm nhất.

Với robot chạy $v = 0,3 \text{ m/s}$ ở khoảng cách $Z = 0,5 \text{ m}$ với $f = 600 \text{ px}$, vận tốc biểu kiến trên ảnh là $v f / Z = 0,3 \times 600 / 0,5 = 360 \text{ px/s}$. Trong $19,2 \text{ ms}$, đó là $360 \times 0,0192 = 6,9 \text{ px}$ lệch giữa hàng đầu và hàng cuối của ảnh. Với một tag chiếm 120 hàng thay vì cả 960, phần lệch trong nội bộ tag là $6,9 \times 120 / 960 = 0,86 \text{ px}$ — nhỏ hơn nhưng vẫn gấp bốn lần $\sigma = 0,2 \text{ px}$, và nó là thiên lệch.

Ba cách xử lý, xếp theo thứ tự tôi khuyến.

Một, dùng global shutter. Đây là câu trả lời đúng và nó gần như bắt buộc nếu bạn phải đo trong khi di chuyển. Giá chênh lệch so với rolling shutter cùng phân khúc là nhỏ so với chi phí một tuần gỡ lỗi hiện tượng này.

Hai, đo khi đứng yên. Nếu chiến lược điều khiển là stop-and-go (B/13) thì rolling shutter không còn là vấn đề, và bạn tiết kiệm được tiền camera.

Ba, mô hình hoá và bù. Đưa vận tốc vào mô hình đo và giải cả pose lẫn vận tốc từ một ảnh [35]. Nó hoạt động và nó thậm chí biến một phiên toái thành một nguồn thông tin, nhưng nó đòi biết t_{line} chính xác (đo được theo [36]) và nó làm bài toán phức tạp hơn hẳn. Chỉ nên dùng khi hai cách trên bị loại vì ràng buộc ngoài.

6.3 Motion blur: nhoè có hướng, và nó gây thiên lệch

Hiệu ứng thứ hai của chuyển động, và nó độc lập với rolling shutter — global shutter không cứu được nó.

Trong thời gian phơi sáng t_e , ảnh của tag di chuyển trên cảm biến một đoạn

$$L = v_{\text{ảnh}} t_e = \frac{v f}{Z} t_e.$$

Với các con số của Q1 — $v = 0,3 \text{ m/s}$, $f = 600 \text{ px}$, $Z = 0,5 \text{ m}$, $t_e = 20 \text{ ms}$:

$$L = \frac{0,3 \times 600}{0,5} \times 0,02 = 360 \times 0,02 = 7,2 \text{ px}.$$

Bảy phẩy hai pixel. Đặt cạnh $\sigma = 0,2 \text{ px}$, đây là một hiệu ứng lớn hơn ba mươi lần nhiều.

Phần thứ hai của Q1 — vì sao nó gây thiên lệch chứ không chỉ mờ — đáng dừng lại, vì đây là chỗ trực giác thường sai. Trực giác nói: nhòe là một phép làm mượt đối xứng, nên vị trí trung bình của cạnh không đổi, nên định vị cạnh vẫn không thiên lệch. Trực giác ấy đúng trong một trường hợp lý tưởng: một cạnh cô lập, với vùng sáng và vùng tối trải dài vô hạn về hai phía.

Trong thực tế nó không đúng, vì hai lý do. Thứ nhất, các cạnh của tag *không cô lập* — một ô đen rộng 5 px nằm giữa hai ô trắng, và khi nhòe 7 px thì hai cạnh của ô đen chồng lên nhau, và profile độ sáng thu được không còn đối xứng quanh vị trí cạnh thật. Thứ hai, detector không tìm cạnh bằng cách lấy trung tâm đối xứng mà bằng cách khớp một mô hình lên profile gradient có trọng số [3]; khi profile bị biến dạng bất đối xứng thì mô hình khớp lệch.

Ghi chú trung thực: tôi dẫn cơ chế thiên lệch này bằng lập luận về tính bất đối xứng của profile (cửa a) và tôi tin nó đúng về cấu trúc. Tôi *chưa* đo biên độ thiên lệch bằng mô phỏng (cửa b) và *chưa* tìm được nguồn phát biểu tường minh đúng dạng này cho trường hợp marker (cửa c). Vì vậy phần định lượng nên coi là ước lượng của tôi. Đây là một mục trong 99-chua-biet.md, và nó đo được: mô phỏng một tag, tích phân ảnh qua một đoạn dịch chuyển, chạy detector, so với chân trị.

Điều làm motion blur thành một dấu hiệu chẩn đoán mạnh — và đây là Q2 — là nó *đổi dấu khi đổi chiều chuyển động*. Nếu bạn đo cùng một điểm khi robot đang tiến và khi đang lùi, và hai kết quả lệch nhau có hệ thống theo hai chiều ngược nhau, thì bạn gần như chắc chắn đang nhìn một hiệu ứng phụ thuộc vận tốc — blur hoặc rolling shutter. Không nguồn sai số nào khác trong cả cây có chữ ký ấy: méo ảnh không quan tâm chiều chuyển động, sai số constellation không quan tâm, hand-eye không quan tâm. Đây là một phép thử rẻ và rất thông tin, và nó nên nằm trong quy trình nghiệm thu ở C/16.

Cách xử lý: *giảm* t_e , và điều đó đòi nhiều ánh sáng hơn — dẫn thẳng tới lập luận về marker chiếu sáng chủ động ở B/14. Hoặc *giảm* v , tức stop-and-go. Hoặc chấp nhận và đưa vào ngân sách như một dòng phụ thuộc vận tốc — một dòng khó chịu vì nó không phải hằng số.

6.4 Lấy mẫu, răng cưa, và ngưỡng kích thước tag

Cảm biến lấy mẫu ảnh quang học thành một lưới pixel rời rạc, và định lý lấy mẫu đặt ra một giới hạn cứng: chi tiết có chu kỳ nhỏ hơn hai pixel không khôi phục được, và tệ hơn, nó xuất hiện lại dưới dạng mẫu hình giả.

Với marker, chi tiết nhỏ nhất là một ô bit. Một tag AprilTag họ 36h11 có 6×6 ô dữ liệu cộng viên, tức khoảng 8×8 ô trên toàn cạnh. Nếu tag chiếm w pixel thì mỗi ô chiếm $w/8$ pixel. Theo Nyquist thì cần tối thiểu 2 px mỗi ô, tức $w \geq 16 \text{ px}$ — nhưng đó là giới hạn lý thuyết cho tín hiệu sạch, và thực tế cần nhiều hơn.

Con số thực hành thường thấy là 3–5 px mỗi ô bit, tức w cỡ 24–40 px cho việc decode. (Đây là bậc độ lớn kinh nghiệm, không phải số đo có điều kiện xác định.) Nhưng decode được không có nghĩa là đo được: **A/02** mục 4 chỉ ra rằng số điều kiện của bài toán xấu đi khi tag nhỏ, và **A/05** chỉ ra sai số tỉ lệ $1/w$. Ngưỡng cho đo chính xác cao hơn ngưỡng cho decode đáng kể.

Hệ quả thiết kế trực tiếp, và nó là một trong những quyết định quan trọng nhất ở giai đoạn thiết kế: chọn số ô bit ít nhất có thể. Một họ tag với 4×4 ô dữ liệu cần ít pixel hơn một họ 6×6 để đọc được ở cùng khoảng cách, đổi lại số ID khả dụng ít hơn và khoảng cách Hamming nhỏ hơn. Với bài toán docking, số ID cần thiết thường rất nhỏ — vài chục là quá đủ — nên đánh đổi ấy gần như luôn nghiêng về phía ít ô hơn. Đây là một chỗ mà lựa chọn mặc định (dùng họ phổ biến nhất) thường không phải lựa chọn tốt nhất.

6.5 Quang học: MTF, nhiễu xạ, và độ sâu trường ảnh

Ba hiệu ứng quang học liên quan chặt với nhau, và chúng cùng nhau đặt một trần cứng lên σ mà không thuật toán nào vượt qua được.

A. MTF: cạnh không bao giờ sắc tuyệt đối

Ống kính thật không tạo ra một cạnh chuyển từ trắng sang đen trong không tới một pixel; nó tạo ra một chuyển tiếp trải qua vài pixel. Mức độ trải ấy được mô tả bởi MTF, và nó tệ hơn ở rìa ảnh, tệ hơn khi khẩu mở to, và tệ hơn ở tần số không gian cao.

Ảnh hưởng lên định vị góc thì tinh tế. Một cạnh trải rộng không nhất thiết làm định vị kém đi — ngược lại, một cạnh trải qua vài pixel cho nhiều pixel mang thông tin gradient hơn, nên khớp đường có thể chính xác hơn một cạnh nhảy bậc trong một pixel. Điều này nghe phản trực giác nhưng nó là lý do vì sao ảnh hơi mờ đôi khi cho subpixel tốt hơn ảnh cực sắc.

Chỗ MTF thật sự gây hại là khi nó không đồng đều: nếu một cạnh của tag nằm ở vùng ảnh sắc nét và cạnh đối diện nằm ở vùng mờ hơn, hai cạnh ấy được định vị với độ chính xác khác nhau và có thể với thiên lệch khác nhau. Đây là một nguồn cho việc Σ_i khác nhau giữa các góc ở **A/03** mục 6.

B. Nhiễu xạ: trần vật lý

Kể cả một ống kính hoàn hảo cũng không hội tụ ánh sáng vào một điểm, vì bản chất sóng của ánh sáng. Điểm nhỏ nhất là đĩa Airy, đường kính

$$d_{\text{Airy}} \approx 2,44 \lambda N,$$

với λ bước sóng và N số f [37]. Với ánh sáng trắng ($\lambda \approx 550$ nm) và $N = 8$:

$$d = 2,44 \times 0,55 \mu\text{m} \times 8 = 10,7 \mu\text{m}.$$

Với pixel 3 μm , đĩa Airy trải qua hơn ba pixel. Nghĩa là ở $N = 8$, camera của bạn không thể phân giải chi tiết một pixel, bất kể cảm biến có bao nhiêu megapixel.

Đây là câu trả lời cho Q5 và cho một nửa của Q3. Khép khẩu làm tăng độ sâu trường ảnh — điều bạn muốn — nhưng nó cũng tăng N nên tăng đường kính Airy, làm ảnh nhoè đi vì nhiễu xạ. Tồn tại một khẩu độ tối ưu, và khép quá thì tệ hơn.

Đây cũng là một nửa câu trả lời cho Q3: tăng độ phân giải không giúp gì nếu quang học đã là nút thắt. Nửa còn lại là nhiễu ảnh — pixel nhỏ hơn thu ít photon hơn nên tỉ số tín hiệu trên nhiễu kém hơn. Phép thử phân biệt hai cơ chế: chụp cùng một cảnh với thời gian phơi sáng dài hơn và nhiều ánh sáng hơn. Nếu σ cải thiện thì nút thắt là nhiễu ảnh; nếu không thì nút thắt là quang học.

C. Độ sâu trường ảnh: ràng buộc thiết kế thật

Trong bài toán docking, robot đi từ vài mét xuống nửa mét, và constellation có tag ở các độ sâu khác nhau (theo lời khuyên của A/04). Không phải mọi thứ đều nét cùng lúc.

Ba lựa chọn. *Khoá nét ở khoảng cách cuối* — đơn giản nhất và thường đúng, vì đó là chỗ cần chính xác nhất; ở xa thì tag hơi mờ nhưng vẫn decode được và bạn chỉ cần điều khiển thô. *Khép khẩu* — tăng độ sâu trường ảnh, giới hạn bởi nhiễu xạ như mục B. *Lấy nét tự động* — tránh, vì nó đổi f và hệ số méo (A/01 mục 8), làm toàn bộ hiệu chuẩn của bạn vô hiệu.

CẦN NHỚ

Quang học đặt một *trần cứng* lên σ , và trần ấy không vượt qua được bằng phần mềm. Hệ quả cho việc lập ngân sách: trước khi hứa hẹn $\sigma = 0,05$ px, hãy kiểm xem quang học của bạn có phân giải nổi mức đó không. Phép kiểm rẻ: chụp một cạnh sắc, vẽ profile độ sáng vuông góc cạnh, và xem chuyển tiếp trải qua bao nhiêu pixel. Nếu nó trải qua 5 px thì hy vọng định vị tới 0,05 px là lạc quan. Hệ quả cho việc mua sắm: một ống kính tốt hơn thường là đồng tiền hiệu quả hơn một cảm biến nhiều megapixel hơn, vì cảm biến chỉ giúp khi quang học chưa phải nút thắt.

6.6 Vật liệu và bề mặt marker

Hiệu ứng cuối cùng, và nó là hiệu ứng duy nhất mà bạn kiểm soát hoàn toàn bằng lựa chọn vật liệu.

Phản xạ gương và loá. Một bề mặt bóng phản xạ nguồn sáng thành một vùng sáng chói. Nếu vùng ấy trùm lên một cạnh tag, cạnh bị “ăn” và góc lệch — một thiên lệch phụ thuộc góc chiếu sáng, tức phụ thuộc tư thế robot. Cách chữa: vật liệu nhám (matte), và tránh bố trí sao cho có đường phản xạ gương từ đèn tới camera.

Retroreflective. Vật liệu phản xạ ngược về nguồn. Với một đèn gắn cạnh camera, nó cho tương phản rất cao và gần như độc lập với ánh sáng môi trường — một lợi thế lớn vì nó làm σ ổn định. Nhược điểm: nó rất nhạy với góc, và ở góc chéo lớn thì hiệu quả giảm; ngoài ra vùng sáng có thể tràn (blooming) sang pixel lân cận, làm cạnh dịch.

Backlit và IR chủ động. Tag phát sáng từ phía sau, hoặc chiếu sáng bằng LED hồng ngoại cộng bộ lọc bandpass trên camera. Đây là cách mạnh nhất để làm σ ổn định, vì nó loại bỏ gần như hoàn toàn ảnh hưởng của ánh sáng môi trường. B/14 bàn kỹ; ở đây chỉ ghi nhận rằng nó tồn tại và rằng lợi ích chính của nó là *tính ổn định* chứ không phải độ chính xác đỉnh.

Độ phẳng và độ cứng của nền. Không phải quang học nhưng thuộc cùng nhóm quyết định vật liệu, và nó quan trọng hơn cả ba mục trên với ngân sách 5 mm. Một tờ giấy in dán lên bề mặt cong lệch khỏi mặt phẳng vài milimét, và A/02 mục 7 đã chỉ ra rằng khi ấy không có homography nào khớp được. Đây là một sai số hệ thống thuần túy cơ khí, và nó thường là dòng lớn nhất trong bảng ngân sách của C/15.

6.7 Tổng kết: nhiễu hay thiên lệch, và cái gì biến mất khi dừng

Bảng 6.1 là thứ đáng mang ra khỏi chương nhất.

Bảng 6.1. Sáu hiệu ứng: loại sai số, phụ thuộc vận tốc, và cách xử lý chính.

Hiệu ứng	Loại	Hết khi dừng?	Cách xử lý chính
Rolling shutter	Thiên lệch	Có	Global shutter; hoặc stop-and-go; hoặc mô hình hoá [35]
Motion blur	Thiên lệch	Có	Giảm t_e (cần nhiều sáng hơn); hoặc stop-and-go
Loá phụ thuộc góc	Thiên lệch	Một phần	Vật liệu nhám; bố trí tránh phản xạ gương
Nhoè do lấy nét	Thiên lệch	Không	Khoá nét ở khoảng cách cuối; khép khẩu vừa phải
Răng cưa khi tag nhỏ	Cả hai	Không	Ngưỡng kích thước tối thiểu; nested multi-scale (B/09)
Bias tâm ellipse	Thiên lệch	Không	Hiệu chỉnh conic [11]; hoặc dùng góc chessboard

Đọc cột thứ ba là đọc được câu trả lời cho Q4. Hai hiệu ứng lớn nhất khi robot chuyển động — rolling shutter và motion blur — đều biến mất hoàn toàn khi vận tốc bằng không, và loá thì giảm vì không còn đổi góc. Đây là lập luận vật lý cho stop-and-go, và nó mạnh hơn lập luận thông thường (“dừng lại để trung bình nhiều khung”): trung bình nhiều khung chỉ giảm *nhiều*, còn dừng lại thì xoá hẳn mấy dòng *thiên lệch* — thứ mà trung bình không bao giờ giảm được.

Hai hiệu ứng không biến mất khi dừng là nhoè do lấy nét và bias tâm ellipse. Cả hai đều xử lý được ở tầng thiết kế: khoá nét đúng khoảng cách, và dùng target có góc thay vì target tròn cho khâu đo tinh.

6.8 Giới hạn và chế độ hỏng

Chương này không đầy đủ. Danh sách sáu hiệu ứng là danh sách những thứ tôi cho là đáng kể với bài toán docking ở mức 5 mm; nó không phải danh sách đầy đủ các hiệu ứng vật lý trong một hệ thị giác. Không bàn tới ở đây: hiện tượng nở vệt sáng (blooming) trong trường hợp nặng, méo do khí quyển (không đáng kể ở cự ly mét), nhiễu đọc và nhiễu nhiệt của cảm biến (chúng vào σ và được xử lý như nhiễu ngẫu nhiên), và hiệu ứng của nén ảnh nếu camera trả về JPEG (tránh — dùng ảnh thô).

Các con số là bậc độ lớn. $t_{\text{line}} = 20 \mu\text{s}$, ngưỡng 3–5 px mỗi ô bit, $\lambda = 550 \text{ nm}$ cho ánh sáng trắng — đây là giá trị điển hình dùng để thấy bậc, không phải số đo của thiết bị cụ thể. Mọi con số milimét dẫn ra từ chúng thừa hưởng cùng mức độ tin cậy ấy.

Phân loại nhiễu/thiên lệch là đơn giản hoá. Một số hiệu ứng có cả hai thành phần. Răng cưa chẳng hạn: nó vừa làm tăng phương sai vừa gây thiên lệch phụ thuộc pha của lưới tag so với lưới pixel. Bảng 6.1 ghi “cả hai” cho nó, nhưng cách xử lý đúng cho từng thành phần khác nhau và chương này không tách chúng ra.

Mệnh đề về thiên lệch do blur chưa được kiểm. Đã ghi rõ ở mục 3. Nếu hoá ra blur chỉ gây nhiễu chứ không gây thiên lệch, thì tầm quan trọng của stop-and-go giảm đi một phần — không hết, vì rolling shutter vẫn còn.

6.9 Thực hành

Bốn việc, theo thứ tự chi phí tăng dần.

Một, làm phép thử đảo chiều. Đo cùng một điểm khi tiến và khi lùi. Nếu thấy lệch có hệ thống theo hai chiều ngược nhau thì bạn có hiệu ứng phụ thuộc vận tốc, và bạn biết ngay phải nhìn vào

rolling shutter và blur. Phép thử này gần như miễn phí và nó là phép thử thông tin nhất trong cả chương.

Hai, đo profile cạnh. Chụp một cạnh sắc, vẽ độ sáng dọc theo một đường vuông góc, đếm xem chuyển tiếp trải qua bao nhiêu pixel. Con số ấy cho bạn một ước lượng thực tế về trần của σ , và nó nói cho bạn biết có nên đầu tư vào subpixel tinh vi hơn không.

Ba, khoá mọi thứ khoá được. Khoá exposure, khoá gain, khoá white balance, khoá nét — rồi dán keo vào vòng lấy nét. Mọi tham số tự động là một nguồn thay đổi không kiểm soát được trong một hệ đo lường.

Bốn, chọn global shutter ngay từ đầu nếu có bất kỳ khả năng nào phải đo trong khi di chuyển. Đây là quyết định mua sắm chứ không phải quyết định kỹ thuật, và sửa nó về sau đắt hơn nhiều.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Nguyên tắc tổ chức của chương	Phân loại theo <i>nhiều hay thiên lệch</i> , vì nhiều chia cho $\sqrt{N}$ còn thiên lệch thì không (A/05 mục 8). Gần như mọi hiệu ứng vật lý thuộc loại thứ hai.
••• Rolling shutter làm gì	Các hàng ảnh ghi cảnh ở các thời điểm khác nhau, nên tag chuyển động bị <i>xé</i> chứ không chỉ mờ. Detector vẫn thấy tứ giác đẹp, decode vẫn được, residual vẫn nhỏ — pose sai im lặng.
••• Độ lớn rolling shutter	$960 \text{ hàng} \times t_{\text{line}} = 20 \mu\text{s} = 19,2 \text{ ms}$ độ trễ. Với $v = 0,3 \text{ m/s}$, $f/Z = 1200$: 6,9 px trên cả ảnh, 0,86 px trong nội bộ tag 120 hàng — gấp bốn lần σ , và là thiên lệch.
••• Chiều dài vệt motion blur	$L = v f t_e / Z$. Ví dụ: $0,3 \text{ m/s}$, $f = 600$, $Z = 0,5 \text{ m}$, $t_e = 20 \text{ ms} \Rightarrow 7,2 \text{ px}$ — lớn hơn σ ba mươi lần.
•• Vì sao blur gây thiên lệch chứ không chỉ mờ	Vì cạnh tag <i>không cô lập</i> : ô đen 5 px nhoè 7 px thì hai cạnh chồng lên nhau và profile mất đối xứng; và detector khớp mô hình lên profile gradient chứ không lấy trung tâm đối xứng. (<i>Tôi suy ra; chưa kiểm bằng mô phỏng.</i>)
••• Phép thử đảo chiều	Đo cùng một điểm khi tiến và khi lùi. Lệch có hệ thống theo hai chiều ngược nhau $\Rightarrow$ hiệu ứng phụ thuộc vận tốc. Không nguồn nào khác trong cây có chữ ký này — méo, constellation, hand-eye đều không quan tâm chiều chuyển động.
••• Bốn trong sáu hiệu ứng hết khi dừng	Rolling shutter, motion blur hết hoàn toàn; loá giảm. Đây là lập luận vật lý cho stop-and-go, và nó mạnh hơn lập luận “dừng để trung bình” — trung bình chỉ giảm nhiễu, dừng thì xoá thiên lệch.
•• Ngưỡng pixel mỗi ô bit	Nyquist cho 2 px; thực tế 3–5 px (<i>bậc độ lớn kinh nghiệm</i>). Với tag 8×8 ô thì w cỡ 24–40 px để <i>decode</i> . Ngưỡng để <i>đo chính xác</i> cao hơn nhiều.
•• Hệ quả thiết kế của ngưỡng ô bit	Chọn họ tag ít ô nhất có thể. Docking chỉ cần vài chục ID, nên đánh đổi gần như luôn nghiêng về phía ít ô — và lựa chọn mặc định (họ phổ biến nhất) thường không tối ưu.
••• Đường kính đĩa Airy	$d \approx 2,44\lambda N$ [37]. Với $\lambda = 550 \text{ nm}$, $N = 8$: $10,7 \mu\text{m}$ — hơn ba pixel $3 \mu\text{m}$. Ở khẩu ấy, không cảm biến nào phân giải nổi chỉ tiết một pixel.
•• Đánh đổi khẩu độ	Khép khẩu tăng độ sâu trường ảnh nhưng tăng N nên tăng đĩa Airy. Có khẩu tối ưu; khép quá thì nhoè hơn.
•• Cạnh hơi mờ có thể tốt hơn cạnh sắc	Vì cạnh trải qua vài pixel cho nhiều pixel mang thông tin gradient hơn, nên khớp đường chính xác hơn. Chỗ MTF thật sự hại là khi nó <i>không đồng đều</i> giữa các cạnh của cùng một tag.
•• Phân biệt nút thất quang học với nút thất nhiễu ảnh	Chụp lại với nhiễu sáng hơn và t_e dài hơn. σ cải thiện $\Rightarrow$ nhiễu ảnh là nút thất. Không cải thiện $\Rightarrow$ quang học là nút thất, và thêm megapixel vô ích.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Vật liệu marker	Nhám chống loá; retroreflective cho tương phản cao nhưng nhạy góc và dễ blooming; backlit/IR cho σ ổn định nhất (B/14). Và quan trọng hơn cả ba: nền phải cứng và phẳng.

• • • phải thuộc • • nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Mô hình rolling shutter [35]	Giải cả pose lẫn vận tốc từ một ảnh, biến phiên toái thành nguồn thông tin	Chỉ dùng khi không thể đổi camera hoặc dừng robot
Hiệu chuẩn t_{line} [36]	Đo độ trễ dòng bằng một target đã biết	Bước bắt buộc trước khi mô hình hoá rolling shutter
Hiệu chỉnh bias tâm [11]	Dùng đại số conic thay vì lấy tâm ellipse	Bắt buộc nếu dùng marker tròn — xem A/02 mục 6
Marker IR chủ động cộng bandpass deltille [8]	Loại ảnh hưởng ánh sáng môi trường, làm σ ổn định Target lưới tam giác; góc saddle không chịu bias kiểu ellipse	Lợi ích chính là <i>ổn định</i> , không phải độ chính xác đỉnh — B/14 Lựa chọn khi cần vừa chính xác vừa không thiên lệch

MINI PROJECT • Đo thiên lệch phụ thuộc vận tốc bằng phép thử đảo chiều • 1–2 ngày

Mục tiêu	tách bạch phần sai số phụ thuộc vận tốc khỏi mọi nguồn khác, bằng một phép thử mà không nguồn nào khác có thể giả mạo.
Dữ liệu	phần cứng thật: một camera, một tag gắn cố định, và một cách di chuyển camera (hoặc tag) với vận tốc kiểm soát được — một trục trượt, hoặc chính robot của bạn.
Các bước	chọn một điểm mốc trên đường đi; đo pose tại điểm ấy trong ba điều kiện: đứng yên (trung bình 50 khung), đang tiến qua nó với vận tốc v , đang lùi qua nó với cùng v ; lặp cho ba giá trị v khác nhau và cho hai giá trị t_e khác nhau; vẽ sai lệch so với giá trị đứng yên theo v , tách riêng cho tiến và lùi.
Xong khi	bạn có một đồ thị trong đó đường “tiến” và đường “lùi” đối xứng nhau qua trục hoành và cả hai đi qua gốc — chữ ký không thể nhầm của hiệu ứng phụ thuộc vận tốc. Độ dốc của chúng là hệ số bạn đưa vào ngân sách nếu buộc phải đo khi di chuyển. Kiểm tra chéo bắt buộc: lặp lại với t_e giảm một nửa; nếu độ dốc giảm khoảng một nửa thì thủ phạm chính là motion blur, nếu nó gần như không đổi thì thủ phạm là rolling shutter — hai cách chữa khác nhau.
Nối với	B/13 cho quyết định stop-and-go; C/15 nhận độ dốc này làm một dòng phụ thuộc vận tốc trong ngân sách.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Vệt dài $L = vft_e/Z = 0,3 \times 600 \times 0,02/0,5 = 7,2$ px — lớn hơn nhiễu định vị góc chừng ba mươi lần. Nó gây thiên lệch chứ không chỉ mờ vì hai lý do. Thứ nhất, các cạnh của tag không cô lập: một ô đen rộng 5 px bị nhoe 7 px thì hai cạnh của nó chồng lên nhau, và profile độ sáng thu được không còn đối xứng quanh vị trí cạnh thật. Thứ hai, detector không tìm cạnh bằng cách lấy trung tâm đối xứng mà bằng cách khớp một mô hình lên profile gradient có trọng số, nên một profile bị biến dạng bất đối xứng cho một đường khớp lệch. Hệ quả thực hành: lấy trung bình nhiều khung *không* chữa được, vì mọi khung đều lệch theo cùng hướng.
- A2.** Motion blur hoặc rolling shutter — cả hai đều phụ thuộc vận tốc và cả hai đều đổi dấu khi đổi chiều. Đây là một dấu hiệu chẩn đoán rất mạnh vì *không nguồn sai số nào khác trong cả cây có chữ ký ấy*: méo ảnh không quan tâm robot đang đi hướng nào, sai số constellation không quan tâm,

sai số hand-eye không quan tâm, ambiguity thì gây nhảy chứ không gây lệch có hướng. Vì vậy nếu bạn thấy chữ ký này thì bạn đã thu hẹp không gian nguyên nhân xuống còn hai, và có một phép thử thứ hai tách hai cái đó: giảm thời gian phơi sáng một nửa — nếu lệch giảm khoảng một nửa thì là blur, nếu gần như không đổi thì là rolling shutter.

- A3. Cơ chế thứ nhất: quang học đã là nút thắt. Đĩa Airy ở $N = 8$ có đường kính $10,7 \mu\text{m}$, trải qua hơn ba pixel $3 \mu\text{m}$, nên camera không phân giải nổi chi tiết một pixel bất kể cảm biến bao nhiêu megapixel — pixel nhỏ hơn chỉ lấy mẫu dày hơn một hình ảnh vốn đã nhòe. Cơ chế thứ hai: nhiễu ảnh. Pixel nhỏ hơn có diện tích thu photon nhỏ hơn, nên tỉ số tín hiệu trên nhiễu kém đi, nên σ tính bằng pixel tăng lên đúng bằng phần mà độ phân giải cải thiện. Phép thử phân biệt: chụp lại với nhiễu ánh sáng hơn và thời gian phơi sáng dài hơn. Nếu σ cải thiện thì nút thắt là nhiễu ảnh và bạn nên đầu tư vào chiếu sáng; nếu không cải thiện thì nút thắt là quang học và bạn nên đầu tư vào ống kính.
- A4. Vì hai hiệu ứng lớn nhất khi di chuyển — rolling shutter và motion blur — đều là *thiên lệch* và đều tỉ lệ với vận tốc, nên chúng biến mất hoàn toàn ở $v = 0$; loá phụ thuộc góc cũng ổn định hơn vì góc không đổi nữa. Lập luận này mạnh hơn lập luận thông thường “dừng lại để trung bình nhiều khung”, vì trung bình chỉ giảm *nhiều* theo $1/\sqrt{N}$ còn dừng lại thì xoá hẳn mấy dòng *thiên lệch* — thứ mà trung bình không bao giờ chạm tới được. Hai hiệu ứng không biến mất khi dừng là nhòe do lấy nét (xử lý bằng cách khoá nét ở khoảng cách cuối) và bias tâm ellipse (xử lý bằng hiệu chỉnh conic, hoặc bằng cách dùng góc chessboard thay cho marker tròn).
- A5. Vì khe khẩu tăng số f N , mà đường kính đĩa Airy tỉ lệ thuận với N : $d \approx 2,44\lambda N$. Khe từ $N = 4$ xuống $N = 16$ làm đĩa Airy to gấp bốn, từ khoảng $5,4 \mu\text{m}$ lên $21,5 \mu\text{m}$ — tức từ chừng hai pixel lên bảy pixel với pixel $3 \mu\text{m}$. Giới hạn vật lý đang chặn bạn là *nhiều xạ*: bản chất sóng của ánh sáng, không phải chất lượng chế tạo ống kính, nên không có ống kính đắt tiền nào vượt qua được. Kết quả là tồn tại một khẩu độ tối ưu cân bằng giữa độ sâu trường ảnh và độ sắc nét do nhiễu xạ, và với bài toán docking thì một lựa chọn thường tốt hơn là bỏ hẳn cuộc đua ấy: khoá nét ở khoảng cách cuối — nơi cần chính xác nhất — và chấp nhận tag hơi mờ ở xa, nơi chỉ cần điều khiển thô.

NÓI LẠI THẬT ĐƠN GIẢN

Mọi thứ cho tới chương này đều giả vờ rằng cái camera là một thiết bị hoàn hảo chụp một thế giới đứng yên. Chương này liệt kê những chỗ giả vờ ấy bị lộ.

Thứ đáng lo nhất không phải là ảnh trông xấu. Ảnh xấu thì ai cũng thấy và ai cũng biết phải sửa. Thứ đáng lo là những sai lệch làm con số lệch đi mà không làm ảnh trông khác gì — và tệ hơn nữa, lệch theo *cùng một hướng* ở mọi lần chụp.

Hãy nghĩ về chuyện bắn bia. Nếu các phát đạn tản mát quanh tâm thì bắn nhiều rồi lấy trung bình là xong. Nhưng nếu súng bị lệch ngắm thì bắn nghìn phát lấy trung bình chỉ cho bạn biết rất chính xác cái chỗ lệch ấy ở đâu — bạn tự tin hơn về một câu trả lời sai. Gần như mọi thứ trong chương này thuộc loại lệch ngắm.

Có hai thủ phạm lớn nhất, và cả hai chỉ xuất hiện khi robot đang đi. Thủ phạm thứ nhất là chuyện nhiều loại camera không chụp cả tấm ảnh cùng một lúc mà quét từ trên xuống dưới như máy scan — nên nếu vật đang chạy ngang thì hàng dưới ghi lại nó ở chỗ khác với hàng trên, và cái hình vuông biến thành hình xiên. Đáng sợ ở chỗ cái hình xiên ấy vẫn trông rất gọn gàng, máy vẫn đọc được, không có gì báo động, và pose thì sai. Thủ phạm thứ hai là vết nhòe: cửa chụp mở hai mươi mili giây trong khi mọi thứ đang trôi, nên cạnh của tag bị kéo thành một vết.

Và đây là tin tốt, tốt hơn nhiều so với công sức bỏ ra để sửa chúng: cả hai thủ phạm ấy *biến mất* nếu robot đứng yên. Không phải giảm đi, mà biến mất. Cho nên cách chữa rẻ nhất trong cả chương là: đi tới gần, dừng hẳn lại một phần tư giây, chụp mấy chục tấm, tính toán, rồi mới đi tiếp. Không cần camera đắt hơn, không cần thuật toán mới.

Hai thứ còn lại không chữa được bằng cách dừng. Một là chuyện ống kính chỉ nét ở một khoảng cách — cái này chữa bằng cách chỉnh nét đúng chỗ mà bạn cần chính xác nhất, tức chỗ sắp cập vào, rồi dán keo lại đừng cho ai vặn. Hai là chuyện cái tâm của hình tròn không nằm ở chỗ bạn tưởng — cái này chữa bằng cách dùng loại marker có góc thay vì loại có vòng tròn.

Còn một chuyện cuối, và nó là chuyện vật lý chứ không phải kỹ thuật. Ánh sáng có bản chất sóng, và vì thế không có ống kính nào — kể cả ống kính hoàn hảo, kể cả ống kính đắt nhất — hội tụ được

ánh sáng vào một điểm. Điểm nhỏ nhất có thể là một cái chấm nhỏ, và kích thước của nó thì tính được. Nếu cái chấm ấy đã rộng ba pixel thì mua cảm biến nhiều pixel hơn không giúp gì cả — bạn chỉ đang lấy mẫu dày hơn một cái chấm.

Chỗ không rút gọn được: việc tách bạch xem một sai lệch cụ thể là nhiễu hay lệch ngấm thì không nhìn ra được từ một lần đo, và đôi khi một hiệu ứng có cả hai phần. Cách duy nhất là đổi một điều kiện có chủ ý — đổi chiều đi, đổi thời gian chụp, đổi ánh sáng — rồi xem cái gì đổi theo.

Một câu: *Gần như mọi thứ trong chương này làm sủng lệch ngấm chứ không làm tay run — và hai cái lệch lớn nhất thì biến mất nếu bạn chịu đứng yên một phần tư giây.*

6.10 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *rolling shutter và motion blur gây thiên lệch phụ thuộc vận tốc, đổi dấu khi đổi chiều* — bị bác bỏ nếu phép thử đảo chiều ở mini project cho thấy hai đường tiến và lùi trùng nhau thay vì đối xứng. Với rolling shutter thì mệnh đề này gần như chắc chắn đúng vì nó là hệ quả trực tiếp của cơ chế đọc hàng; với blur thì nó là suy luận của tôi và chưa được kiểm.

Mệnh đề thứ hai — *blur gây thiên lệch chứ không chỉ tăng phương sai* — là mệnh đề yếu nhất về kiểm chứng trong chương, như đã ghi ở mục 3. Nó bị bác bỏ nếu mô phỏng (tích phân ảnh tag qua một đoạn dịch chuyển, chạy detector, so với chân trị) cho thấy sai lệch trung bình bằng không trong phạm vi sai số thống kê. Nếu vậy thì stop-and-go vẫn đáng làm vì rolling shutter, nhưng lập luận mất một chân.

Mệnh đề thứ ba — *quang học đặt trần cứng lên σ mà phần mềm không vượt được* — là mệnh đề chắc chắn nhất vì nó là vật lý cơ bản [37]. Nó không bị bác bỏ bởi một thuật toán tốt hơn; nó chỉ bị “bác bỏ” nếu ai đó chỉ ra rằng bạn tính sai số f hoặc bước sóng.

Cuối cùng, một mệnh đề mềm: chương này khẳng định rằng *trong một hệ đã tối ưu tốt phần nhiều, ngân sách bị chi phối bởi các dòng trong chương này*. Điều đó bị bác bỏ nếu trong một hệ thực, sau khi đã chuyển sang stop-and-go và global shutter, sai số vẫn không cải thiện đáng kể — khi ấy nút thắt nằm ở chỗ khác, nhiều khả năng là sai số cơ khí của constellation (B/12) hoặc hand-eye. (Tôi phát biểu dựa trên cấu trúc của ngân sách chứ không dựa trên thống kê hệ thực tế.)

6.11 Kết nối

Chương này khép Phần A bằng cách nói ra những gì năm chương trước im lặng bỏ qua, và nó mở đường sang những quyết định thiết kế của Phần B.

Nói lên trên. [A/01-mo-hinh-camera-va-distortion](#) dựng mô hình hình học; chương này liệt kê những thứ mô hình ấy không chứa. Đáng chú ý là hai chương bổ sung nhau theo một cách sạch: chương 1 nói về sai lệch *tĩnh* của quang học (đo một lần, bù mãi mãi), chương này nói về sai lệch *động* (đổi theo vận tốc, khoảng cách, ánh sáng). [A/02-homography-va-hinh-hoc-phang](#) mục 6 cấp bias tâm ellipse, một trong sáu hiệu ứng ở đây. [A/05-lan-truyen-sai-so-pixel-sang-pose](#) mục 8 cấp khung phân loại nhiễu/thiên lệch mà cả chương này dựa vào, và nó cũng nhận lại từ đây một cảnh báo quan trọng: công thức hiệp phương sai của chương 5 hoàn toàn mù với thiên lệch, nên một hệ chịu các hiệu ứng ở đây sẽ báo cáo một bất định nhỏ hơn sự thật.

Nói xuống Phần B. [B/08-corner-refinement](#) nhận từ mục 5 luận điểm rằng cạnh hơi mờ có thể tốt hơn cạnh sắc, và nhận từ mục 3 cảnh báo rằng mọi kỹ thuật subpixel tinh vi đều vô ích nếu ảnh đang bị nhoè do chuyển động. [B/13-docking-control](#) nhận mục 7 làm cơ sở vật lý cho stop-and-go — đây là liên kết quan trọng nhất của chương, vì nó biến một danh sách hiệu ứng thành

một quyết định kiến trúc. B/14-dieu-kien-quan-sat-va-anh-sang nhận toàn bộ mục 6 và mở rộng nó thành hướng dẫn về chiếu sáng, vật liệu, và cấu hình camera.

Nổi sang Phần C. C/15-ngan-sach-sai-so có một nhóm dòng riêng cho các hiệu ứng của chương này, và chúng có một đặc điểm làm chúng khó chịu trong bảng: phần lớn *không phải hằng số* mà phụ thuộc điều kiện vận hành. Cách xử lý đúng trong ngân sách là ghi chúng theo hai cột — giá trị khi đang di chuyển và giá trị khi dừng — và chính sự chênh lệch giữa hai cột ấy là lập luận định lượng cho stop-and-go. C/16-danh-gia-va-nghiem-thu nhận phép thử đảo chiều ở mục 9 làm một mục bắt buộc trong quy trình nghiệm thu.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao rolling shutter gây một chế độ hỏng “im lặng” trong khi motion blur thì ít nhất còn nhìn thấy được trên ảnh?
2. Nêu chữ ký chẩn đoán phân biệt hiệu ứng phụ thuộc vận tốc với mọi nguồn sai số khác trong cây, và giải thích vì sao không nguồn nào khác giả mạo được nó.
3. Bạn đã có chữ ký ấy. Nêu phép thử thứ hai tách motion blur khỏi rolling shutter, và dự đoán kết quả cho từng trường hợp.
4. Vì sao một cạnh trải qua ba pixel có thể cho định vị subpixel tốt hơn một cạnh nhảy bậc trong một pixel — và khi nào thì MTF thật sự gây hại?
5. Với $\lambda = 550 \text{ nm}$ và pixel $3 \mu\text{m}$, ở số f nào thì đĩa Airy bắt đầu vượt hai pixel? Tính nhẩm, rồi nói kết luận thực hành.
6. Bốn trong sáu hiệu ứng biến mất khi dừng. Nêu hai cái còn lại, và với mỗi cái nêu cách xử lý ở tầng thiết kế chứ không ở tầng thuật toán.
7. Một hệ đã dùng global shutter, stop-and-go, và trung bình 50 khung. Sai số vẫn 4 mm. Chương này còn giải thích được bao nhiêu phần, và bạn chuyển sang nghi ngờ chương nào?

Nguồn

- [1] D. C. Brown, "Close-Range Camera Calibration," *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, "AprilTag 2: Efficient and Robust Fiducial Detection," in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, "The Double Sphere Camera Model," in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, "Accurate Calibration of CCD-Cameras," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, "Unified Temporal and Spatial Calibration for Multi-Sensor Systems," in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., "Deltile Grids for Geometric Camera Calibration," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, "Geometric Camera Calibration Using Circular Control Points," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, "Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods," *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, "Infinitesimal Plane-Based Pose Estimation," *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, "TagSLAM: Robust SLAM with Fiducial Markers," in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, "Propagating Covariance in Computer Vision," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, "Statistical Optimization for Geometric Computation: Theory and Practice," 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, "Means and Averaging in the Group of Rotations," *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., "Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera," in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., "Rolling Shutter Camera Calibration," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, "Introduction to Fourier Optics," 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.

Nguồn

- [1] D. C. Brown, “Close-Range Camera Calibration,” *Photogrammetric Engineering*, vol. 37, no. 8, pp. 855–866, 1971, mức tin A; nguồn gốc của mô hình méo radial-tangential (Brown–Conrady).
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [4] J. Kannala and S. S. Brandt, “A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin A; mô hình KB cho lens góc rộng, vol. 28, 2006, pp. 1335–1340.
- [5] V. Usenko, N. Demmel, and D. Cremers, “The Double Sphere Camera Model,” in *International Conference on 3D Vision (3DV)*, mức tin B, 2018.
- [6] H. A. Beyer, “Accurate Calibration of CCD-Cameras,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1992, mức tin B; sai số có cấu trúc trong residual hiệu chuẩn.
- [7] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RIS International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [8] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004, mức tin A; nguồn chuẩn cho homography, DLT, phân rã pose, conic.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điểm bằng tối ưu khoảng cách Hamming.
- [11] J. Heikkilä, “Geometric Camera Calibration Using Circular Control Points,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1066–1077, 2000, mức tin A; nguồn chuẩn cho bias tâm ellipse và cách hiệu chỉnh bằng conic.
- [12] J. Mallon and P. F. Whelan, “Which Pattern? Biasing Aspects of Planar Calibration Patterns and Detection Methods,” *Pattern Recognition Letters*, vol. 28, no. 8, pp. 921–930, 2007, mức tin B; so sánh bias giữa chessboard và target tròn — chỗ dựa cho luận điểm chessboard ít bias hơn.
- [13] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014, mức tin A; IPPE — hai nghiệm cho target phẳng, là nền của SOLVEPNP_IPPE trong OpenCV.

- [14] L. Calvet, P. Gurdjos, C. Griwodz, et al., "Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [15] T. Krajník, M. Nitsche, J. Faigl, et al., "A Practical Multirobot Localization System," *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [16] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [17] R. M. Haralick, C.-N. Lee, K. Ottenberg, et al., "Review and Analysis of Solutions of the Three Point Perspective Pose Estimation Problem," *International Journal of Computer Vision*, vol. 13, no. 3, pp. 331–356, 1994, mức tin A; lịch sử P3P từ Grunert 1841, và phân tích số nghiệm.
- [18] L. Kneip, D. Scaramuzza, and R. Siegwart, "A Novel Parametrization of the Perspective-Three-Point Problem for a Direct Computation of Absolute Camera Position and Orientation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; P3P dạng đóng, bốn nghiệm, 2011.
- [19] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009, mức tin A.
- [20] G. Terzakis and M. Lourakis, "A Consistently Fast and Globally Optimal Solution to the Perspective-n-Point Problem," in *European Conference on Computer Vision (ECCV)*, mức tin B; SQPNP, có trong OpenCV, 2020.
- [21] J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," *arXiv:1812.01537*, 2018, mức tin B; tiền án nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [22] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., "Bundle Adjustment — A Modern Synthesis," in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge freedom và hiệp phương sai, Springer, 2000, pp. 298–372.
- [23] S. Urban, J. Leitloff, and S. Hinz, "MLPnP — A Real-Time Maximum Likelihood Solution to the Perspective-n-Point Problem," in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, mức tin B; PnP có trọng số theo hiệp phương sai từng điểm, 2016.
- [24] S. Agarwal, K. Mierle, et al., "Ceres Solver," <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [25] D. Oberkampf, D. F. DeMenthon, and L. S. Davis, "Iterative Pose Estimation Using Coplanar Feature Points," *Computer Vision and Image Understanding*, vol. 63, no. 3, pp. 495–511, 1996, mức tin A; hai nghiệm cho điểm đồng phẳng, trước Schweighofer.
- [26] G. Schweighofer and A. Pinz, "Robust Pose Estimation from a Planar Target," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 12, pp. 2024–2030, 2006, mức tin A; phân tích hai cực tiểu của hàm mục tiêu với target phẳng — nguồn gốc của pose ambiguity.
- [27] B. Benligray, C. Topal, and C. Akinlar, "STag: A Stable Fiducial Marker System," in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.

- [28] B. Pfrommer and K. Daniilidis, "TagSLAM: Robust SLAM with Fiducial Markers," in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [29] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993, mức tin A; cận Cramér–Rao và ma trận thông tin Fisher.
- [30] JCGM, *Evaluation of Measurement Data — Guide to the Expression of Uncertainty in Measurement (GUM)*. JCGM 100:2008, 2008, mức tin A; chuẩn quốc tế cho ngân sách sai số và phân biệt bias với độ lặp lại.
- [31] R. M. Haralick, "Propagating Covariance in Computer Vision," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 10, no. 5, pp. 561–572, 1996, mức tin B; công thức Jacobian chuẩn cho lan truyền hiệp phương sai qua bài toán bình phương tối thiểu.
- [32] K. Kanatani, "Statistical Optimization for Geometric Computation: Theory and Practice," 1996, mức tin A; lý thuyết sai số hình học, cận lý thuyết cho ước lượng từ điểm ảnh.
- [33] M. Moakher, "Means and Averaging in the Group of Rotations," *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [34] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.
- [35] O. Ait-Aider, N. Andreff, J. M. Lavest, et al., "Simultaneous Object Pose and Velocity Computation Using a Single View from a Rolling Shutter Camera," in *European Conference on Computer Vision (ECCV)*, mức tin A; mô hình rolling shutter cho ước lượng pose, và cách biến nó thành thông tin thay vì nhiễu, 2006.
- [36] L. Oth, P. Furgale, L. Kneip, et al., "Rolling Shutter Camera Calibration," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; đo line delay bằng target đã biết, 2013.
- [37] J. W. Goodman, "Introduction to Fourier Optics," 2005, mức tin A; cơ sở của MTF và giới hạn nhiễu xạ.